

Reproducibility Guide

Neural Collapse & Relative Flatness in Deep Learning Generalization
github.com/jcalataiut-cli/CENIA

Abstract

This document provides a complete, self-contained reproducibility guide for the experiments from the NeurIPS 2025 paper “*Flatness is Necessary, Neural Collapse is Not: Rethinking Generalization via Grokking*” (Han et al., 2025). It includes all source code, environment setup instructions, execution commands, and figure generation procedures for all six experiment families: (1) grokking with neural collapse and flatness metrics, (2) neural collapse clustering on CIFAR-10, (3) relative flatness on ResNet-18, (4) Vision Transformer, (5) BERT, and (6) GPT-2.

Contents

Chapter 1

Reproducibility Overview

1.1 Paper Summary

The paper empirically tests two central claims:

1. **Neural Collapse is not necessary** for generalization — generalization can occur without the collapse of class means to the vertices of a simplex equiangular tight frame (ETF).
2. **Relative Flatness is potentially necessary** for generalization — the phase transition from memorization to generalization in grokking coincides with a sharp drop in the sharpness measure.

The experiments use a **regulariser cancellation** protocol: models are first trained with a regulariser that suppresses either NC or flatness, then at a prescribed epoch the regulariser is removed, weight decay is activated, and the learning rate is bumped up with cosine annealing.

1.2 Repository Structure

The code is located at `NeuralCollapse_vs_Flatness/` within the CENIA repository:

Directory/File	Purpose
<code>requirements.txt</code>	Python dependencies
<code>report_implementation.tex/pdf</code>	Implementation report
<code>analysis_ncc_representations.tex/pdf</code>	NC analysis
<code>relative_flatness_plot_results.ipynb</code>	Notebook for paper figures
<code>grokking_nc_rf/grok/</code>	Core shared library
<code>data.py</code>	Algorithmic dataset generation (modular arithmetic)
<code>transformer.py</code>	Decoder-only transformer with noise injection
<code>training.py</code>	LightningModule, CustomAdamW, SAM, sharpness
<code>metrics.py</code>	Norm-based measures & generalization bounds
<code>measure.py</code>	Keskar sharpness (L-BFGS-B)
<code>visualization.py</code>	Multi-panel plotting utilities
<code>grokking_nc_rf/scripts/</code>	Entry-point scripts
<code>nc_experiments/</code>	Neural Collapse on CIFAR-10
<code>resnet18.py</code>	ResNet-18 from scratch
<code>train.py</code>	NC experiment entry point
<code>training_utils.py</code>	NC metric & training loop

<code>run_script.sh</code>	Experiment launcher
<code>rf_resnet18/</code>	Flatness on ResNet-18 & variants
<code>rf_vit/</code>	Flatness on ViT (ImageNet-100)
<code>rf_bert/</code>	Flatness on BERT (SST-5)
<code>rf_gpt2/</code>	Flatness on GPT-2 (SST-5)

Chapter 2

Environment Setup

2.1 System Requirements

- Python 3.8+ (tested with 3.8–3.11)
- CUDA-capable GPU (recommended, 8+ GB VRAM)
- 16+ GB RAM
- Linux / macOS / WSL2

2.2 Dependencies

The `requirements.txt` file:

```
1 torch==2.4.1
2 torchvision==0.19.1
3 timm==1.0.15
4 matplotlib
5 scikit-learn==1.3.2
6 numpy
7 tqdm
8 python==3.8
9 transformers
10 datasets
```

2.3 Installation Steps

1. Clone the repository:

```
1 git clone https://github.com/jcalataiut-cli/CENIA.git
2 cd CENIA/NeuralCollapse_vs_Flatness
```

2. Create and activate a virtual environment:

```
1 python3 -m venv venv
2 source venv/bin/activate
```

3. Install PyTorch 2.4.1 and torchvision 0.19.1:

```
1 pip install torch==2.4.1 torchvision==0.19.1 --index-url https://download.pytorch.
  org/whl/cu121
```

4. Install remaining dependencies:

```
1 pip install -r requirements.txt
2 pip install pytorch-lightning==1.2.10 scipy sympy blobfile mod
```

5. Install the grokking library in editable mode:

```
1 cd grokking_nc_rf && pip install -e . && cd ..
```

2.4 GPU Configuration

All experiments auto-detect CUDA. To force CPU:

```
1 export CUDA_VISIBLE_DEVICES=""
```

For multi-GPU, set:

```
1 export CUDA_VISIBLE_DEVICES=0,1
```

Chapter 3

Data Generation (Grokking Experiments)

Generate the token and dataset files for modular arithmetic:

```
1 cd grokking_nc_rf
2 python scripts/make_data.py
3 cd ..
```

This creates the `data/` directory with `tokens.txt` and equation files for all operators (+, −, *, /, polynomial forms over $\mathbb{Z}_{97}$, S_5 , etc.).

The data module (`grok/data.py`) implements:

- `ArithmeticTokenizer`: maps token strings to integer IDs
- `ArithmeticDataset`: training/validation splits with configurable `train_data_pct` and `actual_train_data_pct`
- `ArithmeticIterator`: batched PyTorch `IterableDataset`

Chapter 4

Complete Code Listings

Each section below contains the full source code for every file in the project, organized by experiment directory.

4.1 Shared Grokking Library: `grokking_nc_rf/grok/`

4.1.1 `__init__.py`

```
1 from . import transformer
2 from . import data
3 from . import training
4 from . import metrics
5 from . import visualization
```

4.1.2 `data.py` — Dataset Generation

```
1 import itertools
2 import math
3 import os
4 import sys
5 import random
6
7 import torch
8 from torch import Tensor, LongTensor
9 import numpy as np
10 from typing import Tuple, List, Dict, Any, Union, Optional
11 from tqdm import tqdm
12
13 from sympy.combinatorics.permutations import Permutation
14 from mod import Mod
15
16 import blobfile as bf
17
18
19 VALID_OPERATORS = {
20     "+": "addition",
21     "-": "subtraction",
22     "*": "multiplication",
23     "/": "division",
24     "**2+": "squarepoly",
25     "**3+": "cubepoly",
```

```

26     "x**2+y**2_mod_97": "quad1",
27     "x**2+y**2+x*y_mod_97": "quad2",
28     "x**2+y**2+x*y+x_mod_97": "quad3",
29     "x**3+x*y_mod_97": "cube1",
30     "x**3+x*y**2+y_mod_97": "cube2",
31     "(x._value//y)if(y._value%2==1)else(x-y)_mod_97": "mix1",
32     "s5": "s5",
33     "s5conj": "s5conj",
34     "s5aba": "s5aba",
35     "+*": "even-addition_odd-multiplication",
36     "+-": "even-addition_odd-subtraction",
37     "sort": "sort",
38     "reverse": "reverse",
39     "copy": "copy",
40 }
41 EOS_TOKEN = "<|eos|>"
42 EQ_TOKEN = "="
43 MODULUS = 97
44 NUMS = list(range(MODULUS))
45
46 DEFAULT_DATA_DIR = "data"
47
48
49 def render(operand, join_str=""):
50     if (
51         isinstance(operand, list)
52         or isinstance(operand, tuple)
53         or isinstance(operand, np.ndarray)
54     ):
55         return join_str.join(map(render, operand))
56     elif isinstance(operand, Permutation):
57         return "".join(map(str, operand.array_form))
58     elif isinstance(operand, Mod):
59         return str(operand._value)
60     else:
61         return str(operand)
62
63
64 def create_data_files(data_dir: str = DEFAULT_DATA_DIR):
65     ArithmeticTokenizer.create_token_file(data_dir)
66     ArithmeticDataset.create_dataset_files(data_dir)
67
68
69 class ArithmeticTokenizer:
70     """Stores the list of token text to token id mappings and converts between them"""
71
72     token_file = "tokens.txt"
73
74     def __init__(self, data_dir=DEFAULT_DATA_DIR) -> None:
75         self.token_file = bf.join(data_dir, self.token_file)
76
77         self.itos = self.get_tokens()
78
79         self.stoi: Dict[str, int] = dict([(s, i) for i, s in enumerate(self.itos)])
80
81     def _encode(self, s: str) -> Tensor:
82         return LongTensor([self.stoi[t] for t in s.split("_")])
83

```

```

84 def encode(self, obj: Union[str, List]) -> Tensor:
85     """
86     Convert a string of text into a rank-1 tensor of token ids
87     or convert a list of strings of text into a rank-2 tensor of token ids
88
89     :param obj: the string or list of strings to convert
90     :returns: a tensor of the token ids
91     """
92     if isinstance(obj, str):
93         return self._encode(obj)
94     elif isinstance(obj, list):
95         return torch.stack([self._encode(s) for s in obj], dim=0)
96     else:
97         raise NotImplementedError
98
99 def decode(self, tensor: Tensor, with_brackets: bool = False) -> str:
100     """
101     Convert a tensor of token ids into a string of text
102
103     :param tensor: a tensor of the token ids
104     :param with_brackets: if true, the returned string will include <> brackets
105                           around the text corresponding to each token.
106     :returns: string of these tokens.
107     """
108     indices = tensor.long()
109     if with_brackets:
110         l = "<"
111         r = ">"
112     else:
113         l = ""
114         r = ""
115     tokens = [l + self.itos[i] + r for i in indices]
116     return " ".join(tokens)
117
118 def __len__(self) -> int:
119     """
120     :returns: the number of tokens in this vocabulary
121     """
122     return len(self.itos)
123
124 @classmethod
125 def get_tokens(cls):
126     tokens = (
127         [EOS_TOKEN, EQ_TOKEN]
128         + list(sorted(list(VALID_OPERATORS.keys()))))
129         + list(map(render, NUMS))
130         + list(map(render, itertools.permutations(range(5)))) # s5
131     )
132     return tokens
133
134
135 class ArithmeticDataset:
136     """A Dataset of arithmetic equations"""
137
138     @classmethod
139     def splits(
140         cls,
141         train_pct: float,

```

```

142     actual_train_pct: float,
143     operator: str,
144     operand_length: Optional[int] = None,
145     data_dir: str = DEFAULT_DATA_DIR,
146 ):
147     """
148     Creates training and validation datasets
149
150     :param train_pct: percentage of total equations used for training data
151     :param operator: The arithmetic operator for this dataset e.g. '+', '-', '*',
152                     '/', 'sort'
153     :param operand_length: for list based datasets the length of the lists
154     :returns: (train_dataset, validation_dataset)
155     """
156
157     assert (0 < train_pct) and (train_pct < 100)
158
159     ds_name = cls.get_dsname(operator, operand_length)
160     eqs = cls.make_data(operator, operand_length)
161
162     train_rows, actual_train_rows, _ = cls.calc_split_len(train_pct, actual_train_pct
163                                                         , len(eqs)) # return 3 paramters here
164
165     train_ds = cls(ds_name, eqs[:actual_train_rows], train=True, data_dir=data_dir)
166     val_ds = cls(ds_name, eqs[actual_train_rows:], train=False, data_dir=data_dir)
167
168     return train_ds, val_ds
169
170 @classmethod
171 def calc_split_len(cls, train_pct, actual_train_pct, ds_len): # modify here to pass
172     into the argument.
173     train_rows = round(ds_len * (train_pct / 100.0))
174     val_rows = ds_len - train_rows
175     actual_train_rows = round(ds_len*(actual_train_pct / 100.0))
176     return train_rows, actual_train_rows, val_rows
177
178 def __init__(self, name, data: Union[Tensor, List[str]], train, data_dir) -> None:
179     """
180     :param data: A list of equations strings. Each equation must have an '=' in it.
181     """
182     self.tokenizer = ArithmeticTokenizer(data_dir)
183     self.name = name
184     self.train = train
185     if isinstance(data, list):
186         self.data = self.tokenizer.encode(data)
187     else:
188         self.data = data
189
190 def __len__(self) -> int:
191     """
192     :returns: total number of equations in this dataset
193     """
194     return self.data.shape[0]
195
196 # @classmethod
197 # def _render(cls, operand):
198 #     return render(operand, join_str=" ")
199 #

```

```

197 # @classmethod
198 # def _render_eq(parts):
199 #     return " ".join(map(render, parts))
200
201 @classmethod
202 def _make_binary_operation_data(cls, operator: str, operands=None) -> List[str]:
203     if operator == "s5":
204         operands = operands or list(range(5))
205         elems = map(np.array, itertools.permutations(operands))
206         tuples = itertools.product(elems, repeat=2)
207     elif operator in ["s5conj", "s5aba"]:
208         operands = operands or list(range(5))
209         elems = map(Permutation, itertools.permutations(operands))
210         tuples = itertools.product(elems, repeat=2)
211     elif "_mod_" in operator:
212         modulo = int(operator.split("_mod_")[-1])
213         elems = [Mod(i, modulo) for i in range(modulo)]
214         tuples = itertools.product(elems, repeat=2)
215     else:
216         operands = operands or NUMS
217         tuples = itertools.product(operands, repeat=2)
218
219     # if operator == "s5":
220     #     print("elems", list(elems))
221     #     print("tuples", list(tuples))
222     eqs = []
223     for a, b in tuples:
224         if operator == "/":
225             if b == 0:
226                 continue
227             else:
228                 c = a
229                 a = (b * c) % MODULUS
230         elif operator == "s5":
231             c = b[a]
232         elif operator == "s5conj":
233             c = a * b * (a.__invert__())
234         elif operator == "s5aba":
235             c = a * b * a
236         elif operator == "+*":
237             if a % 2 == 0:
238                 c = (a + b) % MODULUS
239             else:
240                 c = (a * b) % MODULUS
241         elif operator == "+-":
242             if a % 2 == 0:
243                 c = (a + b) % MODULUS
244             else:
245                 c = (a - b) % MODULUS
246         elif "_mod_" in operator:
247             expression = operator.split("_mod_")[0]
248             function = eval(f"lambda x, y: {expression}")
249             c = function(a, b)
250         else:
251             c = eval(f"({a}{operator}{b})%{MODULUS}")
252         eq = " ".join(map(render, [a, operator, b, "=", c]))
253         eqs.append(eq)
254

```

```

255     # if operator == "s5":
256     #     print("eqs", eqs)
257     return eqs
258
259     # @staticmethod
260     # def _render_unop_example(operator, lhs, rhs):
261     #     return " ".join([operator, render(lhs), "=", render(rhs)])
262
263     @staticmethod
264     def _make_unary_operation_data(operator: str, operands: Tensor) -> List[str]:
265         """
266         :param operator: The unary operator to apply to each operand e.g. '+'
267         :param operands: A tensor of operands
268         :returns: list of equations"""
269         num_examples = len(operands)
270
271         if operator == "sort":
272             rhs = torch.sort(operands, dim=1)[0]
273         elif operator == "reverse":
274             rhs = torch.flip(operands, dims=(1,))
275         elif operator == "copy":
276             rhs = operands
277         else:
278             raise Exception("unsupported_operator")
279
280         def func(L, R):
281             L = map(str, L)
282             R = map(str, R)
283             return f"{operator}_{' '.join(L)}_{' '.join(R)}"
284
285         if num_examples < 1000000000:
286             eqs = [
287                 func(L, R)
288                 for L, R in tqdm(
289                     zip(operands.tolist(), rhs.tolist()), total=num_examples
290                 )
291             ]
292         else:
293             with ProcessPoolExecutor() as executor:
294                 eqs = executor.map(func, tqdm(zip(operands, rhs), total=num_examples))
295
296         return eqs
297
298     # @staticmethod
299     # def _make_s5_data(abstract=False) -> List[str]:
300     #     elems = itertools.permutations([0, 1, 2, 3, 4])
301     #     pairs = itertools.product(elems, repeat=2)
302     #     eqs = []
303     #     for a, b in pairs:
304     #         a = np.array(a)
305     #         b = np.array(b)
306     #         c = b[a]
307     #         eq = " ".join(map(render, (a, "s5", b, "=", c)))
308     #         eq = cls._render_eq([a, , b, "=", c])
309     #         eqs.append(eq)
310     #
311     #     return eqs
312

```

```

313 @classmethod
314 def get_dsname(cls, operator, operand_length) -> str:
315     operator, noise_level = cls._get_operator_and_noise_level(operator)
316     ds_name = VALID_OPERATORS[operator]
317     if operand_length is not None:
318         ds_name += f"_length-{operand_length}"
319     if noise_level > 0:
320         ds_name += f"_noise-{noise_level}"
321     return ds_name
322
323 @classmethod
324 def get_file_path(cls, operator, operand_length=None, data_dir=DEFAULT_DATA_DIR):
325     ds_name = cls.get_dsname(operator, operand_length)
326     ds_file = bf.join(data_dir, f"{ds_name}_data.txt")
327     return ds_file, ds_name
328
329 @classmethod
330 def _get_operator_and_noise_level(cls, operator):
331     if "_noisy" in operator:
332         operator, noise_level = operator.split("_noisy_")
333         return operator, int(noise_level)
334     else:
335         return operator, 0
336
337 @classmethod
338 def make_data(cls, operator, operands=None, shuffle=True, seed=0) -> List[str]:
339     operator, noise_level = cls._get_operator_and_noise_level(operator)
340     assert operator in VALID_OPERATORS
341
342     if operator not in ["sort", "reverse", "copy"]:
343         data = cls._make_binary_operation_data(operator)
344     else:
345         data = cls._make_unary_operation_data(operator, operands)
346
347     rng = np.random.RandomState(seed=seed)
348     if shuffle:
349         rng.shuffle(data)
350
351     if noise_level > 0:
352         random_answer_eqns = rng.choice(data, size=noise_level)
353         random_answers = [
354             random_eq.split("=_")[1] for random_eq in random_answer_eqns
355         ]
356         for i in range(noise_level):
357             data[i] = data[i].split("=_")[0] + "=_ " + random_answers[i]
358
359     data = [EOS_TOKEN + " " + eq + " " + EOS_TOKEN for eq in data]
360
361     return data
362
363 # @classmethod
364 # def create_data_file(
365 #     cls, operator, operand_length=None, shuffle=True, data_dir=DEFAULT_DATA_DIR
366 # ):
367 #     if VALID_OPERATORS[operator]["binary_eval"]:
368 #         cls.write_dataset(
369 #             cls.make_binary_operation_data(operator), paths["ds_file"]
370 #         )

```

```

371 #
372 #     pass
373
374 # @classmethod
375 # def write_dataset(eqs: List[str], ds_file: str):
376 #     print(f"-> writing {ds_file}", flush=True)
377 #     with open(ds_file, "w") as fh:
378 #         fh.writelines([EOS_TOKEN + " " + eq + " " + EOS_TOKEN + "\n" for eq in eqs])
379
380 @classmethod
381 def _make_lists(cls, sizes=[2, 3], nums=NUMS):
382     lists: dict = {}
383     for size in sizes:
384         lists[size] = torch.tensor(
385             list(itertools.permutations(nums, r=size)),
386             dtype=torch.int,
387         )
388     return lists
389
390
391 class ArithmeticIterator(torch.utils.data.IterableDataset):
392     """
393     An iterator over batches of data in an ArithmeticDataset
394     """
395
396     def __init__(
397         self,
398         dataset: ArithmeticDataset,
399         device: torch.device,
400         batchsize_hint: float = 0,
401         shuffle: bool = True,
402     ) -> None:
403         """
404         :param dataset: the dataset to iterate over
405         :param device: the torch device to send batches to
406         :param batchsize_hint: * 0 means we use a default batchsize
407                               * -1 means the entire dataset
408                               * float between 0 and 1 means each batch is
409                                 that fraction of the DS
410                               * int > 1 means that specific batch size
411         :param shuffle: whether or not to randomly shuffle the dataset
412         """
413         self.dataset = dataset
414         self.batchsize = self.calculate_batchsize(
415             len(dataset), batchsize_hint=batchsize_hint
416         )
417         self.device = device
418         self.reset_iteration(shuffle=shuffle)
419
420     @staticmethod
421     def calculate_batchsize(ds_size: int, batchsize_hint: int = 0) -> int:
422         """
423         Calculates which batch size to use
424
425         :param ds_size: the number of equations in the dataset
426         :param batchsize_hint: * 0 means we use a default batchsize
427                               * -1 means the entire dataset
428                               * float between 0 and 1 means each batch is

```



```

429         that fraction of the DS
430         * int > 1 means that specific batch size
431         :returns: the actual batchsize to use
432         """
433
434         if batchsize_hint == -1:
435             return ds_size
436         elif batchsize_hint == 0:
437             return min(512, math.ceil(ds_size / 2.0))
438         elif (batchsize_hint > 0) and (batchsize_hint < 1):
439             return math.ceil(ds_size * batchsize_hint)
440         elif batchsize_hint > 1:
441             return min(batchsize_hint, ds_size)
442         else:
443             raise ValueError("batchsize_hint must be >=-1")
444
445     def reset_iteration(self, shuffle=True):
446         self.index = 0
447         if shuffle and self.dataset.train:
448             self.permutation = torch.randperm(len(self.dataset))
449         else:
450             self.permutation = torch.arange(len(self.dataset))
451
452     def __iter__(self):
453         """
454         :returns: this iterator
455         """
456         return self
457
458     def __next__(self) -> Dict[str, Tensor]:
459         """
460         Returns one batch of data.
461
462         :raises: StopIteration when we're out of data
463         :returns: batch tensor of shape (self.batchsize, tokens_per_eq)
464         """
465
466         batch_begin = self.index * self.batchsize
467         if batch_begin > len(self.dataset) - 1:
468             self.reset_iteration()
469             raise StopIteration
470         indices = self.permutation[batch_begin : batch_begin + self.batchsize]
471         text = self.dataset.data[indices, :-1]
472         target = self.dataset.data[indices, 1:]
473         batch = {"text": text.to(self.device), "target": target.to(self.device)}
474         self.index += 1
475         return batch
476
477     def __len__(self) -> int:
478         """
479         :returns: the total number of batches
480         """
481         return math.ceil(len(self.dataset) / self.batchsize)

```

4.1.3 transformer.py — Decoder-Only Transformer

```

1 #!/usr/bin/env python
2 from argparse import ArgumentParser, Namespace

```

```
3 from typing import Tuple, List, Dict, Union
4
5 import numpy as np
6 import torch
7 import torch.nn as nn
8 import torch.nn.functional as F
9 from numpy import cos, sin, sqrt
10 from torch import tensor, Tensor
11 from torch.optim.lr_scheduler import LambdaLR
12 import pytorch_lightning as pl
13
14 from argparse import ArgumentParser
15
16
17 class Linear(nn.Linear):
18     def __init__(self, *args, **kwargs):
19         self.weight_noise = kwargs.pop("weight_noise")
20         super().__init__(*args, **kwargs)
21
22     def forward(self, input: Tensor) -> Tensor:
23         if self.weight_noise > 0 and self.training:
24             bias = self.bias if self.bias is None else self.bias + torch.randn_like(self.
25                 bias) * self.weight_noise
26             weight = self.weight + torch.randn_like(self.weight) * self.weight_noise
27             # weight = self.weight * torch.exp(torch.randn_like(self.weight) * self.
28                 weight_noise)
29         else:
30             bias = self.bias
31             weight = self.weight
32
33         return F.linear(
34             input,
35             weight,
36             bias,
37         )
38
39 class LayerNorm(nn.LayerNorm):
40     def __init__(self, *args, **kwargs):
41         self.weight_noise = kwargs.pop("weight_noise")
42         super().__init__(*args, **kwargs)
43
44     def forward(self, input: Tensor) -> Tensor:
45         if self.weight_noise > 0 and self.training:
46             bias = self.bias if self.bias is None else self.bias + torch.randn_like(self.
47                 bias) * self.weight_noise
48             weight = self.weight + torch.randn_like(self.weight) * self.weight_noise
49             # weight = self.weight * torch.exp(torch.randn_like(self.weight) * self.
50                 weight_noise)
51         else:
52             bias = self.bias
53             weight = self.weight
54
55         return F.layer_norm(
56             input,
57             self.normalized_shape,
58             weight,
59             bias,
60             self.eps,
61         )
```

```

57
58
59 class Embedding(nn.Embedding):
60     def __init__(self, *args, **kwargs):
61         self.weight_noise = kwargs.pop("weight_noise")
62         super().__init__(*args, **kwargs)
63
64     def forward(self, input: Tensor) -> Tensor:
65         if self.weight_noise > 0 and self.training:
66             weight = self.weight + torch.randn_like(self.weight) * self.weight_noise
67             # weight = self.weight * torch.exp(torch.randn_like(self.weight) * self.
68                 weight_noise)
69         else:
70             weight = self.weight
71         return F.embedding(
72             input,
73             weight,
74             self.padding_idx,
75             self.max_norm,
76             self.norm_type,
77             self.scale_grad_by_freq,
78             self.sparse,
79         )
80
81 class AttentionHead(nn.Module):
82     def __init__(self, d_model: int, d_key: int, weight_noise: float) -> None:
83
84         super().__init__()
85
86         self.d_key = d_key
87
88         # head projections
89         self.Wq = Linear(d_model, d_key, bias=False, weight_noise=weight_noise)
90         self.Wk = Linear(d_model, d_key, bias=False, weight_noise=weight_noise)
91         self.Wv = Linear(d_model, d_key, bias=False, weight_noise=weight_noise)
92
93         self.softmax = nn.Softmax(dim=-1)
94
95     def forward(
96         self,
97         queries: Tensor,
98         keys: Tensor,
99         values: Tensor,
100         mask: Union[Tensor, None] = None,
101         save_activations: bool = False,
102     ) -> Tuple[Tensor, Union[Tensor, None], Union[Tensor, None]]:
103
104         # project queries, keys, values
105         queries = self.Wq(queries)
106         keys = self.Wk(keys)
107         values = self.Wv(values)
108
109         # calculate compatibility function
110         attn = torch.matmul(queries, torch.transpose(keys, -2, -1))
111         attn = attn / sqrt(self.d_key)
112
113         # Filter out attention to future positions

```

```

114         if mask is not None:
115             attn.masked_fill_(mask == 0, float("-inf"))
116
117         # softmax
118         attn = self.softmax(attn)
119
120         # sum the weighted value vectors
121         result: Tensor = torch.matmul(attn, values) # shape = (max_context_len, d_key)
122         if save_activations:
123             leaf_attn = attn.clone().detach() # type: ignore
124             leaf_values = values.clone().detach() # type: ignore
125         else:
126             leaf_attn = None # type: ignore
127             leaf_values = None # type: ignore
128
129         return result, leaf_attn, leaf_values
130
131
132 class MultiHeadAttention(nn.Module):
133     def __init__(self, d_model: int, heads: int, weight_noise: float = 0.0) -> None:
134         super().__init__()
135         d_key = int(d_model / heads)
136
137         attn_heads = [
138             AttentionHead(d_model, d_key, weight_noise=weight_noise)
139             for _ in range(heads)
140         ]
141         self.attn_heads = nn.ModuleList(attn_heads)
142         self.Wo = Linear(d_model, d_model, bias=False, weight_noise=weight_noise)
143
144     def forward(
145         self,
146         queries: Tensor,
147         keys: Tensor,
148         values: Tensor,
149         mask: Tensor = None,
150         save_activations=False,
151     ) -> Tuple[Tensor, List[Tensor], List[Tensor]]:
152
153         head_outputs = [
154             h(
155                 queries=queries,
156                 keys=keys,
157                 values=values,
158                 mask=mask,
159                 save_activations=save_activations,
160             )
161             for h in self.attn_heads
162         ]
163         head_results = [output[0] for output in head_outputs]
164
165         if save_activations:
166             layer_attns = list([output[1] for output in head_outputs])
167             layer_values = list([output[2] for output in head_outputs])
168         else:
169             layer_attns = []
170             layer_values = []
171

```

```

172     multihead_result = torch.cat(head_results, dim=-1)
173     multihead_result = self.Wo(multihead_result)
174     return multihead_result, layer_attns, layer_values
175
176
177 class FFN(nn.Module):
178     def __init__(
179         self,
180         d_model: int,
181         multiplier: int = 4,
182         non_linearity: str = "relu",
183         weight_noise: float = 0.0,
184     ) -> None:
185         super().__init__()
186
187         d_ff = int(multiplier * d_model)
188
189         non_linearities = {"relu": nn.ReLU, "gelu": nn.GELU}
190
191         self.ffn = nn.Sequential(
192             Linear(d_model, d_ff, bias=False, weight_noise=weight_noise),
193             non_linearities[non_linearity](),
194             Linear(d_ff, d_model, bias=False, weight_noise=weight_noise),
195         )
196
197     def forward(self, x: Tensor) -> Tensor:
198         return self.ffn(x)
199
200
201 class DecoderBlock(nn.Module):
202     def __init__(
203         self,
204         d_model: int,
205         heads: int,
206         dropout: float,
207         non_linearity: str = "relu",
208         weight_noise: float = 0.0,
209     ) -> None:
210         super().__init__()
211
212         self.self_attn = MultiHeadAttention(d_model, heads, weight_noise=weight_noise)
213         # self.self_attn_drop = nn.Dropout(p=dropout)
214         self.self_attn_norm = LayerNorm(d_model, weight_noise=weight_noise)
215
216         self.ffn = FFN(d_model, non_linearity=non_linearity, weight_noise=weight_noise)
217         self.ffn_drop = nn.Dropout(p=dropout)
218         self.ffn_norm = LayerNorm(d_model, weight_noise=weight_noise)
219
220     def forward(
221         self,
222         x: Tensor,
223         self_attn_mask: Tensor = None,
224         save_activations: bool = False,
225     ) -> Tuple[Tensor, List[Tensor], List[Tensor]]:
226         a1, layer_attns, layer_values = self.self_attn(
227             x, x, x, self_attn_mask, save_activations
228         )
229         # a1 = self.self_attn_drop(a1)

```

```

230     a1 = self.self_attn_norm(x + a1)
231
232     a2 = self.ffn(a1)
233     a2 = self.ffn_drop(a2)
234     a2 = self.ffn_norm(a1 + a2)
235
236     return a2, layer_attns, layer_values
237
238
239 class Decoder(nn.Module):
240     def __init__(
241         self,
242         d_model: int,
243         heads: int,
244         num_blocks: int,
245         dropout: float,
246         non_linearity: str = "relu",
247         weight_noise: float = 0.0,
248     ) -> None:
249         super().__init__()
250
251         self.blocks = nn.ModuleList(
252             [
253                 DecoderBlock(
254                     d_model, heads, dropout, non_linearity, weight_noise=weight_noise
255                 )
256                 for _ in range(num_blocks)
257             ]
258         )
259
260     def forward(
261         self,
262         x: Tensor,
263         self_attn_mask: Tensor = None,
264         save_activations=False,
265     ) -> Tuple[Tensor, List[List[Tensor]], List[List[Tensor]]]:
266
267         a = x
268         attentions = []
269         values = []
270         for block in self.blocks:
271             a, layer_attentions, layer_values = block(
272                 a, self_attn_mask, save_activations=save_activations
273             )
274             if save_activations:
275                 attentions.append(layer_attentions)
276                 values.append(layer_values)
277         return a, attentions, values
278
279
280 class Transformer(nn.Module):
281     def __init__(
282         self,
283         n_layers: int = 4,
284         n_heads: int = 4,
285         d_model: int = 256,
286         dropout: float = 0.1,
287         max_context_len: int = 1024,

```

```

288     vocab_len: int = 2000,
289     non_linearity: str = "relu",
290     weight_noise: float = 0.0,
291 ) -> None:
292     super().__init__()
293
294     self.n_layers = n_layers
295     self.n_heads = n_heads
296     self.d_model = d_model
297     self.dropout = dropout
298     self.max_context_len = max_context_len
299     self.non_linearity = non_linearity
300
301     self.vocab_len = vocab_len
302
303     self.embedding = Embedding(vocab_len, d_model, weight_noise=weight_noise) # type
304     : ignore
305     self.register_buffer(
306         "position_encoding", self._position_encoding(max_context_len, d_model)
307     )
308     self.register_buffer("self_attn_mask", self.make_mask(max_context_len))
309
310     self.decoder = Decoder(
311         d_model,
312         n_heads,
313         n_layers,
314         dropout,
315         self.non_linearity,
316         weight_noise=weight_noise,
317     )
318
319     self.linear = Linear(d_model, vocab_len, bias=False, weight_noise=weight_noise)
320
321 @staticmethod
322 def make_mask(context_len: int) -> Tensor:
323     return torch.ones([context_len, context_len]).tril()
324
325 @classmethod
326 def _position_encoding(cls, context_len: int, d_model: int) -> Tensor:
327     rows = [
328         tensor(
329             [
330                 sin(pos / (10000 ** (i / d_model)))
331                 if i % 2 == 0
332                 else cos(pos / (10000 ** ((i - 1) / d_model)))
333                 for i in range(d_model)
334             ]
335         )
336         for pos in range(context_len)
337     ]
338     stack = torch.stack(rows, dim=1)
339
340     return stack.T # type: ignore
341
342 def embed(self, indices: Tensor) -> Tensor:
343     context_len = indices.shape[-1]
344     pe = self.position_encoding[:context_len, :] # type: ignore

```

```

345         embedded = self.embedding(indices)
346
347         return pe + embedded # interesting here cause the positional embeddings are
                               context_len * sequence len, but the embedded shape is [batch size x
                               context_len x sequence len]
348
349     def forward(
350         self,
351         x: Tensor,
352         pos: int = None,
353         save_activations: bool = False,
354     ) -> Tuple[Tensor, Union[Tensor, None], Union[Tensor, None]]:
355         """parameters:
356         x: (rank-1 tensor) vocab indices of decoder input token
357             sequence"""
358
359         # Make sure sampling inputs are on the correct device
360         x = x.to(self.embedding.weight.device)
361
362         # make_attention mask
363         this_max_context_len = x.shape[-1]
364         self_attn_mask = self.self_attn_mask[ # type: ignore
365             :this_max_context_len, :this_max_context_len
366         ]
367
368         # Decode
369         x = self.embed(x)
370         decoded, attentions, values = self.decoder(
371             x, self_attn_mask, save_activations=save_activations
372         )
373
374         # Return predictions for specific token
375         if pos is not None:
376             decoded = decoded[:, pos, :]
377
378         y_hat = self.linear(decoded)
379         return y_hat, attentions, values, decoded

```

4.1.4 training.py — Training Loop, CustomAdamW, SAM, Sharpness

```

1  #!/usr/bin/env python
2
3  import argparse
4  import copy
5  import json
6  import logging
7  import math
8  import os
9  import sys
10 import pickle
11 from argparse import ArgumentParser, Namespace
12 from functools import reduce
13 from typing import Any, Dict, List, Optional, Tuple, Union
14 import time
15
16 import numpy as np
17 import torch
18 import torch.nn.functional as F

```



```

19 from pytorch_lightning import LightningModule, Trainer
20 from pytorch_lightning.callbacks import Callback, ModelCheckpoint
21 from pytorch_lightning.loggers import CSVLogger
22 from torch import Tensor
23 from torch.optim import SGD
24 from torch.optim.lr_scheduler import LambdaLR
25
26 import grok.metrics as metrics
27 from grok.data import (
28     DEFAULT_DATA_DIR,
29     EOS_TOKEN,
30     VALID_OPERATORS,
31     ArithmeticDataset,
32     ArithmeticIterator,
33 )
34 from grok.transformer import Transformer
35 from grok.measure import get_sharpness
36
37 DEFAULT_LOG_DIR = "logs"
38 train_sharpness, train_hessian, train_weight_norm = [], [], []
39 val_sharpness, val_hessian, val_weight_norm = [], [], []
40 train_global_step, val_global_step = [], []
41 all_train_loss, all_val_loss = [], []
42 all_x_lhs_train, all_y_predict_train = [], []
43 all_x_lhs_val, all_y_predict_val = [], []
44 all_train_repr, all_val_repr = [], []
45 all_embedding = []
46
47
48 class TrainableTransformer(LightningModule): # build the transformer based model
49     """
50     Adds training methods to train a generic transformer on arithmetic equations
51     """
52
53     def __init__(self, hparams: Namespace) -> None:
54         """
55         :param hparams: An argparse.Namespace with parameters defined in
56                        self.add_model_specific_args().
57         """
58         super().__init__()
59         self.hparams = hparams # type: ignore
60         # self.hparams.update(dict(hparams))
61         self.prepare_data()
62
63         self.transformer = Transformer(
64             hparams.n_layers,
65             hparams.n_heads,
66             hparams.d_model,
67             hparams.dropout,
68             hparams.max_context_len,
69             len(self.train_dataset.tokenizer),
70             hparams.non_linearity,
71             weight_noise=self.hparams.weight_noise,
72         )
73         # print("yes")
74         # store tokenizer dict here:
75         with open('tokenizer_dict.json', 'w') as json_file:
76             json.dump(self.train_dataset.tokenizer.stoi, json_file)

```

```

77     json_file.close()
78
79     self.margin = torch.Tensor([0])
80     self.next_epoch_to_eval = -1
81     self.next_train_epoch_to_log = 0
82
83     @staticmethod
84     def add_model_specific_args(parser: ArgumentParser) -> ArgumentParser:
85         """
86         Defines the hyperparameter arguments needed by instances of this
87         class. This is intended to be called when parsing command line
88         arguments.
89
90         :param parser: an argparse.ArgumentParser created by the caller
91         :returns: the argument parser with the command line arguments added
92                 for this class.
93         """
94         parser.add_argument(
95             "--batchsize",
96             type=float,
97             # default=0.25,
98             default=0,
99             help="-1->entire_dataset, 0->auto-calculate, 0<N<1->fraction_of_dataset, N>1->N",
100         )
101
102         parser.add_argument("--n_layers", type=int, default=2)
103         parser.add_argument("--n_heads", type=int, default=4)
104         parser.add_argument("--d_model", type=int, default=128)
105         parser.add_argument("--dropout", type=float, default=0.0)
106         parser.add_argument("--weight_noise", type=float, default=0.0)
107         parser.add_argument("--non_linearity", type=str, default="relu")
108         parser.add_argument("--max_context_len", type=int, default=50)
109
110
111         parser.add_argument("--math_operator", type=str, default="+")
112         parser.add_argument(
113             "--operand_length",
114             type=int,
115             help="for list operations, the length of the lists",
116         )
117
118         parser.add_argument("--train_data_pct", type=float, default=50)
119         parser.add_argument("--actual_train_data_pct", type=float, default=50)
120         parser.add_argument("--hessian_coeff", type=float, default=0.01)
121         parser.add_argument("--max_hessian_coeff", type=float, default=1e-3)
122         parser.add_argument("--use_scheduler", type=int, default=0)
123
124         parser.add_argument("--training_date", type=str, default=1119)
125         parser.add_argument("--warmup_steps", type=int, default=10)
126         parser.add_argument("--anneal_lr_steps", type=int, default=100000)
127         parser.add_argument("--hessian_coeff_direction", type=float, default=1.0)
128         parser.add_argument("--use_rglr", type=int, default=0)
129         parser.add_argument("--reg_step", type=int, default=0)
130         parser.add_argument("--use_pow", type=int, default=0)
131         parser.add_argument("--clip_value", type=float, default=0.5, help="if set as 0 then no clipping")

```

```

132     parser.add_argument("--freeze", type=int, default=0, help="0=not freeze, 1=
        only_train_linear_layer_and_other=both_linear_and_embedding_layer")
133
134     parser.add_argument("--opt_type", type=str, default="AdamW")
135
136     parser.add_argument("--anneal_lr", dest="anneal_lr", action="store_true")
137     parser.set_defaults(anneal_lr=False)
138
139     parser.add_argument("--max_lr", type=float, default=1e-3)
140     parser.add_argument("--weight_decay", type=float, default=1)
141     parser.add_argument("--weight_decay_kind", type=str, default="to_zero")
142     parser.add_argument("--noise_factor", type=float, default=0)
143
144     parser.add_argument(
145         "--save_activations", dest="save_activations", action="store_true"
146     )
147     parser.set_defaults(save_activations=False)
148     parser.add_argument("--save_outputs", dest="save_outputs", action="store_true")
149     parser.set_defaults(save_outputs=False)
150
151     parser.add_argument(
152         "--logdir",
153         type=str,
154         default=DEFAULT_LOG_DIR,
155     )
156     parser.add_argument(
157         "--datadir",
158         type=str,
159         default=DEFAULT_DATA_DIR,
160     )
161
162     return parser
163
164 def prepare_data(self) -> None:
165     """
166     Used by pytorch_lightning
167
168     Loads training data to self.train_dataset
169     Loads validation data to self.val_dataset
170     """
171     assert self.hparams.train_data_pct >= self.hparams.actual_train_data_pct
172
173     (self.train_dataset, self.val_dataset,) = ArithmeticDataset.splits(
174         train_pct=self.hparams.train_data_pct, # type: ignore
175         actual_train_pct=self.hparams.actual_train_data_pct, # type: ignore
176         operator=self.hparams.math_operator, # type: ignore
177         operand_length=self.hparams.operand_length, # type: ignore
178         data_dir=self.hparams.datadir, # type: ignore
179     )
180     torch.save(self.train_dataset.data, os.path.join(self.hparams.
        store_train_plot_data, "train_data.pt"))
181     torch.save(self.val_dataset.data, os.path.join(self.hparams.store_val_plot_data,
        "val_data.pt"))
182
183 def train_dataloader(self) -> ArithmeticIterator: # type: ignore
184     """
185     Used by pytorch_lightning
186

```

```

187         :returns: an iterator for self.train_dataset
188         """
189         device = self.transformer.embedding.weight.device
190         iterator = ArithmeticIterator(
191             self.train_dataset,
192             device,
193             batchsize_hint=self.hparams.batchsize, # type: ignore
194         )
195         self.train_batchsize = iterator.batchsize
196         self.batches_per_epoch = len(iterator)
197
198         return iterator
199
200     def val_dataloader(self) -> ArithmeticIterator: # type: ignore
201         """
202         Used by pytorch_lightning
203
204         :returns: an iterator for self.train_dataset
205         """
206         device = self.transformer.embedding.weight.device
207         iterator = ArithmeticIterator(
208             self.val_dataset,
209             device,
210             batchsize_hint=-1, # no need to batch validation data
211         )
212         return iterator
213
214     def test_dataloader(self) -> ArithmeticIterator: # type: ignore
215         """
216         Used by pytorch_lightning
217
218         :returns: an iterator for self.train_dataset
219         """
220         device = self.transformer.embedding.weight.device
221         iterator = ArithmeticIterator(
222             self.val_dataset, device, batchsize_hint=-1 # type: ignore
223         )
224         return iterator
225
226     def _scheduler_lr(self, step: int) -> float:
227         """
228         Used by pytorch_lightning
229
230         :returns: the learning_rate for this training step
231         """
232         max_lr = self.hparams.max_lr # type: ignore
233         min_lr = self.hparams.max_lr / 10 # type: ignore
234         warmup_steps = self.hparams.warmup_steps # type: ignore
235         if not self.hparams.anneal_lr:
236             if step <= warmup_steps:
237                 lr = (float(step) / max(warmup_steps, 1)) * max_lr
238             else:
239                 lr = max_lr
240         else:
241             if step <= warmup_steps:
242                 lr = (float(step) / max(warmup_steps, 1)) * max_lr
243             elif step <= self.hparams.anneal_lr_steps + warmup_steps:
244                 effective_step = step - warmup_steps

```

```

245         t = effective_step / self.hparams.anneal_lr_steps
246         cos = (1 + np.cos(np.pi * t)) / 2
247         lr = min_lr + (max_lr - min_lr) * cos
248         # lr = max_lr - ((effective_step / max_effective_step) * (max_lr - min_lr
249         ))
250     else:
251         lr = min_lr
252     return lr
253
254 def configure_optimizers(self) -> Tuple[List[Any], List[Dict]]:
255     """
256     Used by pytorch_lightning
257
258     :returns: optimizers and schedulers.
259     """
260     if self.hparams.opt_type == "AdamW":
261         optimizer = CustomAdamW(
262             self.parameters(),
263             betas=(0.9, 0.98),
264             eps=1e-8,
265             lr=1,
266             weight_decay=self.hparams.weight_decay,
267             noise_factor=self.hparams.noise_factor,
268             weight_decay_form=self.hparams.weight_decay_kind,
269         )
270         # optimizer = SAM(
271         #     self.parameters(),
272         #     base_optimizer=CustomAdamW,
273         #     rho=0.05,
274         #     betas=(0.9, 0.98),
275         #     eps=1e-8,
276         #     lr=1,
277         #     weight_decay=self.hparams.weight_decay,
278         #     noise_factor=self.hparams.noise_factor,
279         # )
280     else:
281         optimizer = SGD(self.parameters(), lr=1, momentum=0.9, weight_decay=self.
282             hparams.weight_decay)
283     schedulers = [
284         {
285             "scheduler": LambdaLR(optimizer, lr_lambda=self._scheduler_lr),
286             "interval": "step",
287             "frequency": 1,
288         }
289     ]
290     return [optimizer], schedulers
291
292 def _accuracy(self, y_hat: Tensor, y: Tensor) -> Tensor:
293     """
294     Takes the most likely solution predicted for each equation and
295     calculates the frac of equations in the batch for which these
296     answers were correct
297
298     :param y_hat: The softmax tensor output of the transformer
299     :param y: A tensor of the token ids for the correct answers to each
300         equation in the batch
301     :returns: the fraction of equations correctly answered
302     """

```

```

301
302     # find max prediction from output
303     y_hat = torch.max(y_hat, dim=-2).indices # batchsize x num_rhs_tokens
304     row_accuracy = torch.min((y_hat == y), dim=-1).values # shape: batchsize
305     accuracy = row_accuracy.float() * 100 # shape: batchsize
306     return accuracy
307
308 def _step(
309     self,
310     batch: Dict,
311     batch_idx: int,
312     train: bool = True,
313     reduction: str = "mean",
314     grads: bool = False,
315 ) -> Tuple[Tensor, Tensor, float, Tensor, Tensor, Tensor, Tensor]:
316     """
317     Performs one forward pass on a training or validation batch
318
319     :param batch: The batch of equations to process
320     :param batch_idx: which batch this is in the epoch.
321     :param train: True is this is a training batch, false otherwise
322     :returns: The loss from the predicted solutions to the equation,
323             The accuracy of the predicted solutions
324             The fraction of this dataset contained in this batch
325             The portion of the input equations left of the equal sign
326             The softmax probabilities for the solutions to the equations
327             A list lists of attention matrices by layer and head
328             A list lists of value matrices by layer and head
329             Margin for this batch
330     """
331     x = batch["text"] # shape = batchsize * context_len
332     y = batch["target"] # shape = batchsize * context_len
333     y_hat, attentions, values, decoded_outputs = self(
334         x=x, save_activations=self.hparams.save_activations # type: ignore
335     ) # shape = batchsize * context_len * vocab_size
336     y_hat = y_hat.transpose(-2, -1) # shape = batchsize * vocab_size * context_len
337     decoded_outputs = decoded_outputs.transpose(-2, -1)
338
339     # Note: each sample must have exactly one '=' and all of them must
340     # have it in the same position.
341     eq_token_index = self.train_dataset.tokenizer.stoi["="]
342     eq_position_t = torch.nonzero(y[0, :] == eq_token_index, as_tuple=False)
343     eq_position = int(eq_position_t.squeeze())
344
345     # only calculate loss/accuracy on right hand side of the equation
346     y_rhs = y[..., eq_position + 1 :]
347     y_hat_rhs = y_hat[..., eq_position + 1 :]
348     decoded_outputs = decoded_outputs[..., eq_position + 1 :]
349     x_lhs = x[..., : eq_position + 1]
350
351     if train:
352         coeff = float(batch["target"].shape[0]) / len(self.train_dataset)
353
354     else:
355         coeff = float(batch["target"].shape[0]) / len(self.val_dataset)
356
357     # remember to get it back!!!
358     loss = F.cross_entropy(y_hat_rhs, y_rhs, reduction="mean")

```

```

359     all_loss = F.cross_entropy(y_hat_rhs, y_rhs, reduction="none")
360
361     with torch.no_grad():
362         acc = self._accuracy(y_hat_rhs, y_rhs)
363         if reduction == "mean":
364             acc = acc.mean()
365
366     """
367     device = self.transformer.embedding.weight.device
368     self.margin = self.margin.to(device)
369
370     output = y_hat_rhs.clone() # batchsize, vocabsz, rhs tokens
371     output_m = output.clone() # batchsize, vocabsz, rhs tokens
372     target = y_rhs.clone() # batchsize, rhs tokens
373
374     for i in range(output.size(0)): # batch
375         for j in range(output.size(2)): # rhs tokens
376             output_m[i, target[i, j], j] = output_m[i, :, j].min()
377
378     for i in range(output.size(2)): # rhs tokens
379         output_compressed = output[:, target[:, i], i].squeeze().diag()
380         output_m_compressed = (
381             output_m[:, output_m.max(dim=1).indices[:, i], i].squeeze().diag()
382         )
383         self.margin = torch.cat(
384             (
385                 self.margin,
386                 (output_compressed - output_m_compressed),
387             ),
388             0,
389         )
390     """
391     grad_vec = None
392     if grads:
393         loss.backward()
394         for p in self.parameters():
395             p.grad.data.div_(batch["text"].shape[0])
396             if grad_vec is None:
397                 grad_vec = p.grad.data.view(-1)
398             else:
399                 grad_vec = torch.cat((grad_vec, p.grad.data.view(-1)))
400         return loss, grad_vec
401     return loss, acc, coeff, x_lhs, y_hat_rhs, attentions, values, decoded_outputs,
402     all_loss, y_hat, batch["target"][:, -2:]
403     # return loss, acc, coeff, x_lhs, y_hat_rhs, attentions, values, decoded_outputs,
404     all_loss
405
406
407 def _save_inputs(self, outputs: Dict, ds: str) -> None:
408     """
409     Saves the input equations to disk for analysis later
410
411     :param outputs: a list of tuples from self.training_step()
412     :param ds: a string ('train' or 'val') naming which dataset
413                these inputs are from.
414     :param train: True if this is a training batch, false otherwise

```

```

415     """
416     logdir = self.hparams.logdir + "/inputs/" + ds # type: ignore
417     os.makedirs(logdir, exist_ok=True)
418     pickle_file = logdir + f"/{ds}.pt"
419
420     x_lhs = torch.cat([x["x_lhs"] for x in outputs])
421     with open(pickle_file, "wb") as fh:
422         torch.save(x_lhs, fh)
423
424     def _merge_batch_activations(
425         self, partial_activations: List[List[Tensor]]
426     ) -> List[List[Tensor]]:
427         """
428         Merges the head attentions / head_values from all batches in
429         this epoch.
430
431         :param partial_activations: A list of
432                                     (lists of lists of activations by layer and head)
433         :returns: A lists of lists of activations by layer and head
434         """
435         # num_batches = len(partial_activations)
436         num_layers = len(partial_activations[0])
437         num_heads = len(partial_activations[0][0])
438         activations: List = []
439         for _ in range(num_layers):
440             activations.append([])
441             for _ in range(num_heads):
442                 activations[-1].append([])
443
444         for minibatch_activations in partial_activations:
445             for l, layer_activations in enumerate(minibatch_activations):
446                 for h, head_attn in enumerate(layer_activations):
447                     # # print(f"head_attn = {head_attn}")
448                     activations[l][h].append(head_attn)
449
450         for l in range(num_layers):
451             for h in range(num_heads):
452                 activations[l][h] = torch.cat(activations[l][h])
453
454         return activations
455
456     def _save_activations(self, outputs: Dict, ds: str) -> None:
457         """
458         Saves activations out to disk for analysis later
459
460         :param outputs: a list of tuples from self.training_step()
461         """
462
463         output: Dict[str, Any] = {}
464         if self.hparams.save_outputs: # type: ignore
465             y_hat_rhs = torch.cat([x["y_hat_rhs"] for x in outputs])
466             output["y_hat_rhs"] = y_hat_rhs
467         if self.hparams.save_activations: # type: ignore
468             partial_attentions = list([o["partial_attentions"] for o in outputs])
469             attentions = self._merge_batch_activations(partial_attentions)
470             partial_values = list([o["partial_values"] for o in outputs])
471             values = self._merge_batch_activations(partial_values)
472             output["attentions"] = attentions

```



```

473         output["values"] = values
474     if self.hparams.save_outputs or self.hparams.save_activations: # type: ignore
475         logdir = self.hparams.logdir + "/outputs/" + ds # type: ignore
476         os.makedirs(logdir, exist_ok=True)
477         pickle_file = logdir + f"/epoch_{self.current_epoch:010}.pt"
478         with open(pickle_file, "wb") as fh:
479             torch.save(output, fh)
480
481     def _getWeights(self, Xphi, delta):
482         # print("yes")
483         return delta*torch.sum(torch.mul(Xphi, Xphi), dim=-1)**(1/2) # modify here to use
            torch.sum()
484
485     def _getKernelValue(self, x, total_num, kde): # can here move to gpu?
486         return_list = []
487         for i in range(total_num):
488
489             weight = (1 / kde.data.shape[0]) if kde.weights is None else kde.weights[i]
490             dist = np.sqrt(np.linalg.norm(x - kde.data[i]))
491             return_list.append(weight * kde.kernel(dist, bw=kde.bw, norm=kde.norm))
492         return return_list
493
494     def training_step(self, batch, batch_idx):
495         """
496         Used by pytorch_lightning
497         Runs one forward training pass on one batch
498
499         :param batch: The batch of equations to process
500         :param batch_idx: which batch this is in the epoch.
501         :returns: a dict with loss, accuracy, lr, probabilities of solutions,
502                 attentions, and values
503         """
504         if batch_idx == 0:
505             self.training_epoch_start_time = time.time()
506             self.fwd_time_in_epoch = 0
507
508         start = time.time()
509         loss, accuracy, coeff, x_lhs, y_hat_rhs, attentions, values, decoded_outputs,
510             all_loss, y_hat, y_true_labels = self._step(
511             batch=batch, batch_idx=batch_idx, train=True
512         )
513         self.fwd_time_in_epoch += time.time() - start
514
515         with torch.no_grad(): # pay attention here if not use it for loss computation.
516             temp_outputs = torch.mul(y_hat_rhs[:, :, 0], y_hat_rhs[:, :, 1])
517             probs = torch.softmax(temp_outputs, dim=1)
518             hessian_first = torch.sum(torch.mul(probs, 1 - probs), dim=1)
519             concatenated_outputs = torch.cat((decoded_outputs[:, :, 0], decoded_outputs[:, :, 1]),
520                 dim=1)
521             hessian_second = torch.sum(torch.mul(concatated_outputs, concated_outputs), dim
522                 =1)
523             hessian = torch.mul(hessian_first, hessian_second)
524             weights_norm = torch.linalg.norm(self.transformer.linear.weight)
525             sharpness = weights_norm * hessian
526             sharpness2 = weights_norm * weights_norm * hessian
527
528         schedulers = self.trainer.lr_schedulers[0]
529         if self.current_epoch != self.next_train_epoch_to_log:

```

```

527         return {"loss": loss}
528     lr = schedulers["scheduler"].optimizer.param_groups[0]["lr"]
529
530     output = {
531         "loss": loss,
532         "partial_train_loss": coeff * loss,
533         "partial_train_accuracy": coeff * accuracy,
534         "learning_rate": torch.tensor([lr]),
535         "y_hat_rhs": y_hat_rhs,
536         "partial attentions": attentions,
537         "partial_values": values,
538         "all_loss": all_loss,
539         "sharpness": sharpness,
540         "double_sharpness": sharpness2,
541         "hessian_value": hessian,
542         "weight_norm_value": weights_norm,
543         "x_left": batch["text"],
544         "y_predict": y_hat_rhs,
545         "y_repr": decoded_outputs,
546         "y_predict_all": y_hat,
547         "y_true_labels": y_true_labels,
548     }
549
550     if self.current_epoch == 0:
551         output["x_lhs"] = x_lhs
552
553     return output
554
555 def training_epoch_end(self, outputs):
556     """
557     Used by pytorch_lightning
558     Accumulates results of all forward training passes in this epoch
559
560     :param outputs: a list of dicts from self.training_step()
561     :param batch_idx: which batch this is in the epoch.
562     :returns: a dict with loss, accuracy, lr, probabilities of solutions,
563             attentions, and values
564     """
565     epoch_is_to_be_logged = self.current_epoch == self.next_train_epoch_to_log
566     if epoch_is_to_be_logged:
567         self.next_train_epoch_to_log = max(
568             int(1.01 * self.next_train_epoch_to_log),
569             self.next_train_epoch_to_log + 1,
570         )
571         with torch.no_grad():
572             try:
573                 loss = torch.stack([x["partial_train_loss"] for x in outputs]).sum()
574             except Exception as e:
575                 print("!" * 80)
576                 print(outputs)
577                 raise e
578             perplexity = torch.exp(loss)
579             accuracy = torch.stack(
580                 [x["partial_train_accuracy"] for x in outputs]
581             ).sum()
582             all_loss = torch.cat([x["all_loss"] for x in outputs])
583             sharpness = torch.cat([x["sharpness"] for x in outputs])
584             sharpness2 = torch.cat([x["double_sharpness"] for x in outputs])

```

```

585     hessian_value = torch.cat([x["hessian_value"] for x in outputs])
586     weight_norm_value = [x["weight_norm_value"] for x in outputs]
587
588     x_lhs_train = torch.cat([x["x_left"] for x in outputs])
589     train_repr = torch.cat([x["y_repr"] for x in outputs])
590     y_left_predict_train = torch.cat([torch.max(x["y_predict"], dim=-2).
591         indices for x in outputs])
592     y_left_predict_train_all = torch.cat([torch.max(x["y_predict_all"], dim
593         =-2).indices for x in outputs])
594     y_left_predict_train_softmax = torch.cat([x["y_predict"] for x in outputs
595         ])
596     y_true_labels = torch.cat([x["y_true_labels"] for x in outputs])
597
598     # store the training set sharpness and loss
599     np.save(self.hparams.store_train_plot_data + "/hessian_train_" + str(self
600         .trainer.global_step), hessian_value.detach().cpu().numpy())
601     np.save(self.hparams.store_train_plot_data + "/sharpness_train_" + str(
602         self.trainer.global_step), sharpness.detach().cpu().numpy())
603     np.save(self.hparams.store_train_plot_data + "/sharpness2_train_" + str(
604         self.trainer.global_step), sharpness2.detach().cpu().numpy())
605     np.save(self.hparams.store_train_plot_data + "/weightnorm_train_" + str(
606         self.trainer.global_step), np.array([x.detach().cpu().item() for x in
607         weight_norm_value], dtype=np.float32))
608     np.save(self.hparams.store_train_plot_data + "/loss_train_" + str(self.
609         trainer.global_step), all_loss.detach().cpu().numpy())
610     np.save(self.hparams.store_train_plot_data + "/y_predict_train_" + str(
611         self.trainer.global_step), y_left_predict_train.detach().cpu().numpy
612         ())
613     np.save(self.hparams.store_train_plot_data + "/y_predict_train_softmax_"
614         + str(self.trainer.global_step), y_left_predict_train_softmax.detach
615         ().cpu().numpy())
616     np.save(self.hparams.store_train_plot_data + "/y_train_labels_two_" + str
617         (self.trainer.global_step), y_true_labels.detach().cpu().numpy())
618     np.save(self.hparams.store_train_plot_data + "/train_repr_" + str(self.
619         trainer.global_step), train_repr.detach().cpu().numpy())
620
621     # store the necessary elements here for validation to compute
622     representativeness

```

```

623
624     device = self.transformer.embedding.weight.device
625     val_data = self.val_dataset.data.to(device)
626     validation_data = {"text": val_data[:, :-1], "target": val_data[:, 1:]}
627     val_loss, val_acc, val_coeff, val_x_lhs, val_y_hat_rhs, val attentions,
628         val_values, val_decoded_outputs, val_all_loss, val_y_hat,
629         val_y_true_labels = self._step(validation_data, 0)
630
631     val_temp_outputs = torch.mul(val_y_hat_rhs[:, :, 0], val_y_hat_rhs[:, :, 1])
632     val_probs = torch.softmax(val_temp_outputs, dim=1)
633     val_hessian_first = torch.sum(torch.mul(val_probs, 1 - val_probs), dim=1)
634     val_concatated_outputs = torch.cat((val_decoded_outputs[:, :, 0],
635         val_decoded_outputs[:, :, 1]), dim=1)
636     val_hessian_second = torch.sum(torch.mul(val_concatated_outputs,
637         val_concatated_outputs), dim=1)
638     val_hessian = torch.mul(val_hessian_first, val_hessian_second)
639     val_weights_norm = torch.linalg.norm(self.transformer.linear.weight)
640     val_sharpness = val_weights_norm * val_hessian
641     val_sharpness2 = val_weights_norm * val_weights_norm * val_hessian

```

```

623         # validation storing
624         np.save(self.hparams.store_train_plot_data + "/hessian_val_" + str(self.
625             trainer.global_step), val_hessian.detach().cpu().numpy())
626         np.save(self.hparams.store_train_plot_data + "/sharpness_val_" + str(self
627             .trainer.global_step), val_sharpness.detach().cpu().numpy())
628         np.save(self.hparams.store_train_plot_data + "/sharpness2_val_" + str(
629             self.trainer.global_step), val_sharpness2.detach().cpu().numpy())
630         np.save(self.hparams.store_train_plot_data + "/weightnorm_val_" + str(
631             self.trainer.global_step), val_weights_norm.detach().cpu().numpy())
632         np.save(self.hparams.store_train_plot_data + "/loss_val_" + str(self.
633             trainer.global_step), val_all_loss.detach().cpu().numpy())
634         np.save(self.hparams.store_train_plot_data + "/y_predict_val_" + str(self
635             .trainer.global_step), torch.max(val_y_hat_rhs, dim=-2).indices.
636             detach().cpu().numpy())
637         np.save(self.hparams.store_train_plot_data + "/y_predict_val_softmax_" +
638             str(self.trainer.global_step), val_y_hat_rhs.detach().cpu().numpy())
639         np.save(self.hparams.store_train_plot_data + "/y_val_labels_two_" + str(
640             self.trainer.global_step), val_y_true_labels.detach().cpu().numpy())
641         np.save(self.hparams.store_train_plot_data + "/val_repr_" + str(self.
642             trainer.global_step), val_decoded_outputs.detach().cpu().numpy())
643
644         first_lr = outputs[0]["learning_rate"]
645
646         if self.hparams.save_activations or self.hparams.save_outputs:
647             if self.current_epoch == 0:
648                 self._save_inputs(outputs, ds="train")
649                 self._save_activations(outputs, ds="train")
650
651         logs = {
652             "train_loss": loss,
653             "train_accuracy": accuracy,
654             "train_perplexity": perplexity,
655             "learning_rate": first_lr,
656             "len_train_ds": len(self.train_dataset),
657             "len_val_ds": len(self.val_dataset),
658             "batches_per_epoch": self.batches_per_epoch,
659             "time_per_epoch": time.time() - self.training_epoch_start_time,
660             "fwd_time_in_epoch": self.fwd_time_in_epoch,
661         }
662         for k, v in logs.items():
663             self.log(k, v)
664
665     def validation_step(self, batch, batch_idx):
666         # print("The global step is {}".format(self.trainer.global_step))
667         """
668         Used by pytorch_lightning
669         Runs one forward validation pass on one batch
670
671         :param batch: The batch of equations to process
672         :param batch_idx: which batch this is in the epoch.
673         :returns: a dict with val_loss, val_accuracy, probabilities of solutions,
674                 attentions, and values
675         """
676         if self.next_epoch_to_eval < self.current_epoch:
677             self.next_epoch_to_eval = self.current_epoch
678         if self.current_epoch != self.next_epoch_to_eval:
679             return {}
680         with torch.no_grad():

```

```

671         loss, accuracy, coeff, x_lhs, y_hat_rhs, attentions, values, decoded_outputs,
672             all_loss, y_hat, y_true_labels = self._step(
673             batch=batch, batch_idx=batch_idx, train=False
674         )
675
676         temp_outputs = torch.mul(y_hat_rhs[:, :, 0], y_hat_rhs[:, :, 1])
677         probs = torch.softmax(temp_outputs, dim=1)
678         hessian_first = torch.sum(torch.mul(probs, 1 - probs), dim=1)
679         concated_outputs = torch.cat((decoded_outputs[:, :, 0], decoded_outputs[:, :, 1])
680             , dim=1)
681         hessian_second = torch.sum(torch.mul(concated_outputs, concated_outputs), dim
682             =1)
683         hessian = torch.mul(hessian_first, hessian_second)
684         weights_norm = torch.linalg.norm(self.transformer.linear.weight)
685         sharpness = weights_norm * hessian
686         sharpness2 = weights_norm * weights_norm * hessian
687
688         output = {
689             "partial_val_loss": coeff * loss,
690             "partial_val_accuracy": coeff * accuracy,
691             "y_hat_rhs": y_hat_rhs,
692             "partial_attentions": attentions,
693             "partial_values": values,
694             "val_hessian": hessian,
695             "val_sharpness": sharpness,
696             "val_sharpness2": sharpness2,
697             "val_weight_norm": weights_norm,
698             "all_val_loss": all_loss,
699             "x_left": batch["text"],
700             "y_predict": y_hat_rhs,
701             "y_repr": decoded_outputs,
702             "y_predict_all": y_hat,
703         }
704     }
705     if self.current_epoch == 0:
706         output["x_lhs"] = x_lhs
707
708     return output
709
710 def validation_epoch_end(self, outputs):
711     """
712     Used by pytorch_lightning
713     Accumulates results of all forward validation passes in this epoch
714
715     :param outputs: a list of dicts from self.validation_step()
716     :param batch_idx: which batch this is in the epoch.
717     :returns: a dict with val_loss, val_accuracy
718     """
719     validation_is_real = len(outputs[0]) != 0
720
721     if validation_is_real:
722         self.next_epoch_to_eval = max(
723             int(1.02 * self.next_epoch_to_eval), self.next_epoch_to_eval + 1
724         )
725         with torch.no_grad():
726             loss = torch.stack([x["partial_val_loss"] for x in outputs]).sum()
727             perplexity = torch.exp(loss)

```

```

725         accuracy = torch.stack([x["partial_val_accuracy"] for x in outputs]).sum
726         ()
727         x_lhs_val = torch.cat([x["x_left"] for x in outputs])
728         val_repr = torch.cat([x["y_repr"] for x in outputs])
729         y_left_predict_val = torch.cat([torch.max(x["y_predict"], dim=-2).indices
730             for x in outputs])
731         y_left_predict_val_softmax = torch.cat([x["y_predict"] for x in outputs])
732         y_left_predict_val_all = torch.cat([torch.max(x["y_predict_all"], dim=-2)
733             .indices for x in outputs])
734
735     if self.hparams.save_activations or self.hparams.save_outputs:
736         if self.current_epoch == 0:
737             self._save_inputs(outputs, ds="val")
738             self._save_activations(outputs, ds="val")
739
740     logs = {
741         "val_loss": loss,
742         "val_accuracy": accuracy,
743         "val_perplexity": perplexity,
744     }
745     for name, param in self.named_parameters():
746         # n parameters
747         n_params = param.numel()
748         # get the l2 norm of the parameter
749         logs["paramnorm_" + name] = torch.norm(
750             param, 2
751         ).detach().cpu().numpy() / np.sqrt(n_params)
752
753     # train accuracy
754     device = self.transformer.embedding.weight.device
755     train_data = self.train_dataset.data.to(device)
756     training_data = {"text": train_data[:, :-1], "target": train_data[:, 1:]}
757     with torch.no_grad():
758         tr_loss, tr_acc, coeff, x_lhs, y_hat_rhs, attentions, values,
759         decoded_outputs, all_loss, y_hat, y_true_labels = self._step(
760             training_data, 0)
761         logs["full_train_loss"] = tr_loss
762         logs["full_train_acc"] = tr_acc
763
764     for k, v in logs.items():
765         self.log(k, v)
766
767     # save a checkpoint if the epoch is a power of 2
768     if (
769         self.current_epoch > 0
770         and int(2 ** (int(np.log(self.current_epoch) / np.log(2))))
771         == self.current_epoch
772     ):
773         self.trainer.save_checkpoint(
774             os.path.join(
775                 self.hparams.checkpoint_path,
776                 "epoch_" + str(self.current_epoch) + ".ckpt",
777             )
778         )
779     if validation_is_real:
780         return logs

```

```

778
779 def test_step(self, batch, batch_idx):
780     """
781     Used by pytorch_lightning
782     Runs one forward validation pass on one batch
783
784     :param batch: The batch of equations to process
785     :param batch_idx: which batch this is in the epoch.
786     :returns: a dict with val_loss, val_accuracy, probabilities of solutions,
787             attentions, and values
788     """
789
790     loss, accuracy, coeff, x_lhs, y_hat_rhs, attentions, values = self._step(
791         batch=batch, batch_idx=batch_idx, train=False, reduction="none"
792     )
793     output = {
794         "partial_test_loss": coeff * loss,
795         "partial_test_accuracy": coeff * accuracy,
796         "y_hat_rhs": y_hat_rhs,
797         "partial_attentions": attentions,
798         "partial_values": values,
799     }
800     if self.current_epoch == 0:
801         output["x_lhs"] = x_lhs
802
803     return output
804
805 def test_epoch_end(self, outputs):
806     """
807     Used by pytorch_lightning
808     Accumulates results of all forward validation passes in this epoch
809
810     :param outputs: a list of dicts from self.validation_step()
811     :param batch_idx: which batch this is in the epoch.
812     :returns: a dict with val_loss, val_accuracy
813     """
814     loss = torch.cat([x["partial_test_loss"] for x in outputs], dim=0) # .sum()
815     # loss = list([x["partial_test_loss"] for x in outputs]) # .sum()
816     perplexity = torch.exp(loss)
817     accuracy = torch.cat([x["partial_test_accuracy"] for x in outputs], dim=0)
818
819     logs = {
820         "test_loss": loss,
821         "test_accuracy": accuracy,
822         "test_perplexity": perplexity,
823     }
824
825     return {"test_loss": loss, "log": logs}
826
827 def forward(self, *args, **kwargs) -> Any:
828     """Passes all arguments directly to Transformer.forward()"""
829     return self.transformer(*args, **kwargs)
830
831
832 def train(hparams: Namespace) -> None: # training here
833     """
834     This is the main trainer_method. This sets up and runs experiment with
835     the defined hyperparameters

```

```

836
837 :param hparams: An argparse.Namespace with all of the relevant hyperparameters
838 """
839
840 # Process the args
841 if hparams.logdir is None:
842     hparams.logdir = os.environ.get("LOGDIR", ".")
843 hparams.logdir = os.path.abspath(hparams.logdir)
844
845 # Make sure d_model, heads, and d_key are compatible
846 assert (
847     hparams.d_model % hparams.n_heads == 0
848 ), "n_heads=%s does not evenly divide d_model=%s" % (
849     hparams.n_heads,
850     hparams.d_model,
851 )
852 hparams.d_key = hparams.d_model / hparams.n_heads
853
854 # Set up the RNGs for repeatability
855 if hparams.random_seed != -1:
856     torch.manual_seed(hparams.random_seed)
857     torch.cuda.manual_seed(hparams.random_seed)
858     torch.backends.cudnn.deterministic = True
859     torch.backends.cudnn.benchmark = False
860
861 checkpoint_path = hparams.logdir + "/checkpoints_seed{}_train{}_val{}_date{}".format(
862     hparams.random_seed, hparams.actual_train_data_pct, 100-hparams.train_data_pct,
863     hparams.training_date)
864 os.makedirs(checkpoint_path, exist_ok=True)
865 hparams.checkpoint_path = checkpoint_path
866
867 store_val_plot_data = "/home/jovyan/grok/val_plot_data_seed{}_train{}_val{}_date{}".format(
868     hparams.random_seed, hparams.actual_train_data_pct, 100-hparams.
869     train_data_pct, hparams.training_date)
870 if not os.path.exists(store_val_plot_data):
871     os.makedirs(store_val_plot_data)
872 hparams.store_val_plot_data = store_val_plot_data
873
874 store_train_plot_data = "/home/jovyan/grok/train_plot_data_seed{}_train{}_val{}_date
875 {}".format(hparams.random_seed, hparams.actual_train_data_pct, 100-hparams.
876     train_data_pct, hparams.training_date)
877 if not os.path.exists(store_train_plot_data):
878     os.makedirs(store_train_plot_data)
879 hparams.store_train_plot_data = store_train_plot_data
880
881 # Create the model
882 model = TrainableTransformer(hparams).float()
883
884 torch.save(model, os.path.join(checkpoint_path, "init.pt"))
885
886 logger = CSVLogger(hparams.logdir)
887
888 # checkpointer = ModelCheckpoint(
889 #     filepath=checkpoint_path,
890 #     monitor="save_ckpt",
891 #     mode="max",
892 #     save_top_k=len(hparams.ckpt_epochs),
893 #     verbose=False,

```



```

888 # )
889
890 trainer_args = {
891     "max_steps": hparams.max_steps,
892     "min_steps": hparams.max_steps,
893     "max_epochs": int(1e8),
894     "val_check_interval": 1,
895     "profiler": False,
896     # "checkpoint_callback": checkpointer,
897     "logger": logger,
898     "log_every_n_steps": 1,
899     "flush_logs_every_n_steps": 1000,
900     "gradient_clip_val": hparams.clip_value,
901 }
902
903 if torch.cuda.is_available() and hparams.gpu >= 0:
904     trainer_args["gpus"] = [hparams.gpu]
905
906 trainer = Trainer(**trainer_args)
907
908 trainer.fit(model=model) # type: ignore
909 """
910 margin = np.percentile(model.margin.detach().cpu().numpy(), 5)
911 device = transformer.embedding.weight.device
912 measures, bounds = metrics.calculate(
913     transformer,
914     transformer_init.to(device),
915     device,
916     dataset_size,
917     margin,
918     input_dim=hparams.d_model,
919 )
920
921 measures_file = os.path.join(logger.log_dir, "measures.json")
922 bounds_file = os.path.join(logger.log_dir, "bounds.json")
923 with open(measures_file, "w") as fh:
924     json.dump(measures, fh)
925 with open(bounds_file, "w") as fh:
926     json.dump(bounds, fh)
927 """
928
929 return hparams.logdir
930
931
932 def compute_sharpness(hparams: Namespace, ckpts) -> None:
933     """
934     This is the compute_sharpness method. This loads a series of checkpoints in
935     the defined hyperparameters
936
937     :param hparams: An argparse.Namespace with all of the relevant hyperparameters
938     """
939
940     # Process the args
941     if hparams.logdir is None:
942         hparams.logdir = os.environ.get("LOGDIR", ".")
943     hparams.logdir = os.path.abspath(hparams.logdir)
944
945     # Make sure d_model, heads, and d_key are compatible

```

```

946     assert (
947         hparams.d_model % hparams.n_heads == 0
948     ), "n_heads=%s_does_not_evenly_divide_d_model=%s" % (
949         hparams.n_heads,
950         hparams.d_model,
951     )
952     hparams.d_key = hparams.d_model / hparams.n_heads
953
954     # Set up the RNGs for repeatability
955     if hparams.random_seed != -1:
956         torch.manual_seed(hparams.random_seed)
957         torch.cuda.manual_seed(hparams.random_seed)
958         torch.backends.cudnn.deterministic = True
959         torch.backends.cudnn.benchmark = False
960
961     checkpoint_path = hparams.logdir + "/checkpoints"
962     os.makedirs(checkpoint_path, exist_ok=True)
963     hparams.checkpoint_path = checkpoint_path
964
965     # Create the model
966     model = TrainableTransformer(hparams).float()
967
968     torch.save(model, os.path.join(checkpoint_path, "init.pt"))
969
970     logger = CSVLogger(hparams.logdir)
971
972
973     trainer_args = {
974         "max_steps": hparams.max_steps,
975         "min_steps": hparams.max_steps,
976         "max_epochs": int(1e8),
977         # "max_epochs": int(100),
978         "val_check_interval": 1,
979         "profiler": False,
980         # "checkpoint_callback": checkpointer,
981         "logger": logger,
982         "log_every_n_steps": 1,
983         "flush_logs_every_n_steps": 1000,
984     }
985     if torch.cuda.is_available() and hparams.gpu >= 0:
986         trainer_args["gpus"] = [hparams.gpu]
987
988     trainer = Trainer(**trainer_args)
989
990     for ckpt in ckpts:
991         print(f"Loading_checkpoint_{ckpt}")
992         # model = torch.load(ckpt)
993         # model.load_state_dict(torch.load(ckpt))
994
995         checkpoint = torch.load(ckpt)
996         # print(dir(checkpoint), type(checkpoint), "Ckpt")
997         # for k, v in checkpoint.items():
998         #     print(k)
999         # print(checkpoint["hyper_parameters"])
1000
1001         hps = checkpoint["hyper_parameters"]
1002         hps = argparse.Namespace(**hps)
1003         model = TrainableTransformer(hps).float()

```

```

1004     model.load_state_dict(checkpoint["state_dict"])
1005
1006     phi = get_sharpness(model.train_dataloader(), model)
1007     results = {}
1008     results[ckpt] = phi
1009     pickle.dump(results, open(f"results/results_SD-{i}.pkl", "wb"))
1010
1011
1012 def add_args(parser=None) -> Namespace:
1013     """
1014     Parses the command line arguments
1015
1016     :returns: an argparse.Namespace with all of the needed arguments
1017     """
1018     if parser is None:
1019         parser = ArgumentParser()
1020     parser.add_argument("--random_seed", type=int, default=42)
1021     parser.add_argument("--gpu", type=int, default=0)
1022     parser.add_argument("--max_epochs", type=int, default=None)
1023     parser.add_argument("--max_steps", type=int, default=50)
1024     # parser.add_argument("--max_steps", type=int, default=2000000)
1025     # parser.add_argument("--checkpoint_period", type=int, default=1)
1026     parser = TrainableTransformer.add_model_specific_args(parser)
1027     return parser
1028
1029
1030 class CustomAdamW(torch.optim.Optimizer):
1031     def __init__(
1032         self,
1033         params,
1034         lr=1e-3,
1035         betas=(0.9, 0.999),
1036         eps=1e-8,
1037         weight_decay=1e-2,
1038         amsgrad=False,
1039         noise_factor=0.0,
1040         weight_decay_form="to_zero",
1041     ):
1042         if not 0.0 <= lr:
1043             raise ValueError("Invalid learning rate: {}".format(lr))
1044         if not 0.0 <= eps:
1045             raise ValueError("Invalid epsilon value: {}".format(eps))
1046         if not 0.0 <= betas[0] < 1.0:
1047             raise ValueError("Invalid beta parameter at index 0: {}".format(betas[0]))
1048         if not 0.0 <= betas[1] < 1.0:
1049             raise ValueError("Invalid beta parameter at index 1: {}".format(betas[1]))
1050         if not weight_decay_form in ["to_zero", "to_init", "jiggle", "honest"]:
1051             raise ValueError(
1052                 f"Invalid weight decay form: {weight_decay_form}, should be one of ['to_zero', 'to_init', 'jiggle']"
1053             )
1054         # if not 0.0 <= weight_decay:
1055         #     raise ValueError("Invalid weight decay value: {}".format(weight_decay))
1056         defaults = dict(
1057             lr=lr,
1058             betas=betas,
1059             eps=eps,
1060             weight_decay=weight_decay,

```



```

1118         if amsgrad:
1119             # Maintains max of all exp. moving avg. of sq. grad. values
1120             state["max_exp_avg_sq"] = torch.zeros_like(
1121                 p, memory_format=torch.preserve_format
1122             )
1123
1124     if group["weight_decay"] > 0:
1125         if group["weight_decay_form"] == "to_zero":
1126             p.mul_(1 - group["lr"] * group["weight_decay"])
1127         elif group["weight_decay_form"] == "to_init":
1128             p.add_(
1129                 (state["init"] - p) * (group["lr"] * group["weight_decay"])
1130             )
1131         elif group["weight_decay_form"] == "jiggle":
1132             p.mul_(
1133                 torch.exp(
1134                     torch.randn(1).cuda()
1135                     * (group["lr"] * group["weight_decay"])
1136                 )
1137             )
1138         elif group["weight_decay_form"] == "honest":
1139             pass
1140         else:
1141             raise ValueError(
1142                 f"Invalid weight_decay_form: {group['weight_decay_form']}"
1143             )
1144
1145     exp_avg, exp_avg_sq = state["exp_avg"], state["exp_avg_sq"]
1146     if amsgrad:
1147         max_exp_avg_sq = state["max_exp_avg_sq"]
1148     beta1, beta2 = group["betas"]
1149
1150     state["step"] += 1
1151     bias_correction1 = 1 - beta1 ** state["step"]
1152     bias_correction2 = 1 - beta2 ** state["step"]
1153
1154     # Decay the first and second moment running average coefficient
1155     exp_avg.mul_(beta1).add_(grad, alpha=1 - beta1)
1156     exp_avg_sq.mul_(beta2).addcmul_(grad, grad, value=1 - beta2)
1157     if amsgrad:
1158         # Maintains the maximum of all 2nd moment running avg. till now
1159         torch.max(max_exp_avg_sq, exp_avg_sq, out=max_exp_avg_sq)
1160         # Use the max. for normalizing running avg. of gradient
1161         denom = (max_exp_avg_sq.sqrt() / math.sqrt(bias_correction2)).add_(
1162             group["eps"]
1163         )
1164     else:
1165         denom = (exp_avg_sq.sqrt() / math.sqrt(bias_correction2)).add_(
1166             group["eps"]
1167         )
1168
1169     step_size = group["lr"] / bias_correction1
1170
1171     upd = exp_avg / denom
1172     # add uniform gaussian noise to the update
1173     if group["noise_factor"] > 0:
1174         upd += torch.randn_like(upd) * group["noise_factor"]
1175     # if group['noise_factor'] > 0:

```

```

1176         #     upd *= torch.exp(torch.randn_like(upd) * group['noise_factor'])
1177         p.add_(-step_size * upd)
1178
1179     return loss
1180
1181
1182 class SAM(torch.optim.Optimizer):
1183     def __init__(self, params, base_optimizer, rho=0.05, **kwargs):
1184         assert rho >= 0.0, f"Invalid rho, should be non-negative: {rho}"
1185
1186         defaults = dict(rho=rho, **kwargs)
1187         super(SAM, self).__init__(params, defaults)
1188
1189         self.base_optimizer = base_optimizer(self.param_groups, **kwargs)
1190         self.param_groups = self.base_optimizer.param_groups
1191
1192     @torch.no_grad()
1193     def first_step(self, zero_grad=False):
1194         grad_norm = self._grad_norm()
1195         for group in self.param_groups:
1196             scale = group["rho"] / (grad_norm + 1e-12)
1197
1198             for p in group["params"]:
1199                 if p.grad is None:
1200                     continue
1201                 e_w = p.grad * scale.to(p)
1202                 p.add_(e_w) # climb to the local maximum "w + e(w)"
1203                 self.state[p]["e_w"] = e_w
1204
1205         if zero_grad:
1206             self.zero_grad()
1207
1208     @torch.no_grad()
1209     def second_step(self, zero_grad=False):
1210         for group in self.param_groups:
1211             for p in group["params"]:
1212                 if p.grad is None:
1213                     continue
1214                 p.sub_(self.state[p]["e_w"]) # get back to "w" from "w + e(w)"
1215
1216         self.base_optimizer.step() # do the actual "sharpness-aware" update
1217
1218         if zero_grad:
1219             self.zero_grad()
1220
1221     @torch.no_grad()
1222     def step(self, closure=None):
1223         assert (
1224             closure is not None
1225         ), "Sharpness-Aware Minimization requires closure, but it was not provided"
1226         closure = torch.enable_grad()(
1227             closure
1228         ) # the closure should do a full forward-backward pass
1229
1230         self.first_step(zero_grad=True)
1231         closure()
1232         self.second_step()
1233

```

```

1234 def _grad_norm(self):
1235     shared_device = self.param_groups[0]["params"][
1236         0
1237     ].device # put everything on the same device, in case of model parallelism
1238     grad_norms = [
1239         p.grad.norm(p=2).to(shared_device)
1240         for group in self.param_groups
1241         for p in group["params"]
1242         if p.grad is not None
1243     ]
1244     print("grad_norms is", grad_norms, "!" * 1000)
1245     norm = torch.norm(
1246         torch.stack(grad_norms),
1247         p=2,
1248     )
1249     return norm

```

4.1.5 metrics.py — Norm-Based Measures & Bounds

```

1 import torch
2 import math
3 import copy
4 import torch.nn as nn
5 from typing import Callable
6
7 # References:
8 # https://github.com/nitarshan/robust-generalization-measures
9 # https://github.com/bneyshabur/generalization-bounds
10 # https://github.com/bneyshabur/over-parametrization
11
12
13 def compute_measure(
14     model: nn.Module,
15     init_model: nn.Module,
16     measure_func: Callable,
17     operator: str,
18     kwargs: dict = {},
19     p: int = 1,
20 ) -> float:
21     """
22     Computes measure value for each layer given trained network and network at
23     initialization. Then aggregates values per layer using specified operator.
24
25     :param model: trained network
26     :param init_model: network at initialization
27     :param measure_func: callable for the measure to compute
28     :param operator: 'log_product', 'sum', 'max', 'product', or 'norm'
29     :param p: p in L^p
30     :return: value of the desired measure
31     """
32
33     measure_value = 0
34     # weight_modules = ["Linear", "Embedding"]
35     weight_modules = ["Linear"]
36
37     if operator == "product":
38         measure_value = math.exp(
39             compute_measure(model, init_model, measure_func, "log_product", kwargs, p)

```

```

40     )
41     elif operator == "norm":
42         measure_value = (
43             compute_measure(model, init_model, measure_func, "sum", kwargs, p=p)
44         ) ** (1 / p)
45     else:
46         measure_value = 0
47         for child, init_child in zip(model.children(), init_model.children()):
48             module_name = child._get_name()
49             if module_name in weight_modules:
50                 if operator == "log_product":
51                     measure_value += math.log(measure_func(child, init_child, **kwargs))
52                 elif operator == "sum":
53                     measure_value += (measure_func(child, init_child, **kwargs)) ** p
54                 elif operator == "max":
55                     measure_value = max(
56                         measure_value, measure_func(child, init_child, **kwargs)
57                     )
58             else:
59                 measure_value += compute_measure(
60                     child, init_child, measure_func, operator, kwargs, p=p
61                 )
62     return measure_value
63
64
65 def norm(module, init_module, p=2, q=2):
66     """
67     Calculates l_pq norm of a parameter matrix
68     l_p norm of incoming weights to each hidden unit
69     l_q norm on the hidden units
70     """
71     return module.weight.view(module.weight.size(0), -1).norm(p=p, dim=1).norm(q).item()
72
73
74 def op_norm(module, init_module, p=float("Inf")):
75     """
76     Calculates l_p norm of eigenvalues of parameter matrix
77     """
78     _, S, _ = module.weight.view(module.weight.size(0), -1).svd()
79     return S.norm(p).item()
80
81
82 def dist(module, init_module, p=2, q=2):
83     """
84     Calculates l_pq distance of the parameter matrix of a layer from the random
85     initialization:
86     l_p norm of incoming weights to each hidden unit
87     l_q norm on the hidden units
88     """
89     return (
90         (module.weight - init_module.weight)
91         .view(module.weight.size(0), -1)
92         .norm(p=p, dim=1)
93         .norm(q)
94         .item()
95     )
96
97

```



```

98 def h_dist(module, init_module, p=2, q=2):
99     """
100     Calculate l_pq distance of parameters of trained network from random init
101     Includes extra factor depending on number of hidden units
102     """
103     return (n_hidden(module, init_module) ** (1 - 1 / q)) * dist(
104         module, init_module, p=p, q=q
105     )
106
107
108 def h_dist_op_norm(module, init_module, p=2, q=2, p_op=float("Inf")):
109     """
110     Calculate ratio of h_dist to operator norm
111     """
112     return h_dist(module, init_module, p=p, q=q) / op_norm(module, init_module, p=p_op)
113
114
115 def n_hidden(module, init_module):
116     """
117     Number of hidden units
118     """
119     return module.weight.size(0)
120
121
122 def depth(module, init_module):
123     """
124     Depth (always == 1 for any linear layer)
125     """
126     return 1
127
128
129 def n_param(module, init_module):
130     """
131     Num parameters
132     """
133     bparam = 0 if module.bias is None else module.bias.size(0)
134     return bparam + module.weight.size(0) * module.weight.view(
135         module.weight.size(0), -1
136     ).size(1)
137
138
139 def lp_path_norm(model, device, p=2, input_size=[3, 32, 32]):
140     """
141     Path norm (Neyshabur 2015)
142     """
143
144     tmp_model = copy.deepcopy(model)
145     tmp_model.eval()
146     for param in tmp_model.parameters():
147         if param.requires_grad:
148             param.abs_().pow_(p)
149     data_ones = torch.ones(input_size).to(device)
150     return (tmp_model(data_ones).sum() ** (1 / p)).item()
151
152
153 def calculate(trained_model, init_model, device, dataset_size, margin, input_dim):
154     """
155     Calculates various measures given trained model and model at init

```

```

156 Computes:
157     measures: norm based measures on the model
158     bounds: generalization bounds on the model
159     """
160
161 model = copy.deepcopy(trained_model)
162
163 # depth
164 d = compute_measure(model, init_model, depth, "sum", {})
165
166 # number of parameters (not including batch norm)
167 nparam = compute_measure(model, init_model, n_param, "sum", {})
168
169 measure, bound = {}, {}
170 with torch.no_grad():
171
172     # Compute measures
173     measure["L_{1,inf}_norm"] = (
174         compute_measure(
175             model, init_model, norm, "product", {"p": 1, "q": float("Inf")})
176         )
177         / margin
178     )
179     measure["Frobenius_norm"] = (
180         compute_measure(model, init_model, norm, "product", {"p": 2, "q": 2})
181         / margin
182     )
183     measure["L_{3,1.5}_norm"] = (
184         compute_measure(model, init_model, norm, "product", {"p": 3, "q": 1.5})
185         / margin
186     )
187     measure["Spectral_norm"] = (
188         compute_measure(model, init_model, op_norm, "product", {"p": float("Inf")})
189         / margin
190     )
191     measure["L_{1.5}_operator_norm"] = (
192         compute_measure(model, init_model, op_norm, "product", {"p": 1.5}) / margin
193     )
194     measure["Trace_norm"] = (
195         compute_measure(model, init_model, op_norm, "product", {"p": 1}) / margin
196     )
197
198     # input_size = [context_len, emb_dim]
199     # measure["L1_path norm"] = (
200     #     lp_path_norm(
201     #         model, device, p=1, input_size=input_size
202     #     )
203     #     / margin
204     # )
205     # measure["L1.5_path norm"] = (
206     #     lp_path_norm(
207     #         model, device, p=1.5, input_size=input_size
208     #     )
209     #     / margin
210     # )
211     # measure["L2_path norm"] = (
212     #     lp_path_norm(
213     #         model, device, p=2, input_size=input_size

```

```

214     # )
215     # / margin
216     # )
217
218     # Compute generalization bounds without constant or additive logarithmic factors
219
220     # Golowich 2018
221     # https://arxiv.org/pdf/1712.06541.pdf
222     alpha = math.sqrt(d + math.log(1 * input_dim * input_dim))
223
224     # Bartlett Mendelson 2002
225     bound["L1_max_Bound"] = (
226         alpha * measure["L_{1,inf}_norm"] / math.sqrt(dataset_size)
227     )
228
229     # Neyshabur 2015
230     bound["Frobenius_Bound"] = (
231         alpha * measure["Frobenius_norm"] / math.sqrt(dataset_size)
232     )
233
234     # Neyshabur 2015
235     bound["L_{3,1.5}_Bound"] = (
236         alpha * measure["L_{3,1.5}_norm"] / (dataset_size ** (1 / 3))
237     )
238
239     beta = math.log(dataset_size) * math.log(nparam)
240     ratio = compute_measure(
241         model,
242         init_model,
243         h_dist_op_norm,
244         "norm",
245         {"p": 2, "q": 1, "p_op": float("Inf")},
246         p=2 / 3,
247     )
248
249     # Spectral L_{2, 1} Bound
250     # Bartlett 2017
251     bound["Spec_L_{2,1}_Bound"] = (
252         beta * measure["Spectral_norm"] * ratio / math.sqrt(dataset_size)
253     )
254
255     ratio = compute_measure(
256         model,
257         init_model,
258         h_dist_op_norm,
259         "norm",
260         {"p": 2, "q": 2, "p_op": float("Inf")},
261         p=2,
262     )
263
264     # Spectral Frobenius
265     # Neyshabur 2018
266     # https://arxiv.org/pdf/1706.08947.pdf
267     bound["Spec_Fro_Bound"] = (
268         d * measure["Spectral_norm"] * ratio / math.sqrt(dataset_size)
269     )
270
271     return measure, bound

```

4.1.6 measure.py — Keskar Sharpness (L-BFGS-B)

```

1 import logging
2 import torch
3 import numpy as np
4
5 import scipy.optimize
6
7
8 def get_loss_and_grads(x, model, data_loader):
9
10     # if type(x).__module__ == np.__name__:
11     #     x = torch.from_numpy(x).float()
12     #     x = x.cuda()
13
14     model.eval()
15
16     x_start = 0
17     for p in model.parameters():
18         param_size = p.data.size()
19         param_idx = 1
20         for s in param_size:
21             param_idx *= s
22         x_part = x[x_start : x_start + param_idx]
23         p.data = torch.Tensor(x_part.reshape(param_size))
24         x_start += param_idx
25
26     batch_losses = []
27     batch_grads = []
28     for it, batch in enumerate(data_loader):
29
30         # Move data to correct device
31         # inputs = inputs.to(device)
32         # targets = targets.to(device)
33
34         with torch.set_grad_enabled(True):
35             # loss, grads = model(idx=inputs, targets=targets, grads=True)
36             loss, grads = model._step(batch=batch, batch_idx=1, train=True, grads=True)
37
38         # Todo: average over dataset
39         batch_losses.append(loss)
40         # batch_grads.append(None if grads is None else grads.cpu().numpy().astype(np.
41             float64))
42         batch_grads.append(None if grads is None else grads)
43
44     mean_losses = torch.mean(torch.stack(batch_losses))
45     mean_grads = torch.mean(torch.stack(batch_grads), dim=0)
46
47     return (mean_losses, mean_grads.cpu().numpy().astype(np.float64))
48
49 def get_weights(model):
50     """
51     Given a model, return a vector of weights.
52     """
53     x0 = None
54     for p in model.parameters():
55         if x0 is None:
56             x0 = p.data.view(-1)

```

```

57     else:
58         x0 = torch.cat((x0, p.data.view(-1)))
59     return x0.cpu().numpy()
60
61
62 def get_sharpness(data_loader, model, subspace_dim=10, epsilon=1e-3, maxiter=10):
63     """
64     Compute the sharpness around some point in weight space, as specified
65     in Keskar et. al. (2016) Sec 2.2.2:
66     https://arxiv.org/pdf/1609.04836.pdf
67
68     See:
69     https://gist.github.com/arthurmensch/c55ac413868550f89225a0b9212aa4cd
70     https://gist.github.com/gngdb/a9f912df362a85b37c730154ef3c294b
71     https://github.com/keskarnitish/large-batch-training
72     https://github.com/wenwei202/smoothout
73     https://github.com/keras-team/keras/pull/3064
74     """
75
76     x0 = get_weights(model)
77
78     f_x0, _ = get_loss_and_grads(x0, model, data_loader)
79     f_x0 = -f_x0
80     logging.info("min_loss_x0={loss:.4f}".format(loss=f_x0))
81
82     if 0 == subspace_dim:
83         x_min = np.reshape(x0 - epsilon * (np.abs(x0) + 1), (x0.shape[0], 1))
84         x_max = np.reshape(x0 + epsilon * (np.abs(x0) + 1), (x0.shape[0], 1))
85         bounds = np.concatenate([x_min, x_max], 1)
86         func = lambda x: get_loss_and_grads(x, model, data_loader)
87         init_guess = x0
88     else:
89         assert subspace_dim <= x0.shape[0]
90
91         # Computed via Keskar, et. al
92         # https://arxiv.org/pdf/1609.04836.pdf
93
94         A_plus = np.random.rand(subspace_dim, x0.shape[0]) * 2.0 - 1.0
95         A_plus_norm = np.linalg.norm(A_plus, axis=1)
96         A_plus = A_plus / np.reshape(A_plus_norm, (subspace_dim, 1))
97         A = np.linalg.pinv(A_plus)
98
99         abs_bound = epsilon * (np.abs(np.dot(A_plus, x0)) + 1)
100        abs_bound = np.reshape(abs_bound, (abs_bound.shape[0], 1))
101        bounds = np.concatenate([-abs_bound, abs_bound], 1)
102
103        def func(y):
104            f_loss, f_grads = get_loss_and_grads(
105                x0 + np.dot(A, y),
106                model,
107                data_loader,
108            )
109            return f_loss, np.dot(np.transpose(A), f_grads)
110
111        init_guess = np.zeros(subspace_dim)
112
113        minimum_x, f_x, d = scipy.optimize.fmin_l_bfgs_b(
114            func,

```

```

115         init_guess,
116         maxiter=maxiter,
117         bounds=bounds,
118         disp=1,
119     )
120     f_x = -f_x
121     logging.info("max_loss_f_x={loss:.4f}".format(loss=f_x))
122
123     # Eq 4 in Keskar
124     phi = (f_x - f_x0) / (1 + f_x0) * 100
125
126     # Restore parameter values
127     x0 = torch.from_numpy(x0).float()
128     # x0 = x0.cuda()
129     x_start = 0
130     for p in model.parameters():
131         param_size = p.data.size()
132         param_idx = 1
133         for s in param_size:
134             param_idx *= s
135         x_part = x0[x_start : x_start + param_idx]
136         p.data = x_part.view(param_size)
137         x_start += param_idx
138
139     return phi

```

4.1.7 visualization.py — Plotting Utilities

```

1 import csv
2 import logging
3 import os
4 import math
5 import socket
6
7 from collections import defaultdict
8 from copy import deepcopy
9
10 import matplotlib.pyplot as plt
11 import matplotlib.ticker as mtick
12 import numpy as np
13 import torch
14
15 from mpl_toolkits.axes_grid1 import make_axes_locatable
16 from tqdm import tqdm
17
18 from grok.data import ArithmeticDataset
19
20 logging.basicConfig(level=logging.ERROR)
21 logger = logging.getLogger("grok.view_metrics")
22 logger.setLevel(logging.ERROR)
23
24 GROK_DIR = os.path.expanduser("~/data/grok")
25 IMAGE_DIR = f"{GROK_DIR}/images"
26 DATA_DIR = f"{GROK_DIR}/data"
27
28
29 DEFAULT_CMAP = "viridis"
30

```

```

31 default_metric_limits = {
32     "min_val_accuracy": 0,
33     "max_val_accuracy": 100,
34     "min_T": 0, # 0
35     "max_T": 100, # 87.5
36     "min_D": 0, # 8
37     "max_D": 2048, # 256
38     "min_H": 0, # 1
39     "max_H": 1204, # 8
40     "min_L": 0, # 1
41     "max_L": 1024, # 4
42     "min_accuracy": 0,
43     "max_accuracy": 100,
44 }
45
46 default_axis_scales = {"x": "linear", "y": "linear"}
47
48
49 ## Data Loading Functions
50
51
52 def factor_expts(expts):
53     result = {}
54     for expt in expts:
55         expt_s = expt.split("_")
56         arch = "_".join(expt_s[:3])
57         t = int(float(expt_s[3].split("-")[1]))
58         result.setdefault(arch, {})
59         result[arch][t] = expt
60     return result
61
62
63 def load_metric_data(data_dir, epochs=100000, load_partial_data=True):
64     # layers x heads x d_model x train_pct
65     data = {}
66     expts = os.listdir(data_dir)
67     archs = factor_expts(expts)
68     logger.debug(archs)
69     for arch in archs:
70         T = sorted(archs[arch].keys())
71         data[arch] = {
72             "T": torch.LongTensor(T),
73             "metrics": torch.zeros((max(T), 5, epochs)),
74         }
75     # print(f"metrics_shape = {data[arch]['metrics'].shape}")
76     for i, t in tqdm(list(enumerate(T))):
77         expt = archs[arch][t]
78         logger.debug(expt)
79         log_dir = data_dir + "/" + expt
80
81         # print("log_dir", log_dir)
82         try:
83             with open(log_dir + "/default/version_0/metrics.csv", "r") as fh:
84                 logger.debug(f"loading_{log_dir}")
85                 reader = list(csv.DictReader(fh))
86                 val_t = torch.FloatTensor(
87                     [
88

```

```

89         float(r["val_loss"]),
90         float(r["val_accuracy"]),
91     ]
92     for r in reader
93         if r["val_loss"]
94     ]
95     ).T
96     train_t = torch.FloatTensor(
97     [
98         [
99             float(r["learning_rate"]),
100             float(r["train_loss"]),
101             float(r["train_accuracy"]),
102         ]
103         for r in reader
104         if r["train_loss"]
105     ]
106     ).T
107     # logger.debug(val_t.shape)
108     # logger.debug(train_t[0, -3:])
109     if load_partial_data:
110         raise Exception("Not implemented")
111     elif (val_t.shape[-1] >= epochs) and (train_t.shape[-1] >= epochs):
112         data[arch]["metrics"][i] = torch.cat(
113             [train_t[... , :epochs], val_t[... , :epochs]], dim=0
114         )
115     else:
116         data[arch]["T"][i] = 0
117     # except FileNotFoundError:
118     except:
119         data[arch]["T"][i] = 0
120     indices = torch.nonzero(data[arch]["T"]).squeeze()
121     if len(indices.shape) == 0:
122         indices = indices.unsqueeze(0)
123     # print(f"indices.shape = {indices.shape}")
124     data[arch]["T"] = data[arch]["T"][indices]
125     # print(f"data[arch]['T'].shape = {data[arch]['T'].shape}")
126     data[arch]["metrics"] = data[arch]["metrics"][indices]
127     # print(f"data[arch]['metrics'].shape = {data[arch]['metrics'].shape}")
128     data[arch]["metrics"] = torch.transpose(data[arch]["metrics"], 0, 1)
129     # print(f"data[arch]['metrics'].shape = {data[arch]['metrics'].shape}")
130     return data
131
132
133 def most_interesting(metric_data):
134     interesting_metric_data = {}
135     for arch in metric_data:
136         T = metric_data[arch]["T"]
137         max_acc_by_t = torch.max(
138             metric_data[arch]["val_accuracy"], dim=1, keepdim=True
139         ).values.squeeze()
140         max_loss_by_t = torch.max(
141             metric_data[arch]["val_loss"], dim=1, keepdim=True
142         ).values.squeeze()
143         acc_idx = torch.nonzero(max_acc_by_t >= 95).squeeze()
144         if acc_idx.shape == torch.Size([0]):
145             acc_idx = torch.nonzero(max_acc_by_t == max_acc_by_t.max()).squeeze()
146         if acc_idx.shape == torch.Size([0]):

```



```

147         acc_idx = acc_idx.unsqueeze(0)
148         max_loss = torch.max(max_loss_by_t[acc_idx])
149         loss_idx = torch.nonzero(max_loss_by_t[acc_idx] == max_loss)
150         interesting_idx = acc_idx[loss_idx].squeeze()
151
152         interesting_metric_data[arch] = {}
153         for k in metric_data[arch]:
154             interesting_metric_data[arch][k] = metric_data[arch][k][
155                 interesting_idx
156             ].unsqueeze(0)
157
158         return interesting_metric_data
159
160
161 # ## Graph Drawing Functions
162
163
164 def moving_avg(Y, steps):
165     return np.convolve(Y, np.ones(steps), "valid") / steps
166
167
168 def find_inflections(Y, smoothing_steps=100):
169     avg_Y = moving_avg(Y, smoothing_steps)
170     avg_direction = torch.FloatTensor(np.sign(avg_Y[1:] - avg_Y[:-1]))
171     avg_direction = torch.cat([avg_direction[0].unsqueeze(0), avg_direction])
172     avg_inflections = torch.nonzero(avg_direction[1:] - avg_direction[:-1]).squeeze()
173     avg_inflections = [0] + (avg_inflections + 1).tolist() + [len(Y) - 1]
174     logger.debug(f"avg_inflections={avg_inflections}")
175     inflections = []
176     for i in range(2, len(avg_inflections)):
177         low = avg_inflections[i - 2]
178         high = avg_inflections[i]
179         logger.debug(f"low={low}")
180         logger.debug(f"high={high}")
181         if avg_direction[low + 1] < 0:
182             indices = Y[low:high].argmin() + low
183             logger.debug(f"min={Y[{indices}]}={Y[int(indices)]}")
184         else:
185             indices = Y[low:high].argmax() + low
186             logger.debug(f"max={Y[{indices}]}={Y[int(indices)]}")
187         inflections.append(indices)
188     return torch.LongTensor(inflections)
189
190
191 def check_limits(arch_name, limits):
192     L, H, D = [float(v.split("-")[1]) for v in arch_name.split("_")]
193     if (L > limits["max_L"]) or (L < limits["min_L"]):
194         return False
195     if (H > limits["max_H"]) or (H < limits["min_H"]):
196         return False
197     if (D > limits["max_D"]) or (D < limits["min_D"]):
198         return False
199     # if (T > limits['max_T']) or (T < limits['min_T']):
200     #     return False
201     return True
202
203
204 def filter_archs(data, limits={}):

```

```

205 my_limits = deepcopy(default_metric_limits)
206 my_limits.update(limits)
207 limits = my_limits
208 archs = sorted(list(set([a for a in data.keys() if check_limits(a, limits)])))
209 logger.debug(f"archs={archs}")
210 return archs
211
212
213 def get_metric_data(data, limits={}):
214     my_limits = deepcopy(default_metric_limits)
215     my_limits.update(limits)
216     limits = my_limits
217
218     for k in limits.keys():
219         metric = k.replace("min_", "").replace("max_", "")
220         assert (
221             limits["max_" + metric] >= limits["min_" + metric]
222         ), f"invalid_{metric}_limits"
223
224     d = {}
225     for arch in filter_archs(data, limits):
226         logger.debug(arch)
227         indices = torch.nonzero(
228             torch.logical_and(
229                 data[arch]["T"] >= limits["min_T"], data[arch]["T"] <= limits["max_T"]
230             )
231         ).squeeze(dim=-1)
232         logger.debug(f"indices={indices}")
233         learning_rate, train_loss, train_accuracy, val_loss, val_accuracy = data[arch][
234             "metrics"
235         ][:, indices, :]
236         d[arch] = {
237             "T": data[arch]["T"][indices],
238             "learning_rate": data[arch]["metrics"][0, indices, :],
239             "train_loss": data[arch]["metrics"][1, indices, :],
240             "train_accuracy": data[arch]["metrics"][2, indices, :],
241             "val_loss": data[arch]["metrics"][3, indices, :],
242             "val_accuracy": data[arch]["metrics"][4, indices, :],
243         }
244     return d
245
246
247 def add_metric_graph(
248     fig,
249     ax,
250     metric,
251     metric_data,
252     scales=default_axis_scales,
253     cmap=DEFAULT_CMAP,
254     inflection_hline=False,
255     ds_len=None,
256     batchsize=97,
257 ):
258     ax.set_title(metric)
259     ax.set_xscale(scales["x"])
260     ax.set_yscale(scales["y"])
261     if ds_len is None:
262         ax.set_xlabel("epochs")

```

```

263 else:
264     ax.set_xlabel("updates")
265
266     # if 'loss' in metric:
267     #     ymin=0
268     #     ax.axis(ymin=ymin)
269     if "accuracy" in metric:
270         ax.yaxis.set_major_formatter(mtick.PercentFormatter())
271         ymin = 1e-16
272         ymax = 101
273         ax.axis(ymin=ymin, ymax=ymax)
274     if "loss" in metric:
275         ymin = 1e-16
276         ymax = 15
277         ax.axis(ymin=ymin, ymax=ymax)
278
279     total_plots = 0
280     logger.debug(f"processing_{metric}")
281     plots = []
282     for arch in metric_data:
283         metric_data[arch]["T"] = metric_data[arch]["T"].squeeze()
284         logger.debug((" " * 4) + f"arch_{arch}")
285         if len(metric_data[arch]["T"].shape) == 0:
286             metric_data[arch]["T"] = metric_data[arch]["T"].unsqueeze(0)
287         T_min = int(metric_data[arch]["T"][0])
288         T_max = int(metric_data[arch]["T"][-1])
289         # T_min = 0
290         # T_max = 88
291         sm = plt.cm.ScalarMappable(
292             cmap=cmap, norm=plt.Normalize(vmin=T_min, vmax=T_max)
293         )
294         colors = sm.to_rgba(metric_data[arch]["T"])
295         for i, t in enumerate(metric_data[arch]["T"]):
296             if ds_len is None:
297                 steps_per_epoch = 1
298             else:
299                 train_rows, val_rows = ArithmeticDataset.calc_split_len(
300                     t.item(), ds_len
301                 )
302                 steps_per_epoch = math.ceil(train_rows / batchsize)
303
304             logger.debug((" " * 4) + f"t_{t}")
305             # print(
306             #     f"metric_data[arch][metric].shape = {metric_data[arch][metric].shape}"
307             # )
308             Y = metric_data[arch][metric][i]
309             # print(f"Y = {Y}")
310             assert len(Y.shape) == 1, f"Y.shape_{Y.shape}_is_invalid"
311             X = torch.arange(1, Y.shape[0] + 1) * steps_per_epoch
312             assert len(X.shape) == 1, f"X.shape_{X.shape}_is_invalid"
313
314             label = arch + f"_{t}"
315
316             # ax.set_xlim(left=X[0], right=X[-1] + 1)
317             if metric == "val_loss" and inflection_hline:
318                 Y_infs = find_inflections(Y)
319                 ax.axhline(y=Y[Y_infs[0]], color="orange")
320             if metric == "val_accuracy":

```

```

321         label += "␣(max␣=␣%.2f)" % max(Y)
322         total_plots += 1
323         ax.plot(X, Y, label=label, color=colors[i])
324     if T_max - T_min <= 10:
325         pass
326         ax.legend()
327     else:
328         fig.colorbar(
329             sm,
330             ax=ax,
331             label="%␣training␣data",
332             ticks=range(T_min, T_max, int((T_max - T_min) / 5)),
333         )
334
335
336 def add_comm_graph(
337     ax, metric, kind, comm_data, arch, scales=default_axis_scales, cmap=DEFAULT_CMAP
338 ):
339     assert metric in (
340         "loss",
341         "accuracy",
342         "perplexity",
343     )
344     assert kind in (
345         "comm",
346         "non_comm",
347         "modulo",
348         "non_modulo",
349         "assoc",
350         "non_assoc",
351         "zero",
352         "non_zero",
353     )
354     ax.set_title(metric)
355     ax.set_xscale(scales["x"])
356     ax.set_yscale(scales["y"])
357     ax.set_xlabel("epochs")
358     if "accuracy" in metric:
359         ax.yaxis.set_major_formatter(mtick.PercentFormatter())
360     X = [int(r["epoch"]) for r in comm_data]
361     Y = torch.tensor(
362         (
363             [float(r["comm"] + "␣" + metric) for r in comm_data],
364             [float(r["non_comm"] + "␣" + metric) for r in comm_data],
365             # [float(r["assoc"] + "␣" + metric) for r in comm_data],
366             # [float(r["non_assoc"] + "␣" + metric) for r in comm_data],
367             # [float(r["zero"] + "␣" + metric) for r in comm_data],
368             # [float(r["non_zero"] + "␣" + metric) for r in comm_data],
369         )
370     )
371     # label = kind
372     # if kind.endswith("comm"):
373     #     label += "utative"
374
375     labels = ["commutative", "non-commutative"]
376     # labels = ["zero", "non_zero"]
377     # labels = ["associative", "non_associative"]
378     # label = f"{arch} {kind}_ {metric}"

```

```

379 # ax.plot(X, Y, label=label)
380 sm = plt.cm.ScalarMappable(cmap="cividis", norm=plt.Normalize(vmin=0, vmax=len(Y)))
381 colors = sm.to_rgba(range(len(Y)))
382 ax.stackplot(X, Y, baseline="zero", labels=labels, colors=colors)
383 ax.legend()
384
385
386 def add_extremum_graph(
387     ax,
388     metric,
389     kind,
390     metric_data,
391     scales=default_axis_scales,
392     epochs=[-1],
393     show_legend=True,
394 ):
395     assert kind in ("max", "min")
396     ax.set_title(f"{kind}_{metric}")
397     ax.set_xlabel("training_data")
398     ax.xaxis.set_major_formatter(mtick.PercentFormatter())
399     xmin = 0
400     xmax = 100
401     ax.axis(xmin=xmin, xmax=xmax)
402
403     # ax.set_ylabel(metric)
404     ax.set_xscale(scales["x"])
405     ax.set_yscale(scales["y"])
406     if "accuracy" in metric:
407         ax.yaxis.set_major_formatter(mtick.PercentFormatter())
408         ymin = -1
409         ymax = 105
410         ax.axis(ymin=ymin, ymax=ymax)
411
412     # if 'learning' in metric:
413     #     ymin=0
414     #     ymax=0.002
415     #     ax.axis(ymin=ymin, ymax=ymax)
416
417     plots = {}
418
419     total_plots = 0
420     for arch in metric_data:
421         X = metric_data[arch]["T"]
422         if kind == "max":
423             Y = torch.max(
424                 metric_data[arch][metric], dim=1, keepdim=True
425             ).values.squeeze()
426         elif kind == "min":
427             Y = torch.min(
428                 metric_data[arch][metric], dim=1, keepdim=True
429             ).values.squeeze()
430
431         # ax.set_xlim(0, 100)
432         ax.set_xticks(np.arange(0, 100, 5))
433         label = f"{kind}_{metric}_{arch}"
434         ax.plot(X, Y, label=label)
435         total_plots += 1
436

```

```

437     if show_legend and total_plots <= 12:
438         ax.legend()
439         pass
440
441
442 def add_inflection_graphs(
443     ax, metric, metric_data, scales=default_axis_scales, smoothing_steps=100
444 ):
445     ax.set_title(f"{metric}_inflections_by_train_data_pct")
446     ax.set_xlabel("train_data_pct")
447     ax.set_ylabel(f"{metric}_inflections")
448     ax.set_xscale(scales["x"])
449     ax.set_yscale(scales["y"])
450     if "accuracy" in metric:
451         ymin = 0
452         ymax = 100
453         ax.axis(xmin=0, xmax=87.5, ymin=ymin, ymax=ymax)
454     if "learning" in metric:
455         ymin = 0
456         ymax = 0.002
457         ax.axis(xmin=0, xmax=87.5, ymin=ymin, ymax=ymax)
458
459     total_plots = 0
460     for arch in metric_data:
461         for num in range(5):
462             for i, t in enumerate(metric_data[arch]["T"]):
463                 Y = metric_data[arch][metric][i]
464                 X = torch.arange(Y.shape[-1])
465                 inflections = find_inflections(Y, smoothing_steps=smoothing_steps)
466                 ax.plot(X[inflections], Y[inflections], label=f"{arch}_t={t}")
467                 total_plots += 1
468
469     if total_plots <= 12:
470         ax.legend()
471         pass
472
473
474 def colorbar(mappable, ticks=None, labels=None):
475     last_axes = plt.gca()
476     ax = mappable.axes
477     fig = ax.figure
478     divider = make_axes_locatable(ax)
479     cax = divider.append_axes("right", size="5%", pad=0.1)
480     cbar = fig.colorbar(mappable, cax=cax, ticks=ticks)
481     if labels is not None:
482         cbar.ax.set_yticklabels(labels) # vertically oriented colorbar
483     plt.sca(last_axes)
484     return cbar
485
486
487 def add_matshow(
488     fig, ax, t, name, vmin=0, vmax=100, cmap=DEFAULT_CMAP, show_colorbar=True
489 ):
490     sides = ("left", "right", "top", "bottom")
491     labels = {
492         "left": True,
493         "right": False,
494         "top": False,

```

```
495     "bottom": True,
496     "labelleft": True,
497     "labelright": False,
498     "labeltop": False,
499     "labelbottom": True,
500 }
501 m = ax.matshow(
502     t.cpu().detach().numpy(), vmin=vmin, vmax=vmax, origin="lower", cmap=cmap
503 )
504 # c = ax.pcolor(t.cpu(), vmin=vmin, vmax=vmax, cmap=cmap)
505 ax.set_title(name)
506 ax.set_xlabel("A")
507 ax.set_ylabel("B")
508 ax.set_xticks(np.arange(0, t.shape[1], 10))
509 # ax.set_xticklabels(np.arange(1, t.shape[1]+1))
510 # ax.set_yticks(np.arange(0.5, t.shape[0] + .5, 1))
511 ax.set_yticks(np.arange(0, t.shape[0], 10))
512 # ax.set_yticks(np.arange(t.shape[0]))
513 # ax.set_yticklabels(np.arange(1, t.shape[0]+1))
514 ax.tick_params(axis="both", which="both", **labels)
515 if show_colorbar:
516     colorbar(m)
```

4.2 Grokking Scripts: `grokking_nc_rf/scripts/`

4.2.1 `make_data.py`

```
1 #!/usr/bin/env python
2
3 from argparse import ArgumentParser
4 from grok.data import create_data_files, DEFAULT_DATA_DIR
5
6
7 parser = ArgumentParser()
8 parser.add_argument("-d", "--data_directory", type=str, default=DEFAULT_DATA_DIR)
9 args = parser.parse_args()
10 create_data_files(args.data_directory)
```

4.2.2 `train.py`

```
1 #!/usr/bin/env python
2
3 import grok
4 import os
5 from datetime import datetime
6 start_time = datetime.now()
7
8 parser = grok.training.add_args()
9 parser.set_defaults(logdir=os.environ.get("GROK_LOGDIR", "."))
10 hparams = parser.parse_args()
11 hparams.datadir = os.path.abspath(hparams.datadir)
12 hparams.logdir = os.path.abspath(hparams.logdir)
13
14
15 print(hparams)
16 print(grok.training.train(hparams))
17 end_time = datetime.now()
18 print('Duration:␣{}'.format(end_time - start_time))
19 print("TRAINING_COMPLETE")
```

4.2.3 `compute_sharpness.py`

```
1 #!/usr/bin/env python
2
3 import os
4 import grok
5
6 parser = grok.training.add_args()
7 parser.set_defaults(logdir=os.environ.get("LOGDIR", "."))
8 hparams = parser.parse_args()
9 hparams.datadir = os.path.abspath(hparams.datadir)
10 hparams.logdir = os.path.abspath(hparams.logdir)
11
12
13 print(hparams)
14
15 ckpts = [f"./ckpts/L-2_H-4_D-128_T-70_DROP-0_SD-{i}_WU-10_LR-1p0.ckpt" for i in range(20)]
16 print(grok.training.compute_sharpness(hparams, ckpts))
```


4.2.4 create_partial_metrics.py

```

1 #!/usr/bin/env python
2
3  import logging
4
5  logging.basicConfig(level=logging.ERROR)
6  import csv
7  import copy
8  import glob
9  import os
10 import grok
11 import numpy as np
12 import subprocess
13 import torch
14 import sys
15 from torch.multiprocessing import Process
16 from grok import trainer
17 from tqdm import tqdm
18 from argparse import ArgumentParser
19 from collections import Counter
20 from grok_runs import RUNS
21 from grok_metrics_lib import (
22     DATA_DIR,
23     load_metric_data,
24     get_metric_data,
25     most_interesting,
26 )
27
28
29 # Make N_EPOCHS exponentially spaced sets of epochs from 1 to 10,000
30 N_EPOCHS = 32
31 BASE = 9999 ** (1.0 / (N_EPOCHS - 1))
32 epochs = (BASE ** torch.arange(1, N_EPOCHS).float()).long().tolist()
33 DEFAULT_EPOCHS = ",".join([str(i) for i in epochs])
34
35 parser = ArgumentParser()
36 parser.add_argument("--expt_dir", type=str, help="where to find the runs")
37 parser.add_argument("--epochs", type=str, default=DEFAULT_EPOCHS)
38
39
40 def child(hparams):
41     expt_dir = hparams.expt_dir
42     epochs = [int(e) for e in hparams.epochs.split(",")]
43     # print("epochs = ", epochs)
44     device = torch.device(f"cuda:{hparams.gpu}")
45     ckpt_dir = expt_dir + "/" + "checkpoints"
46     # ckpt_files = [ckpt_dir + f"/epoch={epoch}.ckpt" for epoch in epochs]
47     hparams.logdir = expt_dir
48
49     results = {
50         "val_loss": None,
51         "val_accuracy": None,
52     }
53
54     processed_epochs = []
55     # with tqdm(epochs, unit="epochs", initial=epochs[0], total=epochs[-1]) as pbar:
56     #     last_epoch = epochs[0]
57     for idx, epoch in tqdm(list(enumerate(epochs))):

```

```

58     # pbar.update(epoch - last_epoch)
59     # last_epoch = epoch
60     ckpt_files = glob.glob(ckpt_dir + f"/epoch={epoch}-step=*.ckpt")
61     ckpt_files += glob.glob(ckpt_dir + f"/epoch={epoch}.ckpt")
62     try:
63         ckpt_file = ckpt_files[-1]
64         ckpt = torch.load(
65             ckpt_file,
66             map_location=f"cuda:{0}", # FIXME
67         )
68         processed_epochs.append(epoch)
69     except FileNotFoundError:
70         continue
71
72     for k, v in ckpt["hyper_parameters"].items():
73         setattr(hparams, k, v)
74
75     new_state_dict = {}
76     for k, v in ckpt["state_dict"].items():
77         if k.startswith("transformer."):
78             new_state_dict[k] = v
79         else:
80             new_state_dict["transformer." + k] = v
81     ckpt["state_dict"] = new_state_dict
82
83     model = trainer.TrainableTransformer(hparams).float()
84     model.load_state_dict(ckpt["state_dict"])
85     model = model.to(device).eval()
86     dl = model.test_dataloader()
87     dl.reset_iteration(shuffle=False)
88
89     outputs = [model.test_step(batch, idx) for (idx, batch) in enumerate(dl)]
90     r = model.test_epoch_end(outputs)["log"]
91     if results["val_loss"] is None:
92         results["val_loss"] = r["test_loss"].squeeze().unsqueeze(0)
93         results["val_accuracy"] = r["test_accuracy"].squeeze().unsqueeze(0)
94     else:
95         results["val_loss"] = torch.cat(
96             [results["val_loss"], r["test_loss"].squeeze().unsqueeze(0)], dim=0
97         )
98         results["val_accuracy"] = torch.cat(
99             [
100                 results["val_accuracy"],
101                 r["test_accuracy"].squeeze().unsqueeze(0),
102             ],
103             dim=0,
104         )
105
106     for k, v in results.items():
107         results[k] = v.to("cpu")
108     results["epochs"] = torch.LongTensor(processed_epochs, device="cpu")
109     results["dl"] = dl
110
111     os.makedirs(expt_dir + "/activations", exist_ok=True)
112     ptfile = (
113         expt_dir + f"/activations/activations_{epochs[0]:010d}_{epochs[-1]:010d}.pt"
114     )
115     torch.save(results, ptfile)

```

```

116
117
118 if __name__ == "__main__":
119     hparams = trainer.get_args(parser)
120     if hparams.expt_dir is not None:
121         child(hparams)
122     else:
123         for operation in RUNS:
124             print(f"running_{operation}")
125             ds_len, run = RUNS[operation]
126             data = load_metric_data(
127                 f"{DATA_DIR}/{run}", epochs=10000, load_partial_data=False
128             )
129             metric_data = get_metric_data(data)
130             metric_data = most_interesting(metric_data)
131             for arch in metric_data:
132                 interesting_t = int(metric_data[arch]["T"][0].item())
133                 expt = f"{arch}_T-{interesting_t}"
134                 print(f"-->_expt_{expt}")
135                 glb = f"{DATA_DIR}/{run}/{expt}_"
136                 # print(f"glb {glb}")
137                 expt_dir = glob.glob(glb)[0]
138                 cmd = [sys.argv[0], "--expt_dir", expt_dir]
139                 subprocess.run(cmd, check=False, shell=False)
140                 # child(hparams)

```

4.2.5 create_metrics_for_epochs.py

```

1 #!/usr/bin/env python
2
3 import logging
4
5 logging.basicConfig(level=logging.ERROR)
6 import csv
7 import copy
8 import os
9 import grok
10 import numpy as np
11 import sys
12 import subprocess
13 import torch
14 from torch.multiprocessing import Process
15 from grok import trainer
16 from tqdm import tqdm
17 from argparse import ArgumentParser
18 from collections import Counter
19
20
21 torch.multiprocessing.freeze_support()
22 try:
23     torch.multiprocessing.set_start_method("spawn")
24 except RuntimeError:
25     pass
26
27 # Get args
28 EPOCHS = (
29     list(range(10))
30     + list(range(10, 200, 2))

```

```

31     + list(range(200, 5000, 10))
32     + list(range(5000, 10000, 50))
33     + [10000]
34 )
35
36 parser = ArgumentParser()
37 parser.add_argument(
38     "--data_dir", type=str, help="where to find the runs", required=True
39 )
40 parser.add_argument("--expt", type=str, default=None)
41 parser.add_argument("--epochs_per_run", type=int, default=40)
42
43
44 def parent(expts):
45     for expt in expts:
46         print(f"Processing {expt}")
47         all_results = {}
48         for first_epoch in range(0, len(EPOCHS), hparams.epochs_per_run):
49             these_epochs = [
50                 str(e)
51                 for e in EPOCHS[first_epoch : first_epoch + hparams.epochs_per_run]
52             ]
53             expt_dir = data_dir + "/" + expt
54             cmd = [
55                 "./create_partial_metrics.py",
56                 f"--gpu={hparams.gpu}",
57                 f"--expt_dir={expt_dir}",
58                 f"--epochs={','.join(these_epochs)}",
59             ]
60             result = subprocess.run(cmd, capture_output=False, shell=False)
61             if result.returncode != 0:
62                 sys.exit(result.returncode)
63
64
65 hparams = trainer.get_args(parser)
66
67 data_dir = hparams.data_dir
68
69 if hparams.expt is not None:
70     expts = [hparams.expt]
71 else:
72     expts = os.listdir(data_dir)
73
74 parent(expts)

```

4.2.6 visualize_metrics.py

```

1  #!/usr/bin/env python
2  # coding: utf-8
3  import csv
4  import json
5  import logging
6  import os
7  import subprocess
8  from argparse import ArgumentParser
9  from copy import deepcopy
10 from glob import glob
11 from pprint import pprint

```

```
12
13 import blobfile as bf
14 import grok
15 import matplotlib.pyplot as plt
16 import matplotlib.ticker as mtick
17 import numpy as np
18 import torch
19 import yaml
20 from tqdm import tqdm
21
22 logger = logging.getLogger(__name__)
23
24 # take args: input_dir output_dir
25 parser = ArgumentParser()
26 parser.add_argument(
27     "-i",
28     "--input_dir",
29     type=str,
30     required=True,
31 )
32 parser.add_argument(
33     "-o",
34     "--output_dir",
35     type=str,
36     required=True,
37 )
38 parser = grok.training.add_args(parser)
39 args = parser.parse_args()
40 print(args, flush=True)
41
42 if torch.cuda.is_available():
43     device = "cuda"
44 else:
45     device = "cpu"
46
47
48 def load_expt_metrics(
49     expt_dir,
50     args,
51 ):
52     """load the metrics for one experiment"""
53     args = deepcopy(args)
54
55     # load the hparams for this experiment
56     with open(f"{expt_dir}/default/version_0/hparams.yaml", "r") as fh:
57         hparams_dict = yaml.safe_load(fh)
58
59     for k, v in hparams_dict.items():
60         setattr(args, k, v)
61
62     # load the summarized validation and training data for every epoch
63     val_data = {
64         "step": [],
65         "epoch": [],
66         "val_loss": [],
67         "val_accuracy": [],
68     }
69     train_data = {
```

```

70     "step": [],
71     "epoch": [],
72     "train_loss": [],
73     "train_accuracy": [],
74     "learning_rate": [],
75 }
76
77 with open(f"{expt_dir}/default/version_0/metrics.csv", "r") as fh:
78     for row in csv.DictReader(fh):
79         if row["train_loss"] != "":
80             for k in train_data:
81                 if k in ["step", "epoch"]:
82                     v = int(row[k])
83                 else:
84                     v = float(row[k])
85                 train_data[k].append(v)
86         else:
87             for k in val_data:
88                 if k in ["step", "epoch"]:
89                     v = int(row[k])
90                 else:
91                     v = float(row[k])
92                 val_data[k].append(v)
93
94     return {
95         "hparams": hparams_dict,
96         "train": train_data,
97         "val": val_data,
98         # "raw": raw_data,
99     }
100
101
102 def load_run_metrics(
103     run_dir,
104     args=args,
105 ):
106     """load all the metrics for a collection of experiments with the same architecture
107     across various amounts of training data"""
108     metric_data = {}
109     from os import walk
110
111     _, expt_dirs, _ = next(os.walk(run_dir))
112     for expt_dir in tqdm(expt_dirs, unit="expt"):
113         try:
114             expt_data = load_expt_metrics(f"{run_dir}/{expt_dir}", args)
115             train_data_pct = expt_data["hparams"]["train_data_pct"]
116             metric_data[train_data_pct] = expt_data
117         except FileNotFoundError:
118             pass
119     return metric_data
120
121
122 def add_metric_graph(
123     fig,
124     ax,
125     arch,
126     metric,
127     metric_data,

```

```

128     scales,
129     cmap="viridis",
130     by="step", # step or epoch
131     max_increment=0,
132 ):
133     ax.set_title(metric)
134     ax.set_xscale(scales["x"])
135     ax.set_yscale(scales["y"])
136     ax.set_xlabel(by)
137
138     if "accuracy" in metric:
139         ax.yaxis.set_major_formatter(mtick.PercentFormatter())
140         ymin = 1e-16
141         ymax = 101
142         ax.axis(ymin=ymin, ymax=ymax)
143     if "loss" in metric:
144         ymin = 1e-16
145         ymax = 15
146         ax.axis(ymin=ymin, ymax=ymax)
147
148     total_plots = 0
149     logger.debug(f"processing_{metric}")
150     plots = []
151     T = list(sorted(metric_data.keys()))
152     T_max = int(T[-1])
153     T_min = int(T[0])
154     sm = plt.cm.ScalarMappable(cmap=cmap, norm=plt.Normalize(vmin=T[0], vmax=T[-1]))
155     colors = sm.to_rgba(T)
156     for i, t in enumerate(T):
157         if "val" in metric:
158             this_data = metric_data[t]["val"]
159         else:
160             this_data = metric_data[t]["train"]
161
162         X = this_data[by]
163         Y = this_data[metric]
164         if max_increment > 0:
165             X = [x for x in X if x <= max_increment]
166             Y = Y[: len(X)]
167
168         if len(X) != len(Y):
169             logger.warning(f"Mismatched_data:_{metric}_at_t={t}")
170             continue
171         if not Y:
172             logger.warning(f"No_data_for_{metric}_at_t={t}")
173             continue
174
175         label = arch + f"_t={t}"
176
177         if "accuracy" in metric:
178             label += "_ (max_=%%.2f)" % max(Y)
179         elif "loss" in metric:
180             label += "_ (min_=%%.2f)" % min(Y)
181         total_plots += 1
182         ax.plot(X, Y, label=label, color=colors[i])
183     if T_max - T_min <= 10:
184         ax.legend()
185     else:

```

```

186         fig.colorbar(
187             sm,
188             ax=ax,
189             label="%_training_data",
190             ticks=range(T_min, T_max + 1, int((T_max - T_min) / 5)),
191         )
192
193
194 def add_max_accuracy_graph(
195     ax,
196     arch,
197     metric,
198     metric_data,
199     scales,
200     by="step",
201     max_increment=0,
202 ):
203     ax.set_title(f"max_{metric}")
204     ax.set_xlabel("%_of_total_data_trained_on")
205     ax.xaxis.set_major_formatter(mtick.PercentFormatter())
206     xmin = 0
207     xmax = 100
208     ymin = 1e-16
209     ymax = 101
210     ax.axis(xmin=xmin, xmax=xmax, ymin=ymin, ymax=ymax)
211     ax.set_xscale(scales["x"])
212     ax.set_yscale(scales["y"])
213     ax.yaxis.set_major_formatter(mtick.PercentFormatter())
214     ax.xaxis.set_major_formatter(mtick.PercentFormatter())
215
216     T = list(sorted(metric_data.keys()))
217     T_max = int(T[-1])
218     T_min = int(T[0])
219     Y = []
220     for i, t in enumerate(T):
221         if "val" in metric:
222             this_data = metric_data[t]["val"]
223         else:
224             this_data = metric_data[t]["train"]
225         X = this_data[by]
226         if max_increment > 0:
227             X = [x for x in X if x <= max_increment]
228             max_idx = len(X)
229         else:
230             max_idx = -1
231         try:
232             Y.append(max(this_data[metric][:max_idx]))
233         except ValueError:
234             Y.append(np.nan)
235
236     ax.set_xticks(np.arange(0, 100, 5))
237     label = f"max_{metric}_{arch}"
238     ax.plot(T, Y, label=label)
239
240
241 def create_loss_curves(
242     metric_data,
243     arch,

```



```
244     operation,
245     # epochs,
246     most_interesting_only=False,
247     image_dir=args.output_dir,
248     by="step",
249     max_increment=0,
250     cmap="viridis",
251 ):
252     scales = {
253         "x": "log",
254         "y": "linear",
255     }
256
257     ncols = 2
258     nrows = 3
259     fig_width = ncols * 8
260     fig_height = nrows * 5
261     fig, axs = plt.subplots(nrows=nrows, ncols=ncols, figsize=(fig_width, fig_height))
262
263     add_metric_graph(
264         fig,
265         axs[0, 0],
266         arch,
267         "val_loss",
268         metric_data,
269         scales,
270         cmap,
271         by,
272         max_increment=max_increment,
273     )
274     add_metric_graph(
275         fig,
276         axs[0, 1],
277         arch,
278         "val_accuracy",
279         metric_data,
280         scales,
281         cmap,
282         by,
283         max_increment=max_increment,
284     )
285     add_metric_graph(
286         fig,
287         axs[1, 0],
288         arch,
289         "train_loss",
290         metric_data,
291         scales,
292         cmap,
293         by,
294         max_increment=max_increment,
295     )
296     add_metric_graph(
297         fig,
298         axs[1, 1],
299         arch,
300         "train_accuracy",
301         metric_data,
```

```

302     scales,
303     cmap,
304     by,
305     max_increment=max_increment,
306 )
307 add_metric_graph(
308     fig,
309     axs[2, 0],
310     arch,
311     "learning_rate",
312     metric_data,
313     scales,
314     cmap,
315     by,
316     max_increment=max_increment,
317 )
318 fig.suptitle(f"{operation}_{arch}_{max_increment:06d}_{by}s")
319 fig.tight_layout()
320
321 img_file = f"{image_dir}/loss_curves/{operation}_loss_curves_{arch}__upto_{
322     max_increment:010d}_{by}"
323 if most_interesting_only:
324     img_file += "_most_interesting"
325 img_file += ".png"
326 d = os.path.split(img_file)[0]
327 os.makedirs(d, exist_ok=True)
328 print(f"Writing_{img_file}")
329 fig.savefig(img_file)
330 plt.close(fig)
331
332 def create_max_accuracy_curves(
333     metric_data,
334     arch,
335     operation,
336     by="step",
337     max_increment=0,
338     image_dir=args.output_dir,
339 ):
340     scales = {
341         "x": "linear",
342         "y": "linear",
343     }
344
345     ncols = 1
346     nrows = 2
347     fig_width = ncols * 8
348     fig_height = nrows * 5
349     fig, axs = plt.subplots(nrows=nrows, ncols=ncols, figsize=(fig_width, fig_height))
350
351     add_max_accuracy_graph(
352         axs[0],
353         arch,
354         "val_accuracy",
355         metric_data,
356         scales,
357         by=by,
358         max_increment=max_increment,

```

```

359 )
360 axs[0].legend()
361 add_max_accuracy_graph(
362     axs[1],
363     arch,
364     "train_accuracy",
365     metric_data,
366     scales,
367     by=by,
368     max_increment=max_increment,
369 )
370 axs[1].legend()
371 fig.suptitle(f"{operation}_{arch}_{max_increment:06d}_{by}s")
372 fig.tight_layout()
373
374 img_file = f"{image_dir}/max_accuracy/{operation}_max_accuracy_{arch}_upto_{
375     max_increment:010d}_{by}.png"
376 d = os.path.split(img_file)[0]
377 os.makedirs(d, exist_ok=True)
378 print(f"Writing_{img_file}")
379 fig.savefig(img_file)
380 plt.close(fig)
381
382 def create_tsne_graphs(
383     operation,
384     expt,
385     run_dir,
386     image_dir=args.output_dir,
387 ):
388
389     saved_pt_dir = f"{run_dir}/activations"
390     saved_pts = []
391
392     loss_ts = []
393     accuracy_ts = []
394     epochs_ts = []
395     print(f'glob_{saved_pt_dir}_{"/activations_*.pt"}')
396     files = sorted(glob.glob(saved_pt_dir + "/activations_*.pt"))
397     print(f"files_{files}")
398
399     for file in files:
400         print(f"Loading_{file}")
401         saved_pt = torch.load(file)
402         saved_pts.append(saved_pt)
403         loss_ts.append(saved_pt["val_loss"].mean(dim=-1))
404         accuracy_ts.append(saved_pt["val_accuracy"])
405         epochs_ts.append(saved_pt["epochs"].squeeze())
406
407     loss_t = torch.cat(loss_ts, dim=0).T.detach()
408     accuracy_t = torch.cat(accuracy_ts, dim=0).T.detach()
409     epochs_t = torch.cat(epochs_ts, dim=0).detach()
410     print(loss_t.shape)
411     print(accuracy_t.shape)
412     print(epochs_t.shape)
413     #####
414     a = 0
415     num_eqs = len(loss_t)

```

```

416     b = a + num_eqs
417
418     print("Doing T-SNE..")
419     loss_tsne = TSNE(n_components=2, init="pca").fit_transform(loss_t)
420     print("...done T-SNE.")
421
422     ncols = 1
423     nrows = 1
424     fig_width = ncols * 8
425     fig_height = nrows * 5
426     fig, axs = plt.subplots(nrows=nrows, ncols=ncols, figsize=(fig_width, fig_height))
427
428     axs.scatter(loss_tsne[:, 0], loss_tsne[:, 1])
429
430     img_file = f"{image_dir}/tsne/{operation}_{expt}.png"
431     d = os.path.split(img_file)[0]
432     os.makedirs(d, exist_ok=True)
433     print(f"Writing {img_file}")
434     fig.savefig(img_file)
435     plt.close(fig)
436
437
438 def get_arch(metric_data):
439     k = list(metric_data.keys())[0]
440     hparams = metric_data[k]["hparams"]
441     arch = f'L-{hparams["n_layers"]}_H-{hparams["n_heads"]}_D-{hparams["d_model"]}_B-{
442         hparams["batchsize"]}_S-{hparams["random_seed"]}_DR-{hparams["dropout"]}'
443     return arch
444
445 def get_operation(metric_data):
446     k = list(metric_data.keys())[0]
447     hparams = metric_data[k]["hparams"]
448     operator = hparams["math_operator"]
449     operand_length = hparams["operand_length"]
450     _, operation = grok.data.ArithmeticDataset.get_file_path(operator, operand_length)
451     return operation
452
453
454 def get_max_epochs(metric_data):
455     k = list(metric_data.keys())[0]
456     hparams = metric_data[k]["hparams"]
457     return hparams["max_epochs"]
458
459
460 rundir = args.input_dir
461
462 try:
463     metric_data = load_run_metrics(rundir, args)
464     arch = get_arch(metric_data)
465     operation = get_operation(metric_data)
466     max_epochs = get_max_epochs(metric_data)
467
468     for by in ["step", "epoch"]:
469         create_loss_curves(metric_data, arch, operation, by=by)
470
471     by = "epoch"
472     last_i = -1

```

```
473 for i in sorted(list(set(2 ** (np.arange(167) / 10)))):
474     if i > max_epochs:
475         break
476     i = int(round(i))
477     create_max_accuracy_curves(
478         metric_data,
479         arch,
480         operation,
481         by=by,
482         max_increment=i,
483     )
484
485     # make a video
486     in_files = os.path.join(
487         args.output_dir,
488         "max_accuracy",
489         f"{operation}_max_accuracy_{arch}_upto_%.png",
490     )
491     out_file = os.path.join(args.output_dir, f"{operation}_{arch}_max_accuracy.mp4")
492     cmd = [
493         "ffmpeg",
494         "-y",
495         "-r",
496         "16",
497         "-i",
498         in_files,
499         "-vcodec",
500         "libx264",
501         "-crf",
502         "25",
503         "-pix_fmt",
504         "yuv420p",
505         out_file,
506     ]
507     subprocess.check_call(cmd)
508
509 except BaseException as e:
510     print(f"{rundir}_failed: {e}")
```

4.3 NC Experiments: nc_experiments/

4.3.1 resnet18.py — ResNet-18 from Scratch

```

1  """
2  Builds ResNet18 from scratch using PyTorch.
3  This does not build generalized blocks for all ResNets, just for ResNet18.
4  Paper => Deep Residual Learning for Image Recognition.
5  Link => https://arxiv.org/pdf/1512.03385v1.pdf
6  """
7
8  import torch.nn as nn
9  import torch
10
11  from torch import Tensor
12  from typing import Type
13
14
15  class BasicBlock(nn.Module):
16      def __init__(
17          self,
18          in_channels: int,
19          out_channels: int,
20          stride: int = 1,
21          expansion: int = 1,
22          downsample: nn.Module = None,
23      ) -> None:
24          super(BasicBlock, self).__init__()
25          # Multiplicative factor for the subsequent conv2d layer's output channels.
26          # It is 1 for ResNet18 and ResNet34.
27          self.expansion = expansion
28          self.downsample = downsample
29          self.conv1 = nn.Conv2d(
30              in_channels,
31              out_channels,
32              kernel_size=3,
33              stride=stride,
34              padding=1,
35              bias=False,
36          )
37          self.bn1 = nn.BatchNorm2d(out_channels)
38          self.relu = nn.ReLU(inplace=True)
39          self.conv2 = nn.Conv2d(
40              out_channels,
41              out_channels * self.expansion,
42              kernel_size=3,
43              padding=1,
44              bias=False,
45          )
46          self.bn2 = nn.BatchNorm2d(out_channels * self.expansion)
47
48      def forward(self, x: Tensor) -> Tensor:
49          identity = x
50
51          out = self.conv1(x)
52          out = self.bn1(out)
53          out = self.relu(out)
54

```

```

55     out = self.conv2(out)
56     out = self.bn2(out)
57
58     if self.downsample is not None:
59         identity = self.downsample(x)
60
61     out += identity
62     out = self.relu(out)
63     return out
64
65
66 class ResNet(nn.Module):
67     def __init__(
68         self,
69         img_channels: int,
70         num_layers: int,
71         block: Type[BasicBlock],
72         num_classes: int = 1000,
73     ) -> None:
74         super(ResNet, self).__init__()
75         if num_layers == 18:
76             # The following `layers` list defines the number of `BasicBlock`
77             # to use to build the network and how many basic blocks to stack
78             # together.
79             layers = [2, 2, 2, 2]
80             self.expansion = 1
81
82         self.in_channels = 64
83         # All ResNets (18 to 152) contain a Conv2d => BN => ReLU for the first
84         # three layers. Here, kernel size is 7.
85         self.conv1 = nn.Conv2d(
86             in_channels=img_channels,
87             out_channels=self.in_channels,
88             kernel_size=7,
89             stride=2,
90             padding=3,
91             bias=False,
92         )
93         self.bn1 = nn.BatchNorm2d(self.in_channels)
94         self.relu = nn.ReLU(inplace=True)
95         self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
96
97         self.layer1 = self._make_layer(block, 64, layers[0])
98         self.layer2 = self._make_layer(block, 128, layers[1], stride=2)
99         self.layer3 = self._make_layer(block, 256, layers[2], stride=2)
100        self.layer4 = self._make_layer(block, 512, layers[3], stride=2)
101
102        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
103        self.fc = nn.Linear(512 * self.expansion, num_classes)
104
105    def _make_layer(
106        self, block: Type[BasicBlock], out_channels: int, blocks: int, stride: int = 1
107    ) -> nn.Sequential:
108        downsample = None
109        if stride != 1:
110            """
111            This should pass from `layer2` to `layer4` or
112            when building ResNets50 and above. Section 3.3 of the paper

```

```

113     Deep Residual Learning for Image Recognition
114     (https://arxiv.org/pdf/1512.03385v1.pdf).
115     """
116     downsample = nn.Sequential(
117         nn.Conv2d(
118             self.in_channels,
119             out_channels * self.expansion,
120             kernel_size=1,
121             stride=stride,
122             bias=False,
123         ),
124         nn.BatchNorm2d(out_channels * self.expansion),
125     )
126     layers = []
127     layers.append(
128         block(self.in_channels, out_channels, stride, self.expansion, downsample)
129     )
130     self.in_channels = out_channels * self.expansion
131
132     for i in range(1, blocks):
133         layers.append(
134             block(self.in_channels, out_channels, expansion=self.expansion)
135         )
136     return nn.Sequential(*layers)
137
138 def forward(self, x: Tensor) -> Tensor:
139     x = self.conv1(x)
140     x = self.bn1(x)
141     x = self.relu(x)
142     x = self.maxpool(x)
143
144     x = self.layer1(x)
145     x = self.layer2(x)
146     x = self.layer3(x)
147     x = self.layer4(x)
148     # The spatial dimension of the final layer's feature
149     # map should be (7, 7) for all ResNets.
150     x = self.avgpool(x)
151     second_last_outputs = torch.flatten(x, 1)
152     x = self.fc(second_last_outputs)
153
154     return x, second_last_outputs
155
156
157 if __name__ == "__main__":
158     tensor = torch.rand([1, 3, 224, 224])
159     model = ResNet(img_channels=3, num_layers=18, block=BasicBlock, num_classes=1000)
160     print(model)
161
162     # Total parameters and trainable parameters.
163     total_params = sum(p.numel() for p in model.parameters())
164     print(f"{total_params:,}_total_parameters.")
165     total_trainable_params = sum(
166         p.numel() for p in model.parameters() if p.requires_grad
167     )
168     print(f"{total_trainable_params:,}_training_parameters.")
169
170     output = model(tensor)

```


4.3.2 resnet18_torchvision.py — TorchVision Wrapper

```

1 import torchvision.models as models
2 import torch.nn as nn
3
4
5 def build_model(pretrained=True, fine_tune=True, num_classes=1):
6     if pretrained:
7         print("[INFO]: Loading pre-trained weights")
8         model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
9     elif not pretrained:
10        print("[INFO]: Not loading pre-trained weights")
11        model = models.resnet18(weights=None)
12    if fine_tune:
13        print("[INFO]: Fine-tuning all layers...")
14        for params in model.parameters():
15            params.requires_grad = True
16    elif not fine_tune:
17        print("[INFO]: Freezing hidden layers...")
18        for params in model.parameters():
19            params.requires_grad = False
20
21    # Change the final classification head, it is trainable.
22    model.fc = nn.Linear(512, num_classes)
23    return model
24
25
26 if __name__ == "__main__":
27     model = build_model(num_classes=1000)
28     # Total parameters and trainable parameters.
29     total_params = sum(p.numel() for p in model.parameters())
30     print(f"{total_params:,} total parameters.")
31     total_trainable_params = sum(
32         p.numel() for p in model.parameters() if p.requires_grad
33     )
34     print(f"{total_trainable_params:,} training parameters.")

```

4.3.3 training_utils.py — Training Loop & NC Metric

```

1 import torch
2
3 from tqdm import tqdm
4
5
6 def compute_cluster(train_repr, y_predict_train):
7
8     class_dict = {key: [] for key in torch.unique(y_predict_train).tolist()}
9     for sub_train_repr, sub_y_predict_train in zip(train_repr, y_predict_train):
10        class_dict[sub_y_predict_train.tolist()].append(sub_train_repr)
11
12    mean_var_batch_dict = {}
13    for key, value in class_dict.items():
14        temp_mean = torch.mean(torch.stack(value), dim=0)
15        temp_var = torch.mean(torch.stack([torch.linalg.norm(f - temp_mean) ** 2 for f in
16            value]))
17        mean_var_batch_dict[key] = [temp_mean, temp_var]
18
19    # calculate the clustering values

```

```

19 all_train_cdnvs = []
20 mean_list = []
21 var_class_cluster_list = []
22 for c1 in class_dict.keys():
23     for c2 in class_dict.keys():
24         if c2 == c1:
25             continue
26         mu1, var1 = mean_var_batch_dict[c1]
27         mu2, var2 = mean_var_batch_dict[c2]
28         temp_result = (var1 + var2) / (2 * torch.linalg.norm(mu1 - mu2) ** 2)
29         var_class_cluster_list.append(torch.tensor([c1, var1, c2, var2, temp_result])
30                                     )
31         mean_list.append(torch.stack((mu1, mu2), dim=0))
32         all_train_cdnvs.append(temp_result)
33 all_train_cdnvs = torch.mean(torch.stack(all_train_cdnvs))
34 return all_train_cdnvs, mean_list, var_class_cluster_list
35
36 # Training function.
37 def train(model, trainloader, optimizer, criterion, device, args, current_epoch):
38     model.train()
39     print("Training")
40     train_running_loss = 0.0
41     train_running_correct = 0
42     counter = 0
43     weights_norm_list, sharpness_list, sharpness2_list = [], [], []
44     train_repr_list, y_predict_train_list = [], []
45     for i, data in tqdm(enumerate(trainloader), total=len(trainloader), disable=True):
46         counter += 1
47         image, labels = data
48         image = image.to(device)
49         labels = labels.to(device)
50         optimizer.zero_grad()
51         # Forward pass.
52         outputs, second_last_outputs = model(image)
53         # Calculate the loss.
54         loss = criterion(outputs, labels)
55         # train_running_loss += loss.item()
56         # Calculate the accuracy.
57         _, preds = torch.max(outputs.data, 1)
58         train_running_correct += (preds == labels).sum().item()
59
60     if args["use_regulation"]:
61         if args["use_pow"]:
62             metric_loss, _, _ = compute_cluster(second_last_outputs, labels)
63             loss = loss - args["lambda"] * torch.mean(metric_loss)
64             with torch.no_grad():
65                 probs = torch.softmax(outputs, dim=1)
66                 hessian_first = torch.sum(torch.mul(probs, 1 - probs), dim=1)
67                 hessian_second = torch.sum(torch.mul(second_last_outputs,
68                                                         second_last_outputs), dim=1)
69                 hessian = torch.mul(hessian_first, hessian_second)
70                 weights_norm = torch.linalg.norm(model.fc.weight)
71                 sharpness = weights_norm * hessian
72                 sharpness2 = weights_norm * weights_norm * hessian
73         else:
74             probs = torch.softmax(outputs, dim=1)
75             hessian_first = torch.sum(torch.mul(probs, 1 - probs), dim=1)

```

```

74         hessian_second = torch.sum(torch.mul(second_last_outputs,
75             second_last_outputs), dim=1)
76         hessian = torch.mul(hessian_first, hessian_second)
77         weights_norm = torch.linalg.norm(model.fc.weight)
78         sharpness = weights_norm * hessian
79         sharpness2 = weights_norm * weights_norm * hessian
80
81         loss = loss - args["lambda"] * torch.mean(sharpness2)
82     else:
83         # calculate hessian and sharpness here
84         with torch.no_grad():
85             probs = torch.softmax(outputs, dim=1)
86             hessian_first = torch.sum(torch.mul(probs, 1 - probs), dim=1)
87             hessian_second = torch.sum(torch.mul(second_last_outputs,
88                 second_last_outputs), dim=1)
89             hessian = torch.mul(hessian_first, hessian_second)
90             weights_norm = torch.linalg.norm(model.fc.weight)
91             sharpness = weights_norm * hessian
92             sharpness2 = weights_norm * weights_norm * hessian
93
94         # collect the results
95         weights_norm_list.append(weights_norm)
96         sharpness_list.append(sharpness)
97         sharpness2_list.append(sharpness2)
98         train_repr_list.append(second_last_outputs)
99         y_predict_train_list.append(labels)
100         train_running_loss += loss.item()
101
102     # Backpropagation
103     loss.backward()
104
105     optimizer.step()
106
107     # Loss and accuracy for the complete epoch.
108     epoch_loss = train_running_loss / counter
109     epoch_acc = 100.0 * (train_running_correct / len(trainloader.dataset))
110
111     # sort the computation results here
112     with torch.no_grad():
113         sharpness_list = torch.mean(torch.cat(sharpness_list))
114         sharpness2_list = torch.mean(torch.cat(sharpness2_list))
115         weights_norm_list = torch.mean(torch.tensor(weights_norm_list))
116         train_cdnvs_list, mean_list, var_class_cluster_list = compute_cluster(torch.cat(
117             train_repr_list), torch.cat(y_predict_train_list))
118
119     return epoch_loss, epoch_acc, sharpness_list, sharpness2_list, weights_norm_list,
120         train_cdnvs_list, mean_list, var_class_cluster_list
121
122
123
124 # Validation function.
125 def validate(model, testloader, criterion, device):
126     model.eval()
127     print("Validation")

```

```

128     valid_running_loss = 0.0
129     valid_running_correct = 0
130     counter = 0
131
132     with torch.no_grad():
133         for i, data in tqdm(enumerate(testloader), total=len(testloader), disable=True):
134             counter += 1
135
136             image, labels = data
137             image = image.to(device)
138             labels = labels.to(device)
139             # Forward pass.
140             outputs, _ = model(image)
141             # Calculate the loss.
142             loss = criterion(outputs, labels)
143             valid_running_loss += loss.item()
144             # Calculate the accuracy.
145             _, preds = torch.max(outputs.data, 1)
146             valid_running_correct += (preds == labels).sum().item()
147
148     # Loss and accuracy for the complete epoch.
149     epoch_loss = valid_running_loss / counter
150     epoch_acc = 100.0 * (valid_running_correct / len(testloader.dataset))
151     return epoch_loss, epoch_acc

```

4.3.4 train.py — Entry Point

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import argparse
5 import numpy as np
6 import random
7
8 from resnet18 import ResNet, BasicBlock
9 from resnet18_torchvision import build_model
10 from training_utils import train, validate
11 from utils import save_plots, get_data
12 import os
13 from datetime import datetime
14 import torch.nn.functional as F
15
16 parser = argparse.ArgumentParser()
17 parser.add_argument(
18     "-m",
19     "--model",
20     default="scratch",
21     help="choose model built from scratch or the Torchvision model",
22     choices=["scratch", "torchvision"],
23 )
24
25 parser.add_argument("--learning_rate", type=float, default=0.01)
26 parser.add_argument("--training_date", type=str, default="0211")
27 parser.add_argument("--random_seed", type=int, default=42)
28 parser.add_argument("--epochs", type=int, default=1)
29 parser.add_argument("--cancel_epoch", type=int, default=60, help="the epoch to cancel the
    regularization")

```

```

30 parser.add_argument("--cancel_dur_epoch", type=int, default=10, help="the_epoch_to_cancel
    _the_regularization")
31 parser.add_argument("--batch_size", type=int, default=64)
32 parser.add_argument("--weight_decay", type=float, default=0.2)
33 parser.add_argument("--use_regulation", type=int, default=1)
34 parser.add_argument("--lambda", type=float, default=1e-4)
35 parser.add_argument("--clip_value", type=float, default=1.0)
36 parser.add_argument("--optim_type", type=str, default="sgd")
37 parser.add_argument("--training_size", type=float, default=0.0)
38 parser.add_argument("--use_pow", type=int, default=1, help="using_ncc")
39
40
41
42 args = vars(parser.parse_args())
43
44 # Set seed.
45 seed = args["random_seed"]
46 torch.manual_seed(seed)
47 torch.cuda.manual_seed(seed)
48 torch.backends.cudnn.deterministic = True
49 torch.backends.cudnn.benchmark = False
50 np.random.seed(seed)
51 random.seed(seed)
52
53 # Learning and training parameters.
54 epochs = args["epochs"]
55 batch_size = args["batch_size"]
56 learning_rate = args["learning_rate"]
57 device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
58
59 train_loader, valid_loader = get_data(training_ratio=args["training_size"], batch_size=
    batch_size)
60
61 # Make dir to store data
62 output_dir = "Seed{}_Date{}_TSIZE{}_BSZ{}_OPTM{}_LR{}_EPOCH{}_CANECH{}_CANECHDUR{}_WD{}_
    _REGR{}_LIPZ{}_LAMBDA{}_GCP{}".format(
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81 # Define model based on the argument parser string.
82 if args["model"] == "scratch":
83     print("[INFO]:_Training_ResNet18_built_from_scratch...")
84     model = ResNet(img_channels=3, num_layers=18, block=BasicBlock, num_classes=10).to(

```

```

85     device
86 )
87 # model = ResNet(BasicBlock, [2,2,2,2]).to(device)
88 plot_name = "resnet_scratch" + "_" + output_dir
89 if args["model"] == "torchvision":
90     print(f"[INFO]:_Training_the_Torchvision_ResNet18_model...")
91     model = build_model(pretrained=False, fine_tune=True, num_classes=10).to(device)
92     plot_name = "resnet_torchvision" + "_" + output_dir
93 # print(model)
94
95 # Total parameters and trainable parameters.
96 total_params = sum(p.numel() for p in model.parameters())
97 print(f"{total_params:}_total_parameters.")
98 total_trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
99 print(f"{total_trainable_params:}_training_parameters.")
100
101 # Optimizer.
102 if args["optim_type"] == "sgd":
103     optimizer = optim.SGD(model.parameters(), lr=learning_rate, momentum = 0.9,
104         weight_decay = args["weight_decay"])
105 else:
106     optimizer = optim.AdamW(model.parameters(), lr=learning_rate, betas=(0.9, 0.999),
107         weight_decay=args["weight_decay"], eps=1e-8)
108
109 # Loss function.
110 criterion = nn.CrossEntropyLoss()
111
112 for key, value in args.items():
113     print(f"{key}:_{value}")
114 start_time = datetime.now()
115
116 if __name__ == "__main__":
117     # Lists to keep track of losses and accuracies.
118     train_loss, valid_loss = [], []
119     train_acc, valid_acc = [], []
120     # Start the training.
121     for epoch in range(epochs):
122         print(f"[INFO]:_Epoch_{epoch+1}_of_{epochs}")
123         if epoch >= args["cancel_epoch"] and args["use_regulation"]:
124             print("Stop_using_the_regularization_from_Epoch_{epoch}").format(epoch)
125             args["use_regulation"] = 0
126             # args["clip_value"] = 0
127             train_epoch_loss, train_epoch_acc, sharpness, sharpness2, weights_norm,
128             cluster_value, mean_list, var_class_cluster_list = train(
129                 # train_epoch_loss, train_epoch_acc, sharpness, sharpness2, weights_norm = train
130                 (
131                     model, train_loader, optimizer, criterion, device, args, epoch
132                 )
133             )
134             valid_epoch_loss, valid_epoch_acc = validate(
135                 model, valid_loader, criterion, device
136             )
137             train_loss.append(train_epoch_loss)
138             valid_loss.append(valid_epoch_loss)
139             train_acc.append(train_epoch_acc)
140             valid_acc.append(valid_epoch_acc)
141
142         # save values here:
143         with torch.no_grad():

```

```

139     np.save(output_dir + "/sharpness_" + str(epoch), sharpness.detach().cpu().
140           numpy())
141     np.save(output_dir + "/sharpness2_" + str(epoch), sharpness2.detach().cpu().
142           numpy())
143     np.save(output_dir + "/weights_norm_" + str(epoch), weights_norm.numpy())
144     np.save(output_dir + "/cluster_value_" + str(epoch), cluster_value.detach().
145           cpu().numpy())
146     np.save(output_dir + "/mean_value_" + str(epoch), torch.stack(mean_list).
147           detach().cpu().numpy())
148     np.save(output_dir + "/var_classcluster_" + str(epoch), torch.stack(
149           var_class_cluster_list).detach().cpu().numpy())
150     np.save(output_dir + "/train_acc_" + str(epoch), train_epoch_acc)
151     np.save(output_dir + "/train_loss_" + str(epoch), train_epoch_loss)
152     np.save(output_dir + "/val_loss_" + str(epoch), valid_epoch_loss)
153     np.save(output_dir + "/val_acc_" + str(epoch), valid_epoch_acc)
154
155     print(
156         f"Training_loss: {train_epoch_loss:.3f}, training_acc: {train_epoch_acc:.3f}"
157     )
158     print(
159         f"Validation_loss: {valid_epoch_loss:.3f}, validation_acc: {valid_epoch_acc
160           :.3f}"
161     )
162     print("-" * 50)
163     print(datetime.now())
164
165     # Save the loss and accuracy plots.
166     save_plots(train_acc, valid_acc, train_loss, valid_loss, name=plot_name)
167     end_time = datetime.now()
168     print('Duration: {}'.format(end_time - start_time))
169     print("TRAINING COMPLETE")

```

4.3.5 utils.py — Data Loading & Plotting

```

1 import matplotlib.pyplot as plt
2 import os
3
4 from torch.utils.data import DataLoader, Subset
5 from torchvision import datasets, transforms
6 from torchvision.transforms import ToTensor
7
8 plt.style.use("ggplot")
9
10
11 def get_data(training_ratio, batch_size):
12
13     # Define the transformation to apply to the images (e.g., converting to tensor,
14     # normalization)
15     transform = transforms.Compose([transforms.ToTensor(),
16                                     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])
17
18     # CIFAR10 training dataset.
19     dataset_train = datasets.CIFAR10(
20         root="data",
21         train=True,

```

```

20         download=True,
21         transform=transform,
22     )
23
24     # CIFAR10 validation dataset.
25     dataset_valid = datasets.CIFAR10(
26         root="data",
27         train=False,
28         download=True,
29         transform=transform,
30     )
31
32     if training_ratio > 0.0:
33         # Get 10% of the training data
34         total_size = len(trainset)
35         subset_size = int(training_ratio * total_size) # 10% of the total training data
36         indices = np.random.choice(total_size, subset_size, replace=False) # Randomly
37             select indices
38
39         # Create a subset dataset
40         subset_trainset = Subset(trainset, indices)
41
42
43     # Create data loaders.
44     if training_ratio > 0.0:
45         # Create a DataLoader for the subset dataset
46         trainloader = DataLoader(subset_trainset, batch_size=batch_size, shuffle=True)
47     else:
48         train_loader = DataLoader(dataset_train, batch_size=batch_size, shuffle=True)
49
50     valid_loader = DataLoader(dataset_valid, batch_size=batch_size, shuffle=False)
51     return train_loader, valid_loader
52
53
54 def save_plots(train_acc, valid_acc, train_loss, valid_loss, name=None):
55     """
56     Function to save the loss and accuracy plots to disk.
57     """
58     # Accuracy plots.
59     plt.figure(figsize=(10, 7))
60     plt.plot(train_acc, color="tab:blue", linestyle="-", label="train accuracy")
61     plt.plot(valid_acc, color="tab:red", linestyle="-", label="validation accuracy")
62     plt.xlabel("Epochs")
63     plt.ylabel("Accuracy")
64     plt.legend()
65     plt.savefig(os.path.join("outputs", name + "_accuracy.png"))
66
67     # Loss plots.
68     # plt.figure(figsize=(10, 7))
69     # plt.plot(train_loss, color="tab:blue", linestyle="-", label="train loss")
70     # plt.plot(valid_loss, color="tab:red", linestyle="-", label="validation loss")
71     # plt.xlabel("Epochs")
72     # plt.ylabel("Loss")
73     # plt.legend()
74     # plt.savefig(os.path.join("outputs", name + "_loss.png"))

```


4.3.6 run_script.sh

```

1 #!/bin/bash
2
3 readonly LEARNING_RATE=1e-02
4 readonly DATE=$(date +%m%d)
5 readonly SEED=15213
6 readonly TRAINING_EPOCH=250
7 readonly CANCEL_EPOCH=10000
8 readonly BSZ=64
9 readonly WEIGHTDECAY=0
10 readonly USE_REGR=1 # using regularizer
11 readonly USE_POW=1 # using NCC regularizer
12 readonly HECOEFF=1e-3 # coefficient: lambda
13 readonly CLIPPING_VALUE=0
14 readonly OPTM="sgd" # sgd or adamw
15 readonly TRAINING_SIZE=0.0 # if 0 = use all training samples.
16
17
18
19
20 nohup python -u train.py \
21     --learning_rate=${LEARNING_RATE} \
22     --training_date=${DATE} \
23     --random_seed=${SEED} \
24     --epochs=${TRAINING_EPOCH} \
25     --cancel_epoch=${CANCEL_EPOCH} \
26     --batch_size=${BSZ} \
27     --weight_decay=${WEIGHTDECAY} \
28     --use_regulation=${USE_REGR} \
29     --use_pow=${USE_POW} \
30     --lambda=${HECOEFF} \
31     --clip_value=${CLIPPING_VALUE} \
32     --optim_type=${OPTM} \
33     --training_size=${TRAINING_SIZE} \
34     > Seed${SEED}_Date${DATE}_TSZ${TRAINING_SIZE}_BSZ${BSZ}_OPTM${OPTM}_LR${LEARNING_RATE}
    _EPOCH${TRAINING_EPOCH}_CANECH${CANCEL_EPOCH}_WD${WEIGHTDECAY}_REGR${USE_REGR}
    _LIPZ${USE_POW}_LAMBDA${HECOEFF}_GCP${CLIPPING_VALUE}_trainlog.txt 2>&1 &

```

4.4 Relative Flatness: rf_resnet18/

4.4.1 resnet18.py

```

1  """
2  Builds ResNet18 from scratch using PyTorch.
3  This does not build generalized blocks for all ResNets, just for ResNet18.
4  Paper => Deep Residual Learning for Image Recognition.
5  Link => https://arxiv.org/pdf/1512.03385v1.pdf
6  """
7
8  import torch.nn as nn
9  import torch
10
11  from torch import Tensor
12  from typing import Type
13  import torch.nn.functional as F
14
15
16  class BasicBlock(nn.Module):
17      def __init__(
18          self,
19          in_channels: int,
20          out_channels: int,
21          stride: int = 1,
22          expansion: int = 1,
23          downsample: nn.Module = None,
24      ) -> None:
25          super(BasicBlock, self).__init__()
26          # Multiplicative factor for the subsequent conv2d layer's output channels.
27          # It is 1 for ResNet18 and ResNet34.
28          self.expansion = expansion
29          self.downsample = downsample
30          self.conv1 = nn.Conv2d(
31              in_channels,
32              out_channels,
33              kernel_size=3,
34              stride=stride,
35              padding=1,
36              bias=False,
37          )
38          self.bn1 = nn.BatchNorm2d(out_channels)
39          self.relu = nn.ReLU(inplace=True)
40          self.conv2 = nn.Conv2d(
41              out_channels,
42              out_channels * self.expansion,
43              kernel_size=3,
44              padding=1,
45              bias=False,
46          )
47          self.bn2 = nn.BatchNorm2d(out_channels * self.expansion)
48
49      def forward(self, x: Tensor) -> Tensor:
50          identity = x
51
52          out = self.conv1(x)
53          out = self.bn1(out)
54          out = self.relu(out)

```

```

55
56     out = self.conv2(out)
57     out = self.bn2(out)
58
59     if self.downsample is not None:
60         identity = self.downsample(x)
61
62     out += identity
63     out = self.relu(out)
64     return out
65
66
67 class ResNet(nn.Module):
68     def __init__(
69         self,
70         img_channels: int,
71         num_layers: int,
72         block: Type[BasicBlock],
73         num_classes: int = 1000,
74     ) -> None:
75         super(ResNet, self).__init__()
76         if num_layers == 18:
77             # The following `layers` list defines the number of `BasicBlock`
78             # to use to build the network and how many basic blocks to stack
79             # together.
80             layers = [2, 2, 2, 2]
81             self.expansion = 1
82
83         self.in_channels = 64
84         # All ResNets (18 to 152) contain a Conv2d => BN => ReLU for the first
85         # three layers. Here, kernel size is 7.
86         self.conv1 = nn.Conv2d(
87             in_channels=img_channels,
88             out_channels=self.in_channels,
89             kernel_size=7,
90             stride=2,
91             padding=3,
92             bias=False,
93         )
94         self.bn1 = nn.BatchNorm2d(self.in_channels)
95         self.relu = nn.ReLU(inplace=True)
96         self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
97
98         self.layer1 = self._make_layer(block, 64, layers[0])
99         self.layer2 = self._make_layer(block, 128, layers[1], stride=2)
100        self.layer3 = self._make_layer(block, 256, layers[2], stride=2)
101        self.layer4 = self._make_layer(block, 512, layers[3], stride=2)
102
103        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
104        self.fc = nn.Linear(512 * self.expansion, num_classes)
105
106        def _make_layer(
107            self, block: Type[BasicBlock], out_channels: int, blocks: int, stride: int = 1
108        ) -> nn.Sequential:
109            downsample = None
110            if stride != 1:
111                """
112                This should pass from `layer2` to `layer4` or

```

```

113         when building ResNets50 and above. Section 3.3 of the paper
114         Deep Residual Learning for Image Recognition
115         (https://arxiv.org/pdf/1512.03385v1.pdf).
116         """
117         downsample = nn.Sequential(
118             nn.Conv2d(
119                 self.in_channels,
120                 out_channels * self.expansion,
121                 kernel_size=1,
122                 stride=stride,
123                 bias=False,
124             ),
125             nn.BatchNorm2d(out_channels * self.expansion),
126         )
127         layers = []
128         layers.append(
129             block(self.in_channels, out_channels, stride, self.expansion, downsample)
130         )
131         self.in_channels = out_channels * self.expansion
132
133         for i in range(1, blocks):
134             layers.append(
135                 block(self.in_channels, out_channels, expansion=self.expansion)
136             )
137         return nn.Sequential(*layers)
138
139     def forward(self, x: Tensor) -> Tensor:
140         x = self.conv1(x)
141         x = self.bn1(x)
142         x = self.relu(x)
143         x = self.maxpool(x)
144
145         x = self.layer1(x)
146         x = self.layer2(x)
147         x = self.layer3(x)
148         x = self.layer4(x)
149         # The spatial dimension of the final layer's feature
150         # map should be (7, 7) for all ResNets.
151         x = self.avgpool(x)
152         second_last_outputs = torch.flatten(x, 1)
153         x = self.fc(second_last_outputs)
154
155         return x, second_last_outputs
156
157
158 ##### modified network structure
159 #####
160 class MBasicBlock(nn.Module):
161     def __init__(self, in_planes, planes, stride = 1):
162         super(MBasicBlock, self).__init__()
163
164         self.conv1 = nn.Conv2d(in_planes, planes, kernel_size = 3, stride = stride,
165                                 padding = 1, bias = False)
166         self.bn1 = nn.BatchNorm2d(planes)
167
168         self.conv2 = nn.Conv2d(planes, planes, kernel_size = 3, stride = 1, padding = 1,
169                                 bias = False)
170         self.bn2 = nn.BatchNorm2d(planes)

```

```

168
169     self.shortcut = nn.Identity()
170
171     if stride != 1:
172         self.shortcut = nn.Sequential(
173             nn.Conv2d(in_planes, planes, kernel_size = 1, stride = stride
174                       , bias = False),
175             nn.BatchNorm2d(planes)
176         )
177
178     def forward(self, x):
179         out = F.relu(self.bn1(self.conv1(x)))
180         out = self.bn2(self.conv2(out))
181         out += self.shortcut(x)
182         out = F.relu(out)
183
184     return out
185
186 class MResNet(nn.Module):
187     def __init__(self, block, num_blocks, num_classes = 10):
188         super(MResNet, self).__init__()
189         self.in_planes = 64
190
191         self.conv1 = nn.Conv2d(3,64,kernel_size = 3, stride = 1, padding = 1, bias =
192                               False)
193         self.bn1 = nn.BatchNorm2d(64)
194         self.layer1 = self._make_layer(block, 64, num_blocks[0], stride = 1)
195         self.layer2 = self._make_layer(block, 128, num_blocks[1], stride = 2)
196         self.layer3 = self._make_layer(block, 256, num_blocks[2], stride = 2)
197         self.layer4 = self._make_layer(block, 512, num_blocks[3], stride = 2)
198         self.fc = nn.Linear(512, num_classes)
199
200     def _make_layer(self, block, planes, num_blocks, stride):
201         strides = [stride] + [1] * (num_blocks - 1)
202         layers = []
203
204         for stride in strides:
205             layers.append(block(self.in_planes, planes, stride))
206             self.in_planes = planes
207
208         return nn.Sequential(*layers)
209
210     def forward(self, x):
211         out = F.relu(self.bn1(self.conv1(x)))
212         out = self.layer1(out)
213         out = self.layer2(out)
214         out = self.layer3(out)
215         out = self.layer4(out)
216         out = F.avg_pool2d(out, 4)
217         second_last_outputs = out.view(out.size(0), -1) # out.size(0) = out.reshape(0)
218         out = self.fc(second_last_outputs)
219
220     return out, second_last_outputs
221
222 if __name__ == "__main__":
223     tensor = torch.rand([1, 3, 224, 224])
224     model = ResNet(img_channels=3, num_layers=18, block=BasicBlock, num_classes=1000)

```

```

224     print(model)
225
226     # Total parameters and trainable parameters.
227     total_params = sum(p.numel() for p in model.parameters())
228     print(f"{total_params:,} total parameters.")
229     total_trainable_params = sum(
230         p.numel() for p in model.parameters() if p.requires_grad
231     )
232     print(f"{total_trainable_params:,} training parameters.")
233
234     output = model(tensor)

```

4.4.2 resnet18_torchvision.py

```

1  import torchvision.models as models
2  import torch.nn as nn
3
4
5  def build_model(pretrained=True, fine_tune=True, num_classes=1):
6      if pretrained:
7          print("[INFO]: Loading pre-trained weights")
8          model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
9      elif not pretrained:
10         print("[INFO]: Not loading pre-trained weights")
11         model = models.resnet18(weights=None)
12     if fine_tune:
13         print("[INFO]: Fine-tuning all layers...")
14         for params in model.parameters():
15             params.requires_grad = True
16     elif not fine_tune:
17         print("[INFO]: Freezing hidden layers...")
18         for params in model.parameters():
19             params.requires_grad = False
20
21     # Change the final classification head, it is trainable.
22     model.fc = nn.Linear(512, num_classes)
23     return model
24
25
26  if __name__ == "__main__":
27      model = build_model(num_classes=1000)
28      # Total parameters and trainable parameters.
29      total_params = sum(p.numel() for p in model.parameters())
30      print(f"{total_params:,} total parameters.")
31      total_trainable_params = sum(
32          p.numel() for p in model.parameters() if p.requires_grad
33      )
34      print(f"{total_trainable_params:,} training parameters.")

```

4.4.3 wideresnet.py

```

1  import math
2  import torch
3  import torch.nn as nn
4  import torch.nn.functional as F
5
6

```

```

7 class BasicBlock(nn.Module):
8     def __init__(self, in_planes, out_planes, stride, dropRate=0.0):
9         super(BasicBlock, self).__init__()
10        self.bn1 = nn.BatchNorm2d(in_planes)
11        self.relu1 = nn.ReLU(inplace=True)
12        self.conv1 = nn.Conv2d(in_planes, out_planes, kernel_size=3, stride=stride,
13                                padding=1, bias=False)
14        self.bn2 = nn.BatchNorm2d(out_planes)
15        self.relu2 = nn.ReLU(inplace=True)
16        self.conv2 = nn.Conv2d(out_planes, out_planes, kernel_size=3, stride=1,
17                                padding=1, bias=False)
18        self.droprate = dropRate
19        self.equalInOut = (in_planes == out_planes)
20        self.convShortcut = (not self.equalInOut) and nn.Conv2d(in_planes, out_planes,
21                                                                    kernel_size=1, stride=stride,
22                                                                    padding=0, bias=False) or None
23    def forward(self, x):
24        if not self.equalInOut:
25            x = self.relu1(self.bn1(x))
26        else:
27            out = self.relu1(self.bn1(x))
28        out = self.relu2(self.bn2(self.conv1(out if self.equalInOut else x)))
29        if self.droprate > 0:
30            out = F.dropout(out, p=self.droprate, training=self.training)
31        out = self.conv2(out)
32        return torch.add(x if self.equalInOut else self.convShortcut(x), out)
33
34 class NetworkBlock(nn.Module):
35     def __init__(self, nb_layers, in_planes, out_planes, block, stride, dropRate=0.0):
36         super(NetworkBlock, self).__init__()
37         self.layer = self._make_layer(block, in_planes, out_planes, nb_layers, stride,
38                                         dropRate)
39     def _make_layer(self, block, in_planes, out_planes, nb_layers, stride, dropRate):
40         layers = []
41         for i in range(int(nb_layers)):
42             layers.append(block(i == 0 and in_planes or out_planes, out_planes, i == 0
43                                and stride or 1, dropRate))
44         return nn.Sequential(*layers)
45     def forward(self, x):
46         return self.layer(x)
47
48 class WideResNet(nn.Module):
49     def __init__(self, depth, num_classes, widen_factor=1, dropRate=0.0):
50         super(WideResNet, self).__init__()
51         nChannels = [16, 16*widen_factor, 32*widen_factor, 64*widen_factor]
52         assert((depth - 4) % 6 == 0)
53         n = (depth - 4) / 6
54         block = BasicBlock
55         # 1st conv before any network block
56         self.conv1 = nn.Conv2d(3, nChannels[0], kernel_size=3, stride=1,
57                                 padding=1, bias=False)
58         # 1st block
59         self.block1 = NetworkBlock(n, nChannels[0], nChannels[1], block, 1, dropRate)
60         # 2nd block
61         self.block2 = NetworkBlock(n, nChannels[1], nChannels[2], block, 2, dropRate)
62         # 3rd block
63         self.block3 = NetworkBlock(n, nChannels[2], nChannels[3], block, 2, dropRate)
64         # global average pooling and classifier

```

```

62     self.bn1 = nn.BatchNorm2d(nChannels[3])
63     self.relu = nn.ReLU(inplace=True)
64     self.fc = nn.Linear(nChannels[3], num_classes)
65     self.nChannels = nChannels[3]
66
67     for m in self.modules():
68         if isinstance(m, nn.Conv2d):
69             nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
70         elif isinstance(m, nn.BatchNorm2d):
71             m.weight.data.fill_(1)
72             m.bias.data.zero_()
73         elif isinstance(m, nn.Linear):
74             m.bias.data.zero_()
75     def forward(self, x):
76         out = self.conv1(x)
77         out = self.block1(out)
78         out = self.block2(out)
79         out = self.block3(out)
80         out = self.relu(self.bn1(out))
81         out = F.avg_pool2d(out, 8)
82         out = out.view(-1, self.nChannels)
83         return self.fc(out), out

```

4.4.4 training_utils.py

```

1 import torch
2
3 from tqdm import tqdm
4 import numpy as np
5 import torch.nn.functional as F
6
7
8 def compute_cluster(train_repr, y_predict_train):
9
10     class_dict = {key: [] for key in torch.unique(y_predict_train).tolist()}
11     for sub_train_repr, sub_y_predict_train in zip(train_repr, y_predict_train):
12         class_dict[sub_y_predict_train.tolist()].append(sub_train_repr)
13
14     mean_var_batch_dict = {}
15     for key, value in class_dict.items():
16         temp_mean = torch.mean(torch.stack(value), dim=0)
17         temp_var = torch.mean(torch.stack([torch.linalg.norm(f - temp_mean) ** 2 for f in
18             value]))
19         mean_var_batch_dict[key] = [temp_mean, temp_var]
20
21     # calculate the clustering values
22     all_train_cdnvs = []
23     mean_list = []
24     var_class_cluster_list = []
25     for c1 in class_dict.keys():
26         for c2 in class_dict.keys():
27             if c2 == c1:
28                 continue
29             mu1, var1 = mean_var_batch_dict[c1]
30             mu2, var2 = mean_var_batch_dict[c2]
31             temp_result = (var1 + var2) / (2 * torch.linalg.norm(mu1 - mu2) ** 2)
32             var_class_cluster_list.append(torch.tensor([c1, var1, c2, var2, temp_result])
33 )

```



```

32         mean_list.append(torch.stack((mu1,mu2), dim=0))
33         all_train_cdnvs.append(temp_result)
34     all_train_cdnvs = torch.mean(torch.stack(all_train_cdnvs))
35     return all_train_cdnvs, mean_list, var_class_cluster_list
36
37
38 def ortho_rows_fro(W: torch.Tensor, normalize_type: str = "per_entry") -> torch.Tensor:
39     """
40     Encourages row-orthonormality:  $WW^T \approx I_k$ 
41     normalize: "none" | "per_row" | "per_entry"
42     """
43     G = W @ W.t() # [k,k] Gram of rows
44     k = W.shape[0]
45     I = torch.eye(k, device=W.device, dtype=W.dtype)
46     sse = ((G - I) ** 2).sum() #  $\|WW^T - I\|_F^2$ 
47     if normalize_type == "per_entry":
48         return sse / (k * k)
49     elif normalize_type == "per_row":
50         return sse / k
51     else:
52         return sse
53
54 def frobenius_cap(W, max_norm):
55     frob = torch.linalg.norm(W, ord='fro') # Frobenius norm
56     if frob > max_norm:
57         W.mul_(float(max_norm / frob))
58     return float(frob)
59
60 def spectral_cap(W, s_max=5.0, n_iter=1):
61     # Estimate sigma_max(W) with power iteration
62     u = torch.randn(W.size(0), device=W.device)
63     for _ in range(n_iter): # -12 iters usually enough
64         v = (W.t() @ u); v /= (v.norm() + 1e-12)
65         u = (W @ v); u /= (u.norm() + 1e-12)
66     sigma = torch.dot(u, W @ v) # estimated spectral norm
67     if sigma > s_max:
68         W.mul_(float(s_max / sigma))
69     return float(sigma)
70
71 # Training function.
72 def train(model, trainloader, optimizer, criterion, device, args, current_epoch,
73         norm_record_dict):
74     model.train()
75     print("Training")
76     train_running_loss = 0.0
77     train_running_correct = 0
78     counter = 0
79     weights_norm_list, sharpness_list, sharpness2_list, hessian_first_list,
80         hessian_second_list, hessian_list = [], [], [], [], [], []
81     train_repr_list, y_predict_train_list = [], []
82     for i, data in tqdm(enumerate(trainloader), total=len(trainloader), disable=True):
83         counter += 1
84         image, labels = data
85         image = image.to(device)
86         labels = labels.to(device)
87         optimizer.zero_grad()
88         # Forward pass.
89         if args["model"] == "resnet18" or "wideresnet" in args["model"]:

```

```

88         outputs, second_last_outputs = model(image)
89
90     if args["model"] == "efficientnetb0" or args["model"] == "efficientnet_v2":
91         outputs = model(image)
92
93     # Calculate the loss.
94     if args["loss_func"] == "leaky_ramp":
95         loss = criterion(outputs, labels, args["leaky_coef"])
96     else:
97         loss = criterion(outputs, labels)
98
99
100    # Calculate the accuracy.
101    _, preds = torch.max(outputs.data, 1)
102    train_running_correct += (preds == labels).sum().item()
103
104    if args["use_regulation"]:
105        if args["use_pow"]:
106            metric_loss, _, _ = compute_cluster(second_last_outputs, labels)
107
108            loss = loss - args["lambda"] * torch.mean(metric_loss)
109
110        with torch.no_grad():
111            probs = torch.softmax(outputs, dim=1)
112            hessian_first = torch.sum(torch.mul(probs, 1 - probs), dim=1)
113
114            hessian_second = torch.sum(torch.mul(second_last_outputs,
115                                                second_last_outputs), dim=1)
116            weights_norm = torch.linalg.norm(model.fc.weight)
117
118            hessian = torch.mul(hessian_first, hessian_second)
119            sharpness = weights_norm * hessian
120            sharpness2 = weights_norm * weights_norm * hessian
121        else:
122            if args["tau"]:
123                probs = torch.softmax(outputs / args["tau"], dim=1)
124            else:
125                probs = torch.softmax(outputs, dim=1)
126            hessian_first = torch.sum(torch.mul(probs, 1 - probs), dim=1)
127
128            if args["model"] == "resnet18" or "wideresnet" in args["model"]:
129                if args["phi_norm"]:
130                    second_last_outputs = second_last_outputs / (second_last_outputs.
131                                                                norm(dim=1, keepdim=True) + 1e-8)
132                    hessian_second = torch.sum(torch.mul(second_last_outputs,
133                                                        second_last_outputs), dim=1)
134                    weights_norm = torch.linalg.norm(model.fc.weight)
135                if args["model"] == "efficientnetb0" or args["model"] == "efficientnet_v2":
136                    second_last_outputs = F.adaptive_avg_pool2d(model.forward_features(
137                                                                image), (1, 1)).flatten(1)
138                    if args["phi_norm"]:
139                        second_last_outputs = second_last_outputs / (second_last_outputs.
140                                                                    norm(dim=1, keepdim=True) + 1e-8)
141                        hessian_second = torch.sum(torch.mul(second_last_outputs,
142                                                            second_last_outputs), dim=1)
143                        weights_norm = torch.linalg.norm(model.classifier.weight)

```

```

139         hessian = torch.mul(hessian_first, hessian_second)
140         sharpness = weights_norm * hessian
141         sharpness2 = weights_norm * weights_norm * hessian
142
143     if args["loss_ortho"]:
144         reg_ortho = ortho_rows_fro(model.fc.weight, normalize_type="per_entry")
145         loss = loss - args["lambda"] * torch.mean(sharpness2) + args["ortho"] * reg_ortho
146         print(reg_ortho)
147     if args["loss_type1"]:
148         loss = loss - args["lambda"] * torch.mean(sharpness2)
149     if args["loss_type2"]:
150         loss = loss - args["lambda"] * torch.abs(torch.mean(sharpness2)) / (
151             np.abs(loss.item()) + args["loss_type2_coef"])
152 else:
153
154     with torch.no_grad():
155         probs = torch.softmax(outputs, dim=1)
156         hessian_first = torch.sum(torch.mul(probs, 1 - probs), dim=1)
157
158         if args["model"] == "resnet18" or "wideresnet" in args["model"]:
159             hessian_second = torch.sum(torch.mul(second_last_outputs,
160                 second_last_outputs), dim=1)
161             weights_norm = torch.linalg.norm(model.fc.weight)
162         if args["model"] == "efficientnetb0" or args["model"] == "efficientnet_v2":
163             second_last_outputs = F.adaptive_avg_pool2d(model.forward_features(
164                 image), (1, 1)).flatten(1)
165             hessian_second = torch.sum(torch.mul(second_last_outputs,
166                 second_last_outputs), dim=1)
167             weights_norm = torch.linalg.norm(model.classifier.weight)
168
169         hessian = torch.mul(hessian_first, hessian_second)
170         sharpness = weights_norm * hessian
171         sharpness2 = weights_norm * weights_norm * hessian
172
173 # collect the results
174 with torch.no_grad():
175     weights_norm_list.append(weights_norm)
176     hessian_list.append(hessian)
177     hessian_first_list.append(hessian_first)
178     hessian_second_list.append(hessian_second)
179     sharpness_list.append(sharpness)
180     sharpness2_list.append(sharpness2)
181     train_repr_list.append(second_last_outputs)
182     y_predict_train_list.append(labels)
183     train_running_loss += loss.item()
184
185 # Backpropagation
186 loss.backward()
187
188 # print("=====")

```

```

190     # Update the weights.
191     optimizer.step()
192     if args["weight_cap"]:
193         # print("always yes")
194         with torch.no_grad():
195             if args["cap_type"] == "spectral":
196                 _ = spectral_cap(model.fc.weight, s_max=5.0, n_iter=1)
197             elif args["cap_type"] == "frob":
198                 _ = frobenius_cap(model.fc.weight, args["f_max_norm"])
199
200
201     # Loss and accuracy for the complete epoch.
202     epoch_loss = train_running_loss / counter
203     epoch_acc = 100.0 * (train_running_correct / len(trainloader.dataset))
204
205     # sort the computation results here
206     with torch.no_grad():
207         sharpness_list = torch.mean(torch.cat(sharpness_list))
208         sharpness2_list = torch.mean(torch.cat(sharpness2_list))
209         hessian_list = torch.mean(torch.cat(hessian_list))
210         hessian_first_list = torch.mean(torch.cat(hessian_first_list))
211         hessian_second_list = torch.mean(torch.cat(hessian_second_list))
212         weights_norm_list = torch.mean(torch.tensor(weights_norm_list))
213         train_cdnvs_list, mean_list, var_class_cluster_list = compute_cluster(torch.cat(
214             train_repr_list), torch.cat(y_predict_train_list))
215
216     return epoch_loss, epoch_acc, sharpness_list, sharpness2_list, weights_norm_list,
217         hessian_list, hessian_first_list, hessian_second_list, train_cdnvs_list,
218         mean_list, var_class_cluster_list
219
220 # Validation function.
221 def validate(model, testloader, criterion, device, args):
222     model.eval()
223     print("Validation")
224     valid_running_loss = 0.0
225     valid_running_correct = 0
226     counter = 0
227
228     with torch.no_grad():
229         for i, data in tqdm(enumerate(testloader), total=len(testloader), disable=True):
230             counter += 1
231
232             image, labels = data
233             image = image.to(device)
234             labels = labels.to(device)
235             if args["model"] == "resnet18" or "wideresnet" in args["model"]:
236                 # Forward pass.
237                 outputs, _ = model(image)
238             if args["model"] == "efficientnetb0" or args["model"] == "efficientnet_v2":
239                 outputs = model(image)
240             # Calculate the loss.
241
242             if args["loss_func"] == "leaky_ramp":
243                 loss = criterion(outputs, labels, args["leaky_coef"])
244             else:

```

```

245         loss = criterion(outputs, labels)
246         valid_running_loss += loss.item()
247         # Calculate the accuracy.
248         _, preds = torch.max(outputs.data, 1)
249         valid_running_correct += (preds == labels).sum().item()
250
251     # Loss and accuracy for the complete epoch.
252     epoch_loss = valid_running_loss / counter
253     epoch_acc = 100.0 * (valid_running_correct / len(testloader.dataset))
254     return epoch_loss, epoch_acc

```

4.4.5 train.py

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import argparse
5 import numpy as np
6 import random
7
8 from resnet18 import ResNet, BasicBlock, MResNet, MBasicBlock
9 from resnet18_torchvision import build_model
10 from training_utils import train, validate
11 from utils import save_plots, get_data
12 import os
13 from datetime import datetime
14 import torch.nn.functional as F
15 import torchvision
16 import timm
17 from wideresnet import WideResNet
18
19 from torch.optim.lr_scheduler import CosineAnnealingLR
20
21
22
23 print(f"Process_ID: {os.getpid()}")
24 parser = argparse.ArgumentParser()
25 parser.add_argument(
26     "-m",
27     "--model",
28     default="resnet18",
29     help="choose_model_built_from_scratch_or_the_Torchvision_model"
30 )
31
32 parser.add_argument("--learning_rate", type=float, default=0.01)
33 parser.add_argument("--training_date", type=str, default="0211")
34 parser.add_argument("--random_seed", type=int, default=42)
35 parser.add_argument("--epochs", type=int, default=1)
36 parser.add_argument("--cancel_epoch", type=int, default=60, help="the_epoch_to_cancel_the_regularization")
37 parser.add_argument("--help_epoch", type=int, default=60, help="the_epoch_to_cancel_the_regularization")
38 parser.add_argument("--cancel_dur_epoch", type=int, default=10, help="the_epoch_to_cancel_the_regularization")
39 parser.add_argument("--scheduler_stop_epoch", type=int, default=10, help="the_epoch_to_cancel_the_regularization")
40 parser.add_argument("--batch_size", type=int, default=64)
41 parser.add_argument("--weight_decay", type=float, default=0.0)

```

```

42 parser.add_argument("--use_regulation", type=int, default=0, help="using regularizer, if do not use NCC regularizer, then use relative flatness regularizer")
43 parser.add_argument("--use_regulation_help", type=int, default=0, help="aha")
44 parser.add_argument("--lambda", type=float, default=1e-4)
45 parser.add_argument("--clip_value", type=float, default=0.0)
46 parser.add_argument("--clip_norm", type=float, default=0.0)
47 parser.add_argument("--optim_type", type=str, default="sgd")
48 parser.add_argument("--training_size", type=float, default=0.0)
49 parser.add_argument("--tau", type=float, default=0.0)
50 parser.add_argument("--eta", type=float, default=0.0)
51 parser.add_argument("--mnetwork", type=int, default=1)
52 parser.add_argument("--phi_norm", type=int, default=1)
53 parser.add_argument("--weight_cap", type=int, default=0)
54 parser.add_argument("--leaky_coef", type=float, default=0.3)
55 parser.add_argument("--cap_type", type=str, default="frob")
56 parser.add_argument("--use_pow", type=int, default=0, help="using NCC regularizer")
57 parser.add_argument('--loss_type1', type=int, default=0, help='Random seed for reproducibility (default: 42)')
58 parser.add_argument('--loss_type2', type=int, default=0, help='Random seed for reproducibility (default: 42)')
59 parser.add_argument('--loss_ortho', type=int, default=0, help='Random seed for reproducibility (default: 42)')
60 parser.add_argument('--ortho', type=float, default=0, help='Random seed for reproducibility (default: 42)')
61 parser.add_argument('--loss_type2_coef', type=float, default=0, help='Random seed for reproducibility (default: 42)')
62 parser.add_argument('--loss_func', type=str, default="ramp", help='loss function')
63 parser.add_argument('--f_max_norm', type=float, default=10.0, help='loss function')
64 parser.add_argument('--num_workers', type=int, default=0, help='Random seed for reproducibility (default: 42)')
65
66
67
68
69 args = vars(parser.parse_args())
70
71 # Set seed.
72 seed = args["random_seed"]
73 torch.manual_seed(seed)
74 torch.cuda.manual_seed(seed)
75 torch.backends.cudnn.deterministic = True
76 torch.backends.cudnn.benchmark = False
77 np.random.seed(seed)
78 random.seed(seed)
79
80 # Learning and training parameters.
81 epochs = args["epochs"]
82 batch_size = args["batch_size"]
83 learning_rate = args["learning_rate"]
84 device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
85
86 train_loader, valid_loader = get_data(training_ratio=args["training_size"], batch_size=batch_size,
    batch_size, n_workers=args["num_workers"], network_model=args["model"])
87
88 # Make dir to store data
89 output_dir = "Seed{}_Date{}_MT{}_MRes{}_EP{}_CANEP{}_TSIZE{}_BSZ{}_LF{}_LKCF{}_OPTM{}_LR{}_WD{}_REGR{}_PHIN{}_LAMBDA{}_TAU{}_LT1{}_WCAP{}_CAPT{}_FMAX{}".format(
    args["random_seed"],

```

```

91         args["training_date"],
92         args["model"],
93         args["mnetwork"],
94         args["epochs"],
95         args["cancel_epoch"],
96         args["training_size"],
97         args["batch_size"],
98         args["loss_func"],
99         args["leaky_coef"],
100        args["optim_type"],
101        args["learning_rate"],
102        args["weight_decay"],
103        args["use_regulation"],
104        args["phi_norm"],
105        args["lambda"],
106        args["tau"],
107        args["loss_type1"],
108        args["weight_cap"],
109        args["cap_type"],
110        args["f_max_norm"]
111    )
112    os.makedirs(output_dir, exist_ok=True)
113    os.makedirs("outputs", exist_ok=True)
114
115    # Define model based on the argument parser string.
116    if args["model"] == "resnet18":
117        print("[INFO]: Training ResNet18 built from scratch...")
118        if args["mnetwork"]:
119            model = MResNet(MBasicBlock, [2,2,2,2]).to(device)
120        else:
121            model = ResNet(img_channels=3, num_layers=18, block=BasicBlock, num_classes=10).to(device)
122        plot_name = args["model"] + "_" + output_dir
123
124    if args["model"] == "efficientnetb0":
125        print("[INFO]: Training the EfficientNet_b0 from scratch...")
126        model = timm.create_model('efficientnet_b0', pretrained=False, num_classes=10).to(device)
127        # model = build_model(pretrained=False, fine_tune=True, num_classes=10).to(device)
128        plot_name = args["model"] + "_" + output_dir
129
130    if args["model"] == "efficientnet_v2":
131        print("[INFO]: Training the EfficientNet_v2 from scratch...")
132        model = timm.create_model('efficientnetv2_rw_t', pretrained=False, num_classes=10).to(device)
133        # model = build_model(pretrained=False, fine_tune=True, num_classes=10).to(device)
134        plot_name = args["model"] + "_" + output_dir
135
136    if "wideresnet" in args["model"]:
137        print(f"[INFO]: Training the {args['model']} from scratch...")
138        wideresnet_args = args["model"].split("-")
139        model = WideResNet(int(wideresnet_args[1]), 10, int(wideresnet_args[-1]), dropRate=0.0).to(device) # depth, num_class, width, dropout rates(?)
140        plot_name = args["model"] + "_" + output_dir
141
142    if args["model"] == "resnet50":
143        print("[INFO]: Training the torchvision ResNet50 model...")
144        model = torchvision.models.resnet50(weights=None)

```

```

145     model.fc = nn.Linear(model.fc.in_features, 10)
146
147
148 # Total parameters and trainable parameters.
149 total_params = sum(p.numel() for p in model.parameters())
150 print(f"{total_params:,}_total_parameters.")
151 total_trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
152 print(f"{total_trainable_params:,}_training_parameters.")
153
154 # Create a model name dict
155 norm_record_dict = {}
156
157 def set_lr(opt, lr: float):
158     for g in opt.param_groups:
159         g["lr"] = lr
160
161 def mul_lr(opt, factor: float):
162     for g in opt.param_groups:
163         g["lr"] *= factor
164
165 def set_wd(opt, wd: float):
166     for g in opt.param_groups:
167         g["weight_decay"] = wd
168
169 def squared_hinge_loss(logits, targets, margin=1.0):
170     """
171     Multi-class squared hinge loss.
172     logits: [N, C], targets: [N]
173     """
174     N, C = logits.shape
175     y_onehot = F.one_hot(targets, num_classes=C).float() # [N, C]
176
177     z_y = (logits * y_onehot).sum(dim=1, keepdim=True) # true logit [N,1]
178     z_max_other = (logits - 1e9*y_onehot).amax(dim=1, keepdim=True) # max non-true [N,1]
179     m = z_y - z_max_other
180
181     hinge = torch.relu(margin - m).squeeze(1) # [N]
182     return (hinge * hinge).mean()
183
184 def logistic_hinge_loss(logits, targets):
185     """
186     Multi-class logistic hinge (softplus on margin).
187     logits: [N, C], targets: [N]
188     """
189     N, C = logits.shape
190     y_onehot = F.one_hot(targets, num_classes=C).float()
191
192     z_y = (logits * y_onehot).sum(dim=1, keepdim=True)
193     z_max_other = (logits - 1e9*y_onehot).amax(dim=1, keepdim=True)
194     m = (z_y - z_max_other).squeeze(1)
195
196     return F.softplus(-m).mean() # log(1 + exp(-m))
197
198 def logistic_hinge_max(logits, targets, margin=0.1):
199     y1h = F.one_hot(targets, num_classes=logits.size(1)).float()
200     z_y = (logits * y1h).sum(dim=1, keepdim=True)
201     z_max_other = (logits - 1e9 * y1h).amax(dim=1, keepdim=True)
202     v = margin - (z_y - z_max_other) # violation

```



```

203     return F.softplus(v).mean()
204
205
206 def leaky_ramp_loss(logits, targets, leak):
207     y1h = F.one_hot(targets, num_classes=logits.size(1)).float()
208     z_y = (logits * y1h).sum(dim=1, keepdim=True)
209     z_max_other = (logits - 1e9*y1h).amax(dim=1, keepdim=True)
210     m = (z_y - z_max_other).squeeze(1)
211     # standard ramp part for 0 < m < 1
212     mid = torch.clamp(1.0 - m, min=0.0, max=1.0)
213     # add a small linear tail when m <= 0
214     tail = (1.0 - m) # slope = 1
215     loss = torch.where(m <= 0, 1.0 + leak * (tail - 1.0), mid)
216     return loss.mean()
217
218 def ramp_loss(logits, targets):
219     """
220     Ramp loss for multi-class classification.
221
222     Definition:
223         For each sample with margin  $m = z_y - \max_{\{j \neq y\}} z_j$ :
224              $L_{\text{ramp}}(m) = 0$ , if  $m \geq 1$ 
225              $= 1 - m$ , if  $0 < m < 1$ 
226              $= 1$ , if  $m \leq 0$ 
227         Final loss = mean over batch.
228
229     Args:
230         logits: [N, C] raw model outputs (no softmax)
231         targets: [N] ground-truth labels in [0, C-1]
232     Returns:
233         scalar loss (mean over batch)
234     """
235     N, C = logits.shape
236
237     # One-hot encode labels to pick true logit
238     y_onehot = F.one_hot(targets, num_classes=C).float()
239
240     # True class logit z_y
241     z_y = (logits * y_onehot).sum(dim=1, keepdim=True) # [N,1]
242
243     # Max logit among wrong classes
244     z_others = logits.masked_fill(y_onehot.bool(), -1e9)
245     z_max_other = z_others.max(dim=1, keepdim=True).values # [N,1]
246
247     # Margin = true logit - highest wrong logit
248     margin = z_y - z_max_other # [N,1]
249
250     # Ramp loss = clamp(1 - margin, 0, 1)
251     ramp = torch.clamp(1.0 - margin, min=0.0, max=1.0) # [N,1]
252
253     return ramp.mean()
254
255 # Optimizer.
256 if args["optim_type"] == "sgd":
257     optimizer = optim.SGD(model.parameters(), lr=learning_rate, momentum = 0.9,
258                             weight_decay = args["weight_decay"])
259 else:

```

```

259     optimizer = optim.AdamW(model.parameters(), lr=learning_rate, betas=(0.9, 0.999),
260                               weight_decay=args["weight_decay"], eps=1e-8)
261 # scheduler
262 # scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=args["
263     scheduler_stop_epoch"], eta_min=args["eta"])
264 # Loss function.
265 if args["loss_func"] == "ramp":
266     print("using ramp loss")
267     criterion = ramp_loss
268 elif args["loss_func"] == "logistic_hinge":
269     criterion = logistic_hinge_loss
270 elif args["loss_func"] == "squared_hinge":
271     criterion = squared_hinge_loss
272 elif args["loss_func"] == "leaky_ramp":
273     criterion = leaky_ramp_loss
274 elif args["loss_func"] == "logistic_hinge_max":
275     criterion = logistic_hinge_max
276 else:
277     print("using cross entropy loss")
278     criterion = nn.CrossEntropyLoss()
279
280 for key, value in args.items():
281     print(f"{key}: {value}")
282 start_time = datetime.now()
283
284 if __name__ == "__main__":
285     # Lists to keep track of losses and accuracies.
286     train_loss, valid_loss = [], []
287     train_acc, valid_acc = [], []
288
289     # some preparation for regularizer cancellation.
290     BASE_LR = learning_rate
291     CANCEL_EPOCH = args["cancel_epoch"] # your request
292     WD_AFTER = 2e-4
293     BUMP_X = 10.0 # 0.01 -> 0.02
294     ETA_MIN = 1e-4
295
296     scheduler = None
297     has_cancelled = False
298
299     # Start the training.
300
301     for epoch in range(epochs):
302         print(f"[INFO]: Epoch {epoch+1} of {epochs}")
303         # ---- resume safety guard ----
304
305         if epoch > CANCEL_EPOCH and args["use_regulation"]:
306             args["use_regulation"] = 0
307             print(f"[INFO]: Forcing use_regulation=0 at epoch {epoch+1} (resume safety)")
308
309         if (not has_cancelled) and (epoch == CANCEL_EPOCH) and args["use_regulation"]:
310             args["use_regulation"] = 0
311             has_cancelled = True
312             print(f"[INFO]: Cancel Regularizer at Epoch {epoch+1} (human-counted)")
313             print(f"[INFO]: Turning on weight decay and attaching cosine scheduler")
314

```

```

315         # mutate existing optimizer (no rebuild)
316         set_wd(optimizer, WD_AFTER)
317         mul_lr(optimizer, BUMP_X) # small bump to escape sharp basin
318
319         remaining = max(1, epochs - epoch - 1) # e.g., 300 - 150 - 1 = 149
320         scheduler = CosineAnnealingLR(optimizer, T_max=remaining, eta_min=ETA_MIN)
321
322         # sanity log
323         print("[LRs]", [round(g["lr"], 6) for g in optimizer.param_groups],
324               "[WDs]", [g["weight_decay"] for g in optimizer.param_groups])
325
326         train_epoch_loss, train_epoch_acc, sharpness, sharpness2, weights_norm, hessian,
327         hessian_first, hessian_second, cluster_value, mean_list,
328         var_class_cluster_list = train(
329             model, train_loader, optimizer, criterion, device, args, epoch,
330             norm_record_dict
331         )
332
333         valid_epoch_loss, valid_epoch_acc = validate(
334             model, valid_loader, criterion, device, args
335         )
336
337         if scheduler is not None:
338             scheduler.step()
339
340         train_loss.append(train_epoch_loss)
341         valid_loss.append(valid_epoch_loss)
342         train_acc.append(train_epoch_acc)
343         valid_acc.append(valid_epoch_acc)
344
345         # save values here:
346         with torch.no_grad():
347             np.save(output_dir + "/sharpness_" + str(epoch), sharpness.detach().cpu().
348                     numpy())
349             np.save(output_dir + "/sharpness2_" + str(epoch), sharpness2.detach().cpu().
350                     numpy())
351             np.save(output_dir + "/hessian_" + str(epoch), hessian.detach().cpu().numpy())
352             np.save(output_dir + "/hessian_first_" + str(epoch), hessian_first.detach().
353                     cpu().numpy())
354             np.save(output_dir + "/hessian_second_" + str(epoch), hessian_second.detach().
355                     cpu().numpy())
356             np.save(output_dir + "/weights_norm_" + str(epoch), weights_norm.numpy())
357             np.save(output_dir + "/cluster_value_" + str(epoch), cluster_value.detach().
358                     cpu().numpy())
359             np.save(output_dir + "/mean_value_" + str(epoch), torch.stack(mean_list).
360                     detach().cpu().numpy())
361             np.save(output_dir + "/var_classcluster_" + str(epoch), torch.stack(
362                 var_class_cluster_list).detach().cpu().numpy())
363             np.save(output_dir + "/train_acc_" + str(epoch), train_epoch_acc)
364             np.save(output_dir + "/train_loss_" + str(epoch), train_epoch_loss)
365             np.save(output_dir + "/val_loss_" + str(epoch), valid_epoch_loss)
366             np.save(output_dir + "/val_acc_" + str(epoch), valid_epoch_acc)
367
368         print(
369             f"Training_loss: {train_epoch_loss:.3f}, training_acc: {train_epoch_acc:.3f}"
370         )
371         print(

```

```

362         f"Validation loss: {valid_epoch_loss:.3f}, validation acc: {valid_epoch_acc
363             :.3f}"
364     )
365     print("-" * 50)
366     print(datetime.now())
367
368     # Save the loss and accuracy plots.
369     save_plots(train_acc, valid_acc, train_loss, valid_loss, name=plot_name)
370     end_time = datetime.now()
371     print('Duration: {}'.format(end_time - start_time))
372     print("TRAINING COMPLETE")
373
374     # print the name and value of norm
375     print("lower bounds")
376     for key, value in norm_record_dict.items():
377         print(f"Layer name: {key} and max gradient value: {value}")

```

4.4.6 utils.py

```

1 import matplotlib.pyplot as plt
2 import os
3
4 from torch.utils.data import DataLoader, Subset
5 from torchvision import datasets, transforms
6 from torchvision.transforms import ToTensor
7
8 plt.style.use("ggplot")
9
10
11 def get_data(training_ratio, batch_size, n_workers, network_model):
12
13     if "wideresnet" in network_model:
14         # Normalize using CIFAR-10 mean and std (in [0,1] range)
15         transform = transforms.Compose([
16             transforms.ToTensor(),
17             transforms.Normalize(mean=[125.3/255.0, 123.0/255.0, 113.9/255.0],
18                                 std=[63.0/255.0, 62.1/255.0, 66.7/255.0])
19         ])
20     else:
21
22         transform = transforms.Compose([transforms.ToTensor(),
23                                         transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5,
24                                                         0.5, 0.5])])
25
26         # transform = transforms.Compose([transforms.ToTensor(),
27         #                                 transforms.Normalize(mean=[0.4914, 0.4822, 0.4465],
28         #                                                         std=[0.2023, 0.1994, 0.2010])])
29
30     # CIFAR10 training dataset.
31     dataset_train = datasets.CIFAR10(
32         root="data",
33         train=True,
34         download=True,
35         transform=transform,
36     )
37
38     # CIFAR10 validation dataset.
39     dataset_valid = datasets.CIFAR10(
40         root="data",

```

```

37     train=False,
38     download=True,
39     transform=transform,
40 )
41
42 if training_ratio > 0.0:
43     # Get 10% of the training data
44     total_size = len(trainset)
45     subset_size = int(training_ratio * total_size) # 10% of the total training data
46     indices = np.random.choice(total_size, subset_size, replace=False) # Randomly
47         select indices
48
49     # Create a subset dataset
50     subset_trainset = Subset(trainset, indices)
51
52 if "wideresnet" in network_model:
53     # Create data loaders.
54     if training_ratio > 0.0:
55         # Create a DataLoader for the subset dataset
56         trainloader = DataLoader(subset_trainset, batch_size=batch_size, shuffle=True)
57     else:
58         train_loader = DataLoader(dataset_train, batch_size=batch_size, shuffle=True)
59
60     valid_loader = DataLoader(dataset_valid, batch_size=batch_size, shuffle=False)
61
62 else:
63     # Create data loaders.
64     if training_ratio > 0.0:
65         # Create a DataLoader for the subset dataset
66         trainloader = DataLoader(subset_trainset, batch_size=batch_size, shuffle=True,
67                                 , num_workers=n_workers, pin_memory=True)
68     else:
69         train_loader = DataLoader(dataset_train, batch_size=batch_size, shuffle=True,
70                                 num_workers=n_workers, pin_memory=True)
71
72     valid_loader = DataLoader(dataset_valid, batch_size=batch_size, shuffle=False,
73                             num_workers=n_workers, pin_memory=True)
74
75 return train_loader, valid_loader
76
77 def save_plots(train_acc, valid_acc, train_loss, valid_loss, name=None):
78     """
79     Function to save the loss and accuracy plots to disk.
80     """
81     # Accuracy plots.
82     plt.figure(figsize=(10, 7))
83     plt.plot(train_acc, color="tab:blue", linestyle="-", label="train_accuracy")
84     plt.plot(valid_acc, color="tab:red", linestyle="-", label="validation_accuracy")
85     plt.xlabel("Epochs")
86     plt.ylabel("Accuracy")
87     plt.legend()
88     plt.savefig(os.path.join("outputs", name + "_accuracy.png"))

```

4.4.7 run_script.sh

```

1 #!/bin/bash
2

```

```

3 readonly LEARNING_RATE=1e-02
4 readonly DATE=$(date +%m%d)
5 readonly SEED=42
6 readonly TRAINING_EPOCH=400
7 readonly CANCEL_EPOCH=200
8 readonly HELP_EPOCH=10000
9 readonly BSZ=64
10 readonly WEIGHTDECAY=0
11 readonly USE_REGR=1
12 readonly HECOEFF=1e-2
13 readonly CLIPPING_VALUE=0
14 readonly CLIPPING_NORM=0 # using norm as the clipping
15 readonly OPTM="sgd"
16 readonly TRAINING_SIZE=0.0
17 readonly USE_POW=0
18 readonly ETA=0
19 readonly SSEPOCH=0
20 readonly LOSS_TYPE1=1
21 readonly LOSS_TYPE2=0
22 readonly LOSS_TYPE2_COEF=0
23 readonly LOSS_ORTH0=0
24 readonly ORTH0=0
25 readonly MT="resnet18" # resnet18, efficientnetb0, efficientnet_v2, wideresnet-16-8
26 readonly NUM_WORKERS=0 # modify this accordingly
27 readonly LOSS_FUNC="CE" # CE, leaky_ramp, squared_hinge, logistic_hinge,
28 readonly LEAKY=0
29 readonly TAU=2.0
30 readonly WEIGHT_CAP=1
31 readonly CAP_TYPE="frob" # frob or spectral
32 readonly MRES=0
33 readonly PHI_NORM=1
34 readonly F_MAX_NORM=50.0
35
36 nohup python -u train.py \
37     --learning_rate=${LEARNING_RATE} \
38     --eta=${ETA} \
39     --training_date=${DATE} \
40     --mnetwork=${MRES} \
41     --model=${MT} \
42     --loss_func=${LOSS_FUNC} \
43     --leaky_coef=${LEAKY} \
44     --random_seed=${SEED} \
45     --epochs=${TRAINING_EPOCH} \
46     --num_workers=${NUM_WORKERS} \
47     --scheduler_stop_epoch=${SSEPOCH} \
48     --tau=${TAU} \
49     --weight_cap=${WEIGHT_CAP} \
50     --cap_type=${CAP_TYPE} \
51     --f_max_norm=${F_MAX_NORM} \
52     --cancel_epoch=${CANCEL_EPOCH} \
53     --help_epoch=${HELP_EPOCH} \
54     --batch_size=${BSZ} \
55     --weight_decay=${WEIGHTDECAY} \
56     --use_regulation=${USE_REGR} \
57     --phi_norm=${PHI_NORM} \
58     --use_pow=${USE_POW} \
59     --lambda=${HECOEFF} \
60     --loss_type1=${LOSS_TYPE1} \

```

```
61 --loss_type2=${LOSS_TYPE2} \  
62 --loss_type2_coef=${LOSS_TYPE2_COEF} \  
63 --loss_ortho=${LOSS_ORTH0} \  
64 --ortho=${ORTH0} \  
65 --clip_value=${CLIPPING_VALUE} \  
66 --clip_norm=${CLIPPING_NORM} \  
67 --optim_type=${OPTM} \  
68 --training_size=${TRAINING_SIZE} \  
69 > Seed${SEED}_Date${DATE}_MT${MT}_MRes${MRES}_EP${TRAINING_EPOCH}_CANEP${CANCEL_EPOCH}  
   _TSZ${TRAINING_SIZE}_BSZ${BSZ}_LF${LOSS_FUNC}_LKCF${LEAKY}_OPTM${OPTM}_LR${  
   LEARNING_RATE}_WD${WEIGHTDECAY}_REGR${USE_REGR}_PHIN${PHI_NORM}_LAMBDA${HECOEFF}  
   _TAU${TAU}_LT1${LOSS_TYPE1}_WCAP${WEIGHT_CAP}_CAPT${CAP_TYPE}_FMAX${F_MAX_NORM}  
   _trainlog.txt 2>&1 &
```

4.5 Relative Flatness: rf_vit/

4.5.1 train_vit.py

```

1 import os
2 import time
3 import argparse
4 import numpy as np
5 from tqdm import tqdm
6
7 import torch
8 import torch.nn as nn
9 import torch.optim as optim
10 import torch.nn.functional as F
11 from torch.utils.data import DataLoader, Subset
12 from torchvision import datasets, transforms, models
13 import timm
14 from datetime import datetime
15 import random
16 from torch.optim.lr_scheduler import CosineAnnealingLR
17
18 print(f"Process_ID: {os.getpid()}")
19
20
21 def set_lr(opt, lr: float):
22     for g in opt.param_groups:
23         g["lr"] = lr
24
25 def mul_lr(opt, factor: float):
26     for g in opt.param_groups:
27         g["lr"] *= factor
28
29 def set_wd(opt, wd: float):
30     for g in opt.param_groups:
31         g["weight_decay"] = wd
32
33 def frobenius_cap(W, max_norm):
34     frob = torch.linalg.norm(W, ord='fro') # Frobenius norm
35     if frob > max_norm:
36         W.mul_(float(max_norm / frob))
37     return float(frob)
38
39 def spectral_cap(W, s_max=5.0, n_iter=1):
40     # Estimate sigma_max(W) with power iteration
41     u = torch.randn(W.size(0), device=W.device)
42     for _ in range(n_iter): # -12 iters usually enough
43         v = (W.t() @ u); v /= (v.norm() + 1e-12)
44         u = (W @ v); u /= (u.norm() + 1e-12)
45     sigma = torch.dot(u, W @ v) # estimated spectral norm
46     if sigma > s_max:
47         W.mul_(float(s_max / sigma))
48     return float(sigma)
49
50 # Set seed for reproducibility
51 def set_seed(seed=42):
52     torch.manual_seed(seed)
53     torch.cuda.manual_seed_all(seed)
54     np.random.seed(seed)

```



```

55 random.seed(seed)
56 torch.backends.cudnn.deterministic = True
57 torch.backends.cudnn.benchmark = False
58
59 def parse_args():
60     parser = argparse.ArgumentParser(description='Train ViT on ImageNet100')
61     parser.add_argument('--data_root', default='../vit_data', type=str, help='Dataset root directory')
62     parser.add_argument('--batch_size', default=64, type=int, help='Batch size for training')
63     parser.add_argument('--epochs', default=30, type=int, help='Number of training epochs')
64     parser.add_argument("--random_seed", default=42, type=int, help="training seed")
65     parser.add_argument('--lr', default=0.01, type=float, help='Learning rate')
66     parser.add_argument('--weight_decay', default=0.05, type=float, help='Weight decay')
67     parser.add_argument('--pretrained', default=0, type=int, help='Use pretrained weights')
68     parser.add_argument('--num_workers', default=4, type=int, help='Number of data loading workers')
69     parser.add_argument('--checkpoint_dir', default='.', type=str, help='Directory to save checkpoints')
70     parser.add_argument('--log_dir', default='./logs', type=str, help='Directory to save logs')
71     parser.add_argument('--device', default='cuda', type=str, help='Device to use for training')
72     parser.add_argument("--training_date", default=2221, type=int, help="training date")
73     parser.add_argument("--cancel_epoch", default=10000, type=int, help="which epoch to cancel the regularization")
74     parser.add_argument("--clip_value", default=0.0, type=float, help="gradient clipping")
75     parser.add_argument("--use_regulation", default=0, type=int, help="use regularization")
76     parser.add_argument("--cof_lambda", default=0.0, type=float, help="reg coefficient")
77     # ----- add new arguments -----
78     parser.add_argument("--phi_norm", type=int, default=1)
79     parser.add_argument("--tau", type=float, default=2.0)
80     parser.add_argument("--weight_cap", type=int, default=1)
81     parser.add_argument("--cap_type", type=str, default="frob")
82     parser.add_argument("--f_max_norm", type=float, default=80.0)
83     parser.add_argument("--model_type", type=str, default="base")
84     parser.add_argument("--bump_x", type=float, default=5.0)
85     # parser.add_argument("--")
86     return parser.parse_args()
87
88 def get_dataloaders(args):
89     # Mean and std for ImageNet normalization
90     mean = [0.485, 0.456, 0.406]
91     std = [0.229, 0.224, 0.225]
92
93     # Transformations
94     train_transform = transforms.Compose([
95         transforms.Resize(256),
96         transforms.CenterCrop(224),
97         transforms.ToTensor(),
98         transforms.Normalize(mean, std)
99     ])
100
101     val_transform = transforms.Compose([

```

```

102     transforms.Resize(256),
103     transforms.CenterCrop(224),
104     transforms.ToTensor(),
105     transforms.Normalize(mean, std)
106 ]))
107
108 # Load your custom ImageNet100 dataset
109 # Assuming your data is organized in the standard ImageNet format with subdirectories
    for each class
110 train_dataset = datasets.ImageFolder(
111     root=os.path.join(args.data_root, 'train_data'),
112     transform=train_transform
113 )
114
115 val_dataset = datasets.ImageFolder(
116     root=os.path.join(args.data_root, 'val_data'),
117     transform=val_transform
118 )
119
120 # Create data loaders
121 train_loader = DataLoader(
122     train_dataset,
123     batch_size=args.batch_size,
124     shuffle=True,
125     num_workers=args.num_workers,
126     pin_memory=True
127 )
128
129 val_loader = DataLoader(
130     val_dataset,
131     batch_size=args.batch_size,
132     shuffle=False,
133     num_workers=args.num_workers,
134     pin_memory=True
135 )
136
137 return train_loader, val_loader
138
139 def get_model(args):
140     # num_classes = 100 # Update this if your dataset has a different number of classes
141
142     # if args.pretrained:
143     #     print("Using pretrained ViT-B/16 model...")
144     #     # model = timm.create_model('vit_tiny_patch16_224', pretrained=False,
        num_classes=100)
145     #     model = timm.create_model('vit_base_patch16_224', pretrained=True, num_classes
        =100)
146     #     # model = models.vit_b_16(weights=ViT_B_16_Weights.IMAGENET1K_V1)
147     #     # Modify the head for your number of classes
148     #     # num_fts = model.heads.head.in_features
149     #     # model.heads.head = nn.Linear(num_fts, num_classes)
150     # else:
151     #     print("Training ViT-B/16 model from scratch...")
152     #     model = timm.create_model('vit_base_patch16_224', pretrained=False, num_classes
        =100)
153     #     # model = models.vit_b_16(num_classes=num_classes)
154
155

```

```

156 num_classes = 100
157 common_kwargs = dict(
158     num_classes=num_classes,
159     drop_rate=0.0,          # MLP/head dropout
160     attn_drop_rate=0.0,    # attention prob dropout
161     drop_path_rate=0.0     # stochastic depth
162 )
163 if args.pretrained:
164     print("Using_timm_ViT-B/16_pretrained_on_ImageNet-1k...")
165     model = timm.create_model('vit_base_patch16_224', pretrained=True, **
166                             common_kwargs)
167 else:
168     if args.model_type == "tiny":
169         print("Training_timm_ViT-tiny_from_scratch...")
170         model = timm.create_model('vit_tiny_patch16_224', pretrained=False, **
171                                 common_kwargs)
172     elif args.model_type == "small":
173         print("Training_timm_ViT-small_from_scratch...")
174         model = timm.create_model('vit_small_patch16_224', pretrained=False, **
175                                 common_kwargs)
176     else:
177         print("Training_timm_ViT-B/16_from_scratch...")
178         model = timm.create_model('vit_base_patch16_224', pretrained=False, **
179                                 common_kwargs)
180 return model
181
182 def train_one_epoch(model, train_loader, criterion, optimizer, device, epoch, hparams):
183     model.train()
184     running_loss = 0.0
185     correct = 0
186     total = 0
187     weights_norm_list, sharpness_list, sharpness2_list, hessian_first_list,
188     hessian_second_list, hessian_list = [], [], [], [], [], []
189
190     pbar = tqdm(train_loader, desc=f"Epoch_{epoch+1}_{Train}", disable=True)
191     for i, (inputs, targets) in enumerate(pbar):
192
193         inputs, targets = inputs.to(device), targets.to(device)
194
195         # Zero the parameter gradients
196         optimizer.zero_grad()
197
198         # Forward pass
199         outputs = model(inputs)
200         loss = criterion(outputs, targets)
201
202         # plug in the regularization loss here
203         if hparams.use_regulation:
204             if hparams.tau:
205                 # print("yes1")
206                 probs = torch.softmax(outputs / hparams.tau, dim=1)
207             else:
208                 probs = torch.softmax(outputs, dim=1)
209
210             hessian_first = torch.sum(torch.mul(probs, 1 - probs), dim=1)
211             second_last_outputs = model.forward_features(inputs)[:,-1]
212             if hparams.phi_norm:

```

```

209         # print("yes2")
210         second_last_outputs = second_last_outputs / (second_last_outputs.norm(dim
211             =1, keepdim=True) + 1e-8)
211         hessian_second = torch.sum(torch.mul(second_last_outputs, second_last_outputs
212             ), dim=1)
212         hessian = torch.mul(hessian_first, hessian_second)
213         weights_norm = torch.linalg.norm(model.head.weight)
214         sharpness = weights_norm * hessian
215         sharpness2 = weights_norm * weights_norm * hessian
216
217
218         # regularization loss:
219         loss = loss - hparams.cof_lambda * torch.mean(sharpness2)
220     else:
221         # calculate hessian and sharpness here
222         with torch.no_grad():
223             probs = torch.softmax(outputs, dim=1)
224             hessian_first = torch.sum(torch.mul(probs, 1 - probs), dim=1)
225             second_last_outputs = model.forward_features(inputs)[:0]
226             hessian_second = torch.sum(torch.mul(second_last_outputs,
227                 second_last_outputs), dim=1)
227             hessian = torch.mul(hessian_first, hessian_second)
228             weights_norm = torch.linalg.norm(model.head.weight)
229             sharpness = weights_norm * hessian
230             sharpness2 = weights_norm * weights_norm * hessian
231
232     # Backward pass and optimize
233     loss.backward()
234     optimizer.step()
235     if hparams.weight_cap:
236         with torch.no_grad():
237             if hparams.cap_type == "spectral":
238                 _ = spectral_cap(model.head.weight, s_max=5.0, n_iter=1)
239             elif hparams.cap_type == "frob":
240                 _ = frobenius_cap(model.head.weight, hparams.f_max_norm)
241
242     # Update metrics
243     running_loss += loss.item()
244     _, predicted = outputs.max(1)
245     total += targets.size(0)
246     correct += predicted.eq(targets).sum().item()
247
248     # Update progress bar
249     pbar.set_postfix({
250         'loss': running_loss / (i + 1),
251         'acc': 100. * correct / total
252     })
253
254
255     # collect the results
256     with torch.no_grad():
257         weights_norm_list.append(float(weights_norm.detach().cpu()))
258         hessian_list.append(hessian.detach().cpu())
259         hessian_first_list.append(hessian_first.detach().cpu())
260         hessian_second_list.append(hessian_second.detach().cpu())
261         sharpness_list.append(sharpness.detach().cpu())
262         sharpness2_list.append(sharpness2.detach().cpu())
263

```

```
264
265     train_loss = running_loss / len(train_loader)
266     train_acc = 100. * correct / total
267
268
269     # sort the computation results here
270     with torch.no_grad():
271         sharpness_list = torch.mean(torch.cat(sharpness_list))
272         sharpness2_list = torch.mean(torch.cat(sharpness2_list))
273         hessian_list = torch.mean(torch.cat(hessian_list))
274         hessian_first_list = torch.mean(torch.cat(hessian_first_list))
275         hessian_second_list = torch.mean(torch.cat(hessian_second_list))
276         weights_norm_list = torch.mean(torch.tensor(weights_norm_list))
277
278     return train_loss, train_acc, sharpness_list, sharpness2_list, weights_norm_list,
           hessian_list, hessian_first_list, hessian_second_list
279
280 def validate(model, val_loader, criterion, device, epoch):
281     model.eval()
282     running_loss = 0.0
283     correct = 0
284     total = 0
285
286     with torch.no_grad():
287         pbar = tqdm(val_loader, desc=f"Epoch_{epoch+1}_{Val}", disable=True)
288         for i, (inputs, targets) in enumerate(pbar):
289
290             inputs, targets = inputs.to(device), targets.to(device)
291
292             # Forward pass
293             outputs = model(inputs)
294             loss = criterion(outputs, targets)
295
296             # Update metrics
297             running_loss += loss.item()
298             _, predicted = outputs.max(1)
299             total += targets.size(0)
300             correct += predicted.eq(targets).sum().item()
301
302             # Update progress bar
303             pbar.set_postfix({
304                 'loss': running_loss / (i + 1),
305                 'acc': 100. * correct / total
306             })
307
308     val_loss = running_loss / len(val_loader)
309     val_acc = 100. * correct / total
310
311
312     return val_loss, val_acc
313
314
315 def main():
316     start_time = datetime.now()
317     args = parse_args()
318
319     # Print parameters
320     print("\nArguments:")
```

```
321     for arg in vars(args):
322         print(f"_{arg}:_{getattr(args, _arg)}")
323
324     # Create directories
325     args.checkpoint_dir = os.path.join(args.checkpoint_dir, "Seed{} _Date{} _BSZ{}
        _PRETRAINED{} _MD{} _LR{} _EPOCH{} _CANCEL{} _BUMP{} _WD{} _REGR{} _PHIN{} _LAMBDA{} _TAU{}
        _WCAP{} _CAPT{} _FMAX{}".format(args.random_seed,
326
327
328
329
330
331
332
333
334
335
```

```
336
337
338
339
340
341
342 os.makedirs(args.checkpoint_dir, exist_ok=True)
343 os.makedirs(args.log_dir, exist_ok=True)
344
345 # Set up device
346 device = torch.device(args.device if torch.cuda.is_available() else "cpu")
347 print(f"Using device: {device}")
348
349 # Set random seed
350 set_seed(args.random_seed)
351
352 # Get data loaders
353 train_loader, val_loader = get_dataloaders(args)
354 print(f"Train set size: {len(train_loader.dataset)}")
355 print(f"Validation set size: {len(val_loader.dataset)}")
356
357 train_classes = train_loader.dataset.classes
358 val_classes = val_loader.dataset.classes
359
360 assert len(train_classes) == 100, f"Train classes={len(train_classes)} (expected 100)."
361 assert len(val_classes) == 100, f"Val classes={len(val_classes)} (expected 100)."
362
363 # Optional: ensure class names (and mapping) match between train/val
364 train_cti = train_loader.dataset.class_to_idx
365 val_cti = val_loader.dataset.class_to_idx
366 if train_cti != val_cti:
367     missing_in_val = sorted(set(train_cti) - set(val_cti))
```

```

368     missing_in_train = sorted(set(val_cti) - set(train_cti))
369     raise ValueError(
370         f"Class mapping mismatch.\n"
371         f"Missing in val: {missing_in_val}\n"
372         f"Missing in train: {missing_in_train}"
373     )
374     # (Optional) print a quick peek
375     print(f"✓ Class count OK (100/100). First 5 classes: {train_classes[:5]}")
376
377     # Get model
378     model = get_model(args)
379     total_params = sum(p.numel() for p in model.parameters())
380     print(f"Total parameters: {total_params:,}") # 86,567,656 (~86.6M)
381     model.to(device)
382
383     # Define loss function and optimizer
384     criterion = nn.CrossEntropyLoss()
385     # optimizer = optim.AdamW(model.parameters(), lr=args.lr, weight_decay=args.
386         weight_decay)
387     # SGD optimizer instead of AdamW
388     optimizer = optim.SGD(
389         model.parameters(),
390         lr=args.lr, # You might want to use a higher learning rate for SGD, like 0.01
391         momentum=0.9, # Adding momentum generally helps SGD converge better
392         weight_decay=args.weight_decay # Weight decay for regularization
393     )
394
395     # Initialize best accuracy
396     best_acc = 0.0
397
398     # some preparation for regularizer cancellation.
399     BASE_LR = args.lr
400     CANCEL_EPOCH = args.cancel_epoch # your request
401     WD_AFTER = 2e-4
402     BUMP_X = args.bump_x # 0.01 -> 0.02
403     ETA_MIN = 1e-4
404
405     scheduler = None
406     has_cancelled = False
407
408     # Training loop
409     for epoch in range(args.epochs):
410
411         # ---- resume safety guard ----
412         if epoch > CANCEL_EPOCH and args.use_regulation:
413             args.use_regulation = 0
414             print(f"[INFO]: Forcing use_regulation=0 at epoch {epoch+1} (resume safety)")
415
416         if (not has_cancelled) and (epoch == CANCEL_EPOCH) and args.use_regulation:
417             args.use_regulation = 0
418             has_cancelled = True
419             print(f"[INFO]: Cancel Regularizer at Epoch {epoch+1} (human-counted)")
420             print(f"[INFO]: Turning on weight decay and attaching cosine scheduler")
421
422             # mutate existing optimizer (no rebuild)
423             set_wd(optimizer, WD_AFTER)
424             mul_lr(optimizer, BUMP_X) # small bump to escape sharp basin

```



```

425     remaining = max(1, args.epochs - epoch - 1) # e.g., 300 - 150 - 1 = 149
426     scheduler = CosineAnnealingLR(optimizer, T_max=remaining, eta_min=ETA_MIN)
427
428     # sanity log
429     print("[LRs]", [round(g["lr"], 6) for g in optimizer.param_groups],
430           "[WDs]", [g["weight_decay"] for g in optimizer.param_groups])
431
432
433     train_loss, train_acc, sharpness, sharpness2, weights_norm, hessian,
434     hessian_first, hessian_second = train_one_epoch(model, train_loader,
435     criterion, optimizer, device, epoch, args)
436
437     # Validate
438     val_loss, val_acc = validate(model, val_loader, criterion, device, epoch)
439
440     if scheduler is not None:
441         scheduler.step()
442
443     # Save related metrics:
444     with torch.no_grad():
445         np.save(args.checkpoint_dir + "/sharpness_" + str(epoch), sharpness.detach().
446         cpu().numpy())
447         np.save(args.checkpoint_dir + "/sharpness2_" + str(epoch), sharpness2.detach
448         ().cpu().numpy())
449         np.save(args.checkpoint_dir + "/weights_norm_" + str(epoch), weights_norm.
450         detach().cpu().numpy())
451         np.save(args.checkpoint_dir + "/hessian_" + str(epoch), hessian.detach().cpu()
452         .numpy())
453         np.save(args.checkpoint_dir + "/hessian_first_" + str(epoch), hessian_first.
454         detach().cpu().numpy())
455         np.save(args.checkpoint_dir + "/hessian_second_" + str(epoch), hessian_second
456         .detach().cpu().numpy())
457         np.save(args.checkpoint_dir + "/train_acc_" + str(epoch), float(train_acc))
458         np.save(args.checkpoint_dir + "/train_loss_" + str(epoch), float(train_loss))
459         np.save(args.checkpoint_dir + "/val_loss_" + str(epoch), float(val_loss))
460         np.save(args.checkpoint_dir + "/val_acc_" + str(epoch), float(val_acc))
461
462     # Print metrics
463     print(f"Epoch_{epoch+1}/{args.epochs}_Train_Loss:_{train_loss:.4f},_Train_Acc:_
464     {train_acc:.2f}%,_Val_Loss:_{val_loss:.4f},_Val_Acc:_{val_acc:.2f}%")
465
466     print("-" * 50)
467     print(datetime.now())
468
469     # Save checkpoint if the model has the best accuracy
470     if val_acc > best_acc:
471         best_acc = val_acc
472
473     print(f"Training_complete!_Best_validation_accuracy:_{best_acc:.2f}%")
474     end_time = datetime.now()
475     print('Duration:_{}'.format(end_time - start_time))
476     # writer.close()
477
478 if __name__ == "__main__":
479     main()

```

4.5.2 run_script.sh

```

1 #!/bin/bash
2
3 readonly LEARNING_RATE=1e-02
4 readonly DATE=$(date +%m%d)
5 readonly SEED=42
6 readonly TRAINING_EPOCH=300
7 readonly CANCEL_EPOCH=150 # larger than the training epoch indicating no cancellation.
8 readonly BUMP_X=5.0
9 readonly BSZ=256 # batch size
10 readonly WEIGHTDECAY=0
11 readonly NWORKER=2
12 readonly PRETRAINED=0
13 readonly USE_REGR=1 # using regularizer
14 readonly HECOEFF=1e-2 # coefficient: lambda
15 readonly TAU=2.0
16 readonly WEIGHT_CAP=1
17 readonly CAP_TYPE="frob" # frob or spectral
18 readonly PHI_NORM=1
19 readonly F_MAX_NORM=150.0
20 readonly MODEL_TYPE="tiny"
21
22
23 nohup python -u train_vit.py \
24     --lr=${LEARNING_RATE} \
25     --model_type=${MODEL_TYPE} \
26     --training_date=${DATE} \
27     --random_seed=${SEED} \
28     --epochs=${TRAINING_EPOCH} \
29     --tau=${TAU} \
30     --weight_cap=${WEIGHT_CAP} \
31     --bump_x=${BUMP_X} \
32     --cap_type=${CAP_TYPE} \
33     --f_max_norm=${F_MAX_NORM} \
34     --cancel_epoch=${CANCEL_EPOCH} \
35     --batch_size=${BSZ} \
36     --pretrained=${PRETRAINED} \
37     --weight_decay=${WEIGHTDECAY} \
38     --use_regulation=${USE_REGR} \
39     --num_workers=${NWORKER} \
40     --phi_norm=${PHI_NORM} \
41     --cof_lambda=${HECOEFF} \
42     > Seed${SEED}_Date${DATE}_BSZ${BSZ}_PRETRAINED${PRETRAINED}_MD${MODEL_TYPE}_LR${LEARNING_RATE}_EPOCH${TRAINING_EPOCH}_CANECH${CANCEL_EPOCH}_BUMP${BUMP_X}_WD${WEIGHTDECAY}_REGR${USE_REGR}_PHIN${PHI_NORM}_LAMBDA${HECOEFF}_TAU${TAU}_WCAP${WEIGHT_CAP}_CAPT${CAP_TYPE}_FMAX${F_MAX_NORM}_trainlog.txt 2>&1 &

```

4.6 Relative Flatness: rf_bert/

4.6.1 bert_sst5.py

```

1 # bert_sst_sgd_fp32_eval_each_epoch.py
2 import os, random, argparse
3 os.environ["TOKENIZERS_PARALLELISM"] = "false"
4 print(f"Process_ID: {os.getpid()}")
5 import numpy as np
6 import torch
7 from torch.utils.data import DataLoader
8 from tqdm import tqdm
9 from datetime import datetime
10 from datasets import load_dataset
11 from transformers import (
12     BertConfig, BertForSequenceClassification,
13     BertTokenizerFast, DataCollatorWithPadding
14 )
15 from torch.optim.lr_scheduler import CosineAnnealingLR
16
17
18 def set_lr(opt, lr: float):
19     for g in opt.param_groups:
20         g["lr"] = lr
21
22 def mul_lr(opt, factor: float):
23     for g in opt.param_groups:
24         g["lr"] *= factor
25
26 def set_wd(opt, wd: float):
27     for g in opt.param_groups:
28         g["weight_decay"] = wd
29
30 def frobenius_cap(W, max_norm):
31     frob = torch.linalg.norm(W, ord='fro') # Frobenius norm
32     if frob > max_norm:
33         W.mul_(float(max_norm / frob))
34     return float(frob)
35
36 def spectral_cap(W, s_max=5.0, n_iter=1):
37     # Estimate sigma_max(W) with power iteration
38     u = torch.randn(W.size(0), device=W.device)
39     for _ in range(n_iter): # -12 iters usually enough
40         v = (W.t() @ u); v /= (v.norm() + 1e-12)
41         u = (W @ v); u /= (u.norm() + 1e-12)
42     sigma = torch.dot(u, W @ v) # estimated spectral norm
43     if sigma > s_max:
44         W.mul_(float(s_max / sigma))
45     return float(sigma)
46
47 def set_seed(seed=42):
48     random.seed(seed); np.random.seed(seed)
49     torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)
50     torch.backends.cudnn.deterministic = True
51     torch.backends.cudnn.benchmark = False
52
53 @torch.no_grad()
54 def eval_loop(model, dataloader, device, args, desc="Eval"):

```

```

55 model.eval()
56 total_loss = 0.0; correct = 0; total = 0
57 for batch in tqdm(dataloader, desc=desc, leave=False, disable=True):
58     allowed = {"input_ids", "attention_mask", "token_type_ids", "labels"}
59     batch = {k: v.to(device) for k, v in batch.items() if k in allowed}
60     outputs = model(**batch)
61     total_loss += outputs.loss.item() * batch["input_ids"].size(0)
62     preds = outputs.logits.argmax(dim=-1)
63     correct += (preds == batch["labels"]).sum().item()
64     total += batch["labels"].size(0)
65 return {"loss": total_loss / total, "acc": correct / total}
66
67 def train_one_epoch(model, dataloader, optimizer, device, args):
68     model.train()
69     run_loss = 0.0; correct = 0; total = 0
70     weights_norm_list, sharpness_list, sharpness2_list, hessian_first_list,
71     hessian_second_list, hessian_list = [], [], [], [], [], []
72     for batch in tqdm(dataloader, desc="Train", leave=False, disable=True):
73         allowed = {"input_ids", "attention_mask", "token_type_ids", "labels"}
74         batch = {k: v.to(device) for k, v in batch.items() if k in allowed}
75         optimizer.zero_grad(set_to_none=True)
76         outputs = model(**batch)
77         loss = outputs.loss
78         logits = outputs.logits
79         second_last_outputs = outputs.hidden_states[-1][:, 0, :] # CLS
80
81         if args.use_regulation:
82             if args.tau:
83                 probs = torch.softmax(logits / args.tau, dim=1)
84             else:
85                 probs = torch.softmax(logits, dim=1)
86             hessian_first = torch.sum(torch.mul(probs, 1 - probs), dim=1)
87             if args.phi_norm:
88                 second_last_outputs = second_last_outputs / (second_last_outputs.norm(dim=1, keepdim=True) + 1e-8)
89             hessian_second = torch.sum(torch.mul(second_last_outputs, second_last_outputs), dim=1)
90             hessian = torch.mul(hessian_first, hessian_second)
91             weights_norm = torch.linalg.norm(model.classifier.weight)
92             sharpness = weights_norm * hessian
93             sharpness2 = weights_norm * weights_norm * hessian
94
95             loss = loss - args.lmbd * torch.mean(sharpness2)
96         else:
97             with torch.no_grad():
98                 probs = torch.softmax(logits, dim=1)
99                 hessian_first = torch.sum(torch.mul(probs, 1 - probs), dim=1)
100                 if args.phi_norm:
101                     second_last_outputs = second_last_outputs / (second_last_outputs.norm(dim=1, keepdim=True) + 1e-8)
102                 hessian_second = torch.sum(torch.mul(second_last_outputs, second_last_outputs), dim=1)
103                 hessian = torch.mul(hessian_first, hessian_second)
104                 weights_norm = torch.linalg.norm(model.classifier.weight)
105                 sharpness = weights_norm * hessian
106                 sharpness2 = weights_norm * weights_norm * hessian
107
108     loss.backward()

```

```

108     optimizer.step()
109
110     if args.weight_cap:
111         with torch.no_grad():
112             if args.cap_type == "spectral":
113                 _ = spectral_cap(model.classifier.weight, s_max=5.0, n_iter=1)
114             elif args.cap_type == "frob":
115                 _ = frobenius_cap(model.classifier.weight, args.f_max_norm)
116
117     run_loss += loss.item() * batch["input_ids"].size(0)
118     preds = outputs.logits.argmax(dim=-1)
119     correct += (preds == batch["labels"]).sum().item()
120     total += batch["labels"].size(0)
121
122     # collect results here
123     with torch.no_grad():
124         weights_norm_list.append(weights_norm)
125         hessian_list.append(hessian)
126         hessian_first_list.append(hessian_first)
127         hessian_second_list.append(hessian_second)
128         sharpness_list.append(sharpness)
129         sharpness2_list.append(sharpness2)
130
131     return (
132         run_loss / total,
133         correct / total,
134         torch.mean(torch.cat(sharpness_list)),
135         torch.mean(torch.cat(sharpness2_list)),
136         torch.mean(torch.cat(hessian_list)),
137         torch.mean(torch.cat(hessian_first_list)),
138         torch.mean(torch.cat(hessian_second_list)),
139         torch.mean(torch.tensor(weights_norm_list))
140     )
141
142 def main():
143     ap = argparse.ArgumentParser()
144     ap.add_argument("--task", type=str, default="sst2", choices=["sst2", "sst5"])
145     # ap.add_argument("--model_name", type=str, default="huawei-noah/
146     TinyBERT_General_4L_312D")
147     ap.add_argument("--model_name", type=str, default="bert-base-uncased")
148     ap.add_argument("--batch_size", type=int, default=64)
149     ap.add_argument("--epochs", type=int, default=8)
150     ap.add_argument("--lr", type=float, default=6e-4)
151     ap.add_argument("--momentum", type=float, default=0.9)
152     ap.add_argument("--dropout", type=float, default=0.0, help="BERT_hidden/attn_dropout")
153
154     ap.add_argument("--max_length", type=int, default=128)
155     ap.add_argument("--seed", type=int, default=42)
156     ap.add_argument("--num_workers", type=int, default=2)
157     ap.add_argument("--output_dir", type=str, default=".")
158     # ---- regularizer / extras ----
159     ap.add_argument("--use_regulation", type=int, default=1)
160     ap.add_argument("--lmbd", type=float, default=1e-4)
161     ap.add_argument("--training_date", type=str, default="0211")
162     ap.add_argument("--weight_decay", type=float, default=0.0)
163     ap.add_argument("--phi_norm", type=int, default=1)
164     ap.add_argument("--tau", type=float, default=2.0)
165     ap.add_argument("--weight_cap", type=int, default=1)

```

```

164 ap.add_argument("--cap_type", type=str, default="frob")
165 ap.add_argument("--f_max_norm", type=float, default=80.0)
166 ap.add_argument("--head_only", type=int, default=0)
167 ap.add_argument("--train_subset", type=int, default=0, help="limit_training_set_size_
    (0=use_full)")
168 ap.add_argument("--bump_x", type=float, default=5.0)
169 ap.add_argument("--cancel_epoch", default=10000, type=int, help="which_epoch_to_
    cancel_the_regularization")
170
171 args = ap.parse_args()
172
173 set_seed(args.seed)
174 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
175 print(f"Device: {device}")
176
177 # create the output dir
178 args.output_dir = "BERT_{}_Seed{}_Date{}_EPOCH{}_CANCEL{}_BUMP{}_BSZ{}_TSZ{}_LR{}_HD
    {_}_DP{}_WD{}_REGR{}_PHIN{}_LAMBDA{}_TAU{}_WCAP{}_CAPT{}_FMAX{}".format(
179     args.task.upper(),
180     args.seed,
181     args.training_date,
182     args.epochs,
183     args.cancel_epoch,
184     args.bump_x,
185     args.batch_size,
186     args.train_subset,
187     args.lr,
188     args.head_only,
189     args.dropout,
190     args.weight_decay,
191     args.use_regulation,
192     args.phi_norm,
193     args.lmbd,
194     args.tau,
195     args.weight_cap,
196     args.cap_type,
197     args.f_max_norm
198 )
199 os.makedirs(args.output_dir, exist_ok=True)
200
201 print("-----Arguments-----")
202 for k, v in vars(args).items():
203     print(f"{k}: {v}")
204 print("-----")
205
206 # ----- Dataset: SST-2 (GLUE) or SST-5 (SetFit/sst5) -----
207 if args.task == "sst2":
208     ds = load_dataset("glue", "sst2")
209     TEXT_COL = "sentence"
210     LABEL_COL = "label"
211     num_labels = 2
212 else:
213     ds = load_dataset("SetFit/sst5") # confirmed correct dataset
214     TEXT_COL = "text"
215     LABEL_COL = "label"
216     num_labels = 5
217     # make a validation split if missing
218     if "validation" not in ds:

```

```

219         split = ds["train"].train_test_split(test_size=0.1, seed=args.seed)
220         ds["train"] = split["train"]
221         ds["validation"] = split["test"]
222
223     if args.train_subset > 0:
224         ds["train"] = ds["train"].select(range(args.train_subset))
225
226     # standardize label column name to 'labels'
227     if "labels" not in ds["train"].column_names:
228         ds["train"] = ds["train"].rename_column(LABEL_COL, "labels")
229     if "labels" not in ds["validation"].column_names:
230         ds["validation"] = ds["validation"].rename_column(LABEL_COL, "labels")
231
232     tokenizer = BertTokenizerFast.from_pretrained(args.model_name)
233
234     def preprocess(batch):
235         return tokenizer(batch[TEXT_COL], truncation=True, max_length=args.max_length,
236                           padding=False)
237
238     # remove all columns except 'labels' after tokenization
239     cols_to_remove_train = [c for c in ds["train"].column_names if c != "labels"]
240     cols_to_remove_val = [c for c in ds["validation"].column_names if c != "labels"]
241
242     train_ds = ds["train"].map(preprocess, batched=True, remove_columns=
243                               cols_to_remove_train)
244     val_ds = ds["validation"].map(preprocess, batched=True, remove_columns=
245                                  cols_to_remove_val)
246
247     collator = DataCollatorWithPadding(tokenizer=tokenizer)
248     train_loader = DataLoader(train_ds, batch_size=args.batch_size, shuffle=True,
249                              collate_fn=collator, num_workers=args.num_workers,
250                              pin_memory=True)
251     val_loader = DataLoader(val_ds, batch_size=args.batch_size, shuffle=False,
252                            collate_fn=collator, num_workers=args.num_workers,
253                            pin_memory=True)
254
255     # Model (num_labels via config; dropout configurable)
256     config = BertConfig.from_pretrained(
257         args.model_name,
258         num_labels=num_labels,
259         output_hidden_states=True,
260         hidden_dropout_prob=args.dropout,
261         attention_probs_dropout_prob=args.dropout
262     )
263     model = BertForSequenceClassification.from_pretrained(args.model_name, config=config)
264         .to(device)
265
266     # Optimizer (head-only or full FT)
267     if args.head_only:
268         print("tuning head")
269         for p in model.bert.parameters():
270             p.requires_grad = False
271         optimizer = torch.optim.SGD(
272             model.classifier.parameters(),
273             lr=args.lr, momentum=args.momentum, weight_decay=args.weight_decay
274         )
275     else:

```

```

271     optimizer = torch.optim.SGD(
272         model.parameters(),
273         lr=args.lr, momentum=args.momentum, weight_decay=args.weight_decay
274     )
275
276     # some preparation for regularizer cancellation.
277     BASE_LR      = args.lr
278     CANCEL_EPOCH = args.cancel_epoch    # your request
279     WD_AFTER     = 3e-4
280     BUMP_X       = args.bump_x          # 0.01 -> 0.02
281     ETA_MIN      = 1e-4
282
283     print("Preparation for regularizer cancellation:")
284     print(f"BASE_LR={BASE_LR}")
285     print(f"CANCEL_EPOCH={CANCEL_EPOCH}")
286     print(f"WD_AFTER={WD_AFTER}")
287     print(f"BUMP_X={BUMP_X}")
288     print(f"ETA_MIN={ETA_MIN}")
289
290     scheduler = None
291     has_cancelled = False
292     # Train + eval each epoch
293     for epoch in range(1, args.epochs + 1):
294         print(f"\nEpoch_{epoch}/{args.epochs}")
295
296         # ---- resume safety guard ----
297         if epoch > CANCEL_EPOCH and args.use_regulation:
298             args.use_regulation = 0
299             print(f"[INFO]: Forcing use_regulation=0 at epoch_{epoch+1} (resume safety)")
300
301         if (not has_cancelled) and (epoch == CANCEL_EPOCH) and args.use_regulation:
302             args.use_regulation = 0
303             has_cancelled = True
304             print(f"[INFO]: Cancel Regularizer at Epoch_{epoch+1} (human-counted)")
305             print(f"[INFO]: Turning on weight decay and attaching cosine scheduler")
306
307             # mutate existing optimizer (no rebuild)
308             set_wd(optimizer, WD_AFTER)
309             mul_lr(optimizer, BUMP_X) # small bump to escape sharp basin
310
311             remaining = max(1, args.epochs - epoch - 1) # e.g., 300 - 150 - 1 = 149
312             scheduler = CosineAnnealingLR(optimizer, T_max=remaining, eta_min=ETA_MIN)
313
314             # sanity log
315             print("[LRs]", [round(g["lr"], 6) for g in optimizer.param_groups],
316                   "[WDs]", [g["weight_decay"] for g in optimizer.param_groups])
317
318             (tr_loss, tr_acc, sharpness, sharpness2, hessian,
319              hessian_first, hessian_second, weights_norm) = train_one_epoch(
320                 model, train_loader, optimizer, device, args
321             )
322             va = eval_loop(model, val_loader, device, args, desc="Val")
323             print(f"Train_loss: {tr_loss:.4f} acc: {tr_acc:.4f}")
324             print(f"Val_loss: {va['loss']:.4f} acc: {va['acc']:.4f}")
325
326             if scheduler is not None:
327                 scheduler.step()
328
329     # save results:

```



```

329     np.save(args.output_dir + "/sharpness_" + str(epoch), sharpness.detach().cpu().
        numpy())
330     np.save(args.output_dir + "/sharpness2_" + str(epoch), sharpness2.detach().cpu().
        numpy())
331     np.save(args.output_dir + "/hessian_" + str(epoch), hessian.detach().cpu().numpy
        ())
332     np.save(args.output_dir + "/hessian_first_" + str(epoch), hessian_first.detach().
        cpu().numpy())
333     np.save(args.output_dir + "/hessian_second_" + str(epoch), hessian_second.detach
        ().cpu().numpy())
334     np.save(args.output_dir + "/weights_norm_" + str(epoch), weights_norm.numpy())
335     np.save(args.output_dir + "/train_acc_" + str(epoch), tr_acc)
336     np.save(args.output_dir + "/train_loss_" + str(epoch), tr_loss)
337     np.save(args.output_dir + "/val_loss_" + str(epoch), va['loss'])
338     np.save(args.output_dir + "/val_acc_" + str(epoch), va['acc'])
339     print("-" * 50)
340     print(datetime.now())
341
342 if __name__ == "__main__":
343     main()
344     print("Training done")

```

4.6.2 run_script.sh

```

1  #!/bin/bash
2
3  readonly LEARNING_RATE=1e-3
4  readonly DP=0.0
5  readonly DATE=$(date +%m%d)
6  readonly SEED=42
7  readonly TRAINING_EPOCH=50
8  readonly CANCEL_EPOCH=2000
9  readonly BSZ=32
10 readonly WEIGHTDECAY=0
11 readonly USE_REGR=0
12 readonly HECOEFF=0
13 readonly TAU=0
14 readonly WEIGHT_CAP=0
15 readonly CAP_TYPE="None" # frob or spectral
16 readonly PHI_NORM=0
17 readonly F_MAX_NORM=0
18 readonly HEAD_ONLY=0
19 readonly TASK="sst5"
20 readonly TSZ=0
21 readonly BUMPX=0
22
23
24 nohup python -u bert_sst5.py \
25     --lr=${LEARNING_RATE} \
26     --task=${TASK} \
27     --head_only=${HEAD_ONLY} \
28     --dropout=${DP} \
29     --training_date=${DATE} \
30     --seed=${SEED} \
31     --epochs=${TRAINING_EPOCH} \
32     --cancel_epoch=${CANCEL_EPOCH} \
33     --bump_x=${BUMPX} \
34     --tau=${TAU} \

```

```
35 --train_subset=${TSZ} \
36 --weight_cap=${WEIGHT_CAP} \
37 --cap_type=${CAP_TYPE} \
38 --f_max_norm=${F_MAX_NORM} \
39 --batch_size=${BSZ} \
40 --weight_decay=${WEIGHTDECAY} \
41 --use_regulation=${USE_REGR} \
42 --phi_norm=${PHI_NORM} \
43 --lmbd=${HECOEFF} \
44 > BERT_${TASK}_Seed${SEED}_Date${DATE}_EPOCH${TRAINING_EPOCH}_CANCEL${CANCEL_EPOCH}
   _BUMPX${BUMPX}_BSZ${BSZ}_TSZ${TSZ}_LR${LEARNING_RATE}_HD${HEAD_ONLY}_DP${DP}_WD${
   WEIGHTDECAY}_REGR${USE_REGR}_PHIN${PHI_NORM}_LMBD${HECOEFF}_TAU${TAU}_WCAP${
   WEIGHT_CAP}_CAPT${CAP_TYPE}_FMAX${F_MAX_NORM}_trainlog.txt 2>&1 &
```

4.7 Relative Flatness: rf_gpt2/

4.7.1 gpt_sst5.py

```

1 # gpt_sst_sgd_fp32_eval_each_epoch.py
2 import os, random, argparse
3 os.environ["TOKENIZERS_PARALLELISM"] = "false"
4 print(f"Process_ID: {os.getpid()}")
5 import numpy as np
6 import torch
7 from torch.utils.data import DataLoader
8 from tqdm import tqdm
9 from datetime import datetime
10 from datasets import load_dataset
11
12 from transformers import (
13     GPT2Config, GPT2ForSequenceClassification,
14     GPT2TokenizerFast, DataCollatorWithPadding
15 )
16
17 from torch.optim.lr_scheduler import CosineAnnealingLR
18
19
20 def set_lr(opt, lr: float):
21     for g in opt.param_groups:
22         g["lr"] = lr
23
24 def mul_lr(opt, factor: float):
25     for g in opt.param_groups:
26         g["lr"] *= factor
27
28 def set_wd(opt, wd: float):
29     for g in opt.param_groups:
30         g["weight_decay"] = wd
31
32 def frobenius_cap(W, max_norm):
33     frob = torch.linalg.norm(W, ord='fro') # Frobenius norm
34     if frob > max_norm:
35         W.mul_(float(max_norm / frob))
36     return float(frob)
37
38 def spectral_cap(W, s_max=5.0, n_iter=1):
39     # Estimate sigma_max(W) with power iteration
40     u = torch.randn(W.size(0), device=W.device)
41     for _ in range(n_iter): # -12 iters usually enough
42         v = (W.t() @ u); v /= (v.norm() + 1e-12)
43         u = (W @ v); u /= (u.norm() + 1e-12)
44     sigma = torch.dot(u, W @ v) # estimated spectral norm
45     if sigma > s_max:
46         W.mul_(float(s_max / sigma))
47     return float(sigma)
48
49 def set_seed(seed=42):
50     random.seed(seed); np.random.seed(seed)
51     torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)
52     torch.backends.cudnn.deterministic = True
53     torch.backends.cudnn.benchmark = False
54

```

```

55 @torch.no_grad()
56 def eval_loop(model, dataloader, device, args, desc="Eval"):
57     model.eval()
58     total_loss = 0.0; correct = 0; total = 0
59     for batch in tqdm(dataloader, desc=desc, leave=False, disable=True):
60         allowed = {"input_ids", "attention_mask", "labels"}
61         batch = {k: v.to(device) for k, v in batch.items() if k in allowed}
62         outputs = model(**batch)
63         total_loss += outputs.loss.item() * batch["input_ids"].size(0)
64         preds = outputs.logits.argmax(dim=-1)
65         correct += (preds == batch["labels"]).sum().item()
66         total += batch["labels"].size(0)
67     return {"loss": total_loss / total, "acc": correct / total}
68
69 def train_one_epoch(model, dataloader, optimizer, device, args):
70     model.train()
71     run_loss = 0.0; correct = 0; total = 0
72     weights_norm_list, sharpness_list, sharpness2_list, hessian_first_list,
73     hessian_second_list, hessian_list = [], [], [], [], [], []
74     for batch in tqdm(dataloader, desc="Train", leave=False, disable=True):
75         allowed = {"input_ids", "attention_mask", "labels"}
76         batch = {k: v.to(device) for k, v in batch.items() if k in allowed}
77         optimizer.zero_grad(set_to_none=True)
78         outputs = model(**batch)
79         loss = outputs.loss
80         logits = outputs.logits
81         # Get the index of last non-pad token per batch
82         last_token_indices = batch["attention_mask"].sum(dim=1) - 1
83         # Gather last meaningful hidden states
84         last_hidden_state = outputs.hidden_states[-1]
85         second_last_outputs = last_hidden_state[torch.arange(last_hidden_state.size(0)),
86             last_token_indices]
87
88     if args.use_regulation:
89         if args.tau:
90             probs = torch.softmax(logits / args.tau, dim=1)
91         else:
92             probs = torch.softmax(logits, dim=1)
93         hessian_first = torch.sum(torch.mul(probs, 1 - probs), dim=1)
94         if args.phi_norm:
95             second_last_outputs = second_last_outputs / (second_last_outputs.norm(dim=
96                 =1, keepdim=True) + 1e-8)
97             hessian_second = torch.sum(torch.mul(second_last_outputs, second_last_outputs
98                 ), dim=1)
99             hessian = torch.mul(hessian_first, hessian_second)
100             weights_norm = torch.linalg.norm(model.score.weight)
101             sharpness = weights_norm * hessian
102             sharpness2 = weights_norm * weights_norm * hessian
103
104             loss = loss - args.lmbd * torch.mean(sharpness2)
105     else:
106         with torch.no_grad():
107             probs = torch.softmax(logits, dim=1)
108             hessian_first = torch.sum(torch.mul(probs, 1 - probs), dim=1)
109             if args.phi_norm:
110                 second_last_outputs = second_last_outputs / (second_last_outputs.norm
111                     (dim=1, keepdim=True) + 1e-8)

```

```

108         hessian_second = torch.sum(torch.mul(second_last_outputs,
109                                             second_last_outputs), dim=1)
110         hessian = torch.mul(hessian_first, hessian_second)
111         weights_norm = torch.linalg.norm(model.score.weight)
112         sharpness = weights_norm * hessian
113         sharpness2 = weights_norm * weights_norm * hessian
114
115     loss.backward()
116     optimizer.step()
117
118     if args.weight_cap:
119         with torch.no_grad():
120             if args.cap_type == "spectral":
121                 _ = spectral_cap(model.score.weight, s_max=5.0, n_iter=1)
122             elif args.cap_type == "frob":
123                 _ = frobenius_cap(model.score.weight, args.f_max_norm)
124
125     run_loss += loss.item() * batch["input_ids"].size(0)
126     preds = outputs.logits.argmax(dim=-1)
127     correct += (preds == batch["labels"]).sum().item()
128     total += batch["labels"].size(0)
129
130     # collect results here
131     with torch.no_grad():
132         weights_norm_list.append(weights_norm)
133         hessian_list.append(hessian)
134         hessian_first_list.append(hessian_first)
135         hessian_second_list.append(hessian_second)
136         sharpness_list.append(sharpness)
137         sharpness2_list.append(sharpness2)
138
139     return (
140         run_loss / total,
141         correct / total,
142         torch.mean(torch.cat(sharpness_list)),
143         torch.mean(torch.cat(sharpness2_list)),
144         torch.mean(torch.cat(hessian_list)),
145         torch.mean(torch.cat(hessian_first_list)),
146         torch.mean(torch.cat(hessian_second_list)),
147         torch.mean(torch.tensor(weights_norm_list))
148     )
149
150 def main():
151     ap = argparse.ArgumentParser()
152     ap.add_argument("--task", type=str, default="sst5", choices=["sst2", "sst5"])
153     ap.add_argument("--model_name", type=str, default="distilgpt2")
154     ap.add_argument("--batch_size", type=int, default=64)
155     ap.add_argument("--epochs", type=int, default=8)
156     ap.add_argument("--lr", type=float, default=1e-3)
157     ap.add_argument("--momentum", type=float, default=0.9)
158     ap.add_argument("--dropout", type=float, default=0.0, help="hidden/attn_dropout")
159     ap.add_argument("--max_length", type=int, default=128)
160     ap.add_argument("--seed", type=int, default=42)
161     ap.add_argument("--num_workers", type=int, default=2)
162     ap.add_argument("--output_dir", type=str, default=".")
163     # ---- regularizer / extras ----
164     ap.add_argument("--use_regulation", type=int, default=1)
165     ap.add_argument("--lmbd", type=float, default=1e-4)

```

```

165 ap.add_argument("--training_date", type=str, default="0211")
166 ap.add_argument("--weight_decay", type=float, default=0.0)
167 ap.add_argument("--phi_norm", type=int, default=1)
168 ap.add_argument("--tau", type=float, default=2.0)
169 ap.add_argument("--weight_cap", type=int, default=1)
170 ap.add_argument("--cap_type", type=str, default="frob")
171 ap.add_argument("--f_max_norm", type=float, default=80.0)
172 ap.add_argument("--head_only", type=int, default=0)
173 ap.add_argument("--train_subset", type=int, default=0, help="limit_training_set_size_
    (0=use_full)")
174 ap.add_argument("--bump_x", type=float, default=5.0)
175 ap.add_argument("--cancel_epoch", default=10000, type=int, help="which_epoch_to_
    cancel_the_regularization")
176 args = ap.parse_args()
177
178 set_seed(args.seed)
179 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
180 print(f"Device: {device}")
181
182 # create the output dir
183 args.output_dir = "GPT_{}_Seed{}_Date{}_EPOCH{}_CANCEL{}_BUMP{}_BSZ{}_TSZ{}_LR{}_HD{}_
    _DP{}_WD{}_REGR{}_PHIN{}_LAMBDA{}_TAU{}_WCAP{}_CAPT{}_FMAX{}".format(
184     args.task.upper(),
185     args.seed,
186     args.training_date,
187     args.epochs,
188     args.cancel_epoch,
189     args.bump_x,
190     args.batch_size,
191     args.train_subset,
192     args.lr,
193     args.head_only,
194     args.dropout,
195     args.weight_decay,
196     args.use_regulation,
197     args.phi_norm,
198     args.lmbd,
199     args.tau,
200     args.weight_cap,
201     args.cap_type,
202     args.f_max_norm
203 )
204 os.makedirs(args.output_dir, exist_ok=True)
205
206 print("-----Arguments-----")
207 for k, v in vars(args).items():
208     print(f"{k}: {v}")
209 print("-----")
210
211 # ----- Dataset: SST-2 (GLUE) or SST-5 (SetFit/sst5) -----
212 if args.task == "sst2":
213     ds = load_dataset("glue", "sst2")
214     TEXT_COL = "sentence"
215     LABEL_COL = "label"
216     num_labels = 2
217 else:
218     ds = load_dataset("SetFit/sst5") # confirmed correct dataset
219     TEXT_COL = "text"

```

```

220 LABEL_COL = "label"
221 num_labels = 5
222 # make a validation split if missing
223 if "validation" not in ds:
224     split = ds["train"].train_test_split(test_size=0.1, seed=args.seed)
225     ds["train"] = split["train"]
226     ds["validation"] = split["test"]
227
228 if args.train_subset > 0:
229     ds["train"] = ds["train"].select(range(args.train_subset))
230
231 # standardize label column name to 'labels'
232 if "labels" not in ds["train"].column_names:
233     ds["train"] = ds["train"].rename_column(LABEL_COL, "labels")
234 if "labels" not in ds["validation"].column_names:
235     ds["validation"] = ds["validation"].rename_column(LABEL_COL, "labels")
236
237 tokenizer = GPT2TokenizerFast.from_pretrained(args.model_name)
238 if tokenizer.pad_token is None:
239     tokenizer.pad_token = tokenizer.eos_token
240 tokenizer.padding_side = "right"
241
242 def preprocess(batch):
243     return tokenizer(batch[TEXT_COL], truncation=True, max_length=args.max_length,
244                       padding=False)
245
246 # remove all columns except 'labels' after tokenization
247 cols_to_remove_train = [c for c in ds["train"].column_names if c != "labels"]
248 cols_to_remove_val = [c for c in ds["validation"].column_names if c != "labels"]
249
250 train_ds = ds["train"].map(preprocess, batched=True, remove_columns=
251                             cols_to_remove_train)
252 val_ds = ds["validation"].map(preprocess, batched=True, remove_columns=
253                               cols_to_remove_val)
254
255 collator = DataCollatorWithPadding(tokenizer=tokenizer)
256 train_loader = DataLoader(train_ds, batch_size=args.batch_size, shuffle=True,
257                           collate_fn=collator, num_workers=args.num_workers,
258                           pin_memory=True)
259 val_loader = DataLoader(val_ds, batch_size=args.batch_size, shuffle=False,
260                         collate_fn=collator, num_workers=args.num_workers,
261                         pin_memory=True)
262
263 # Model (num_labels via config; dropout configurable)
264
265 config = GPT2Config.from_pretrained(
266     args.model_name,
267     num_labels=num_labels,
268     output_hidden_states=True,
269     embd_pdrop=args.dropout,
270     attn_pdrop=args.dropout,
271     resid_pdrop=args.dropout
272 )
273 model = GPT2ForSequenceClassification.from_pretrained(args.model_name, config=config)
274 model.config.pad_token_id = tokenizer.pad_token_id
275 model.to(device)

```

```

273
274 # Optimizer (head-only or full FT)
275 if args.head_only:
276     print("tuning_head")
277     for p in model.transformer.parameters():
278         p.requires_grad = False
279     optimizer = torch.optim.SGD(
280         model.score.parameters(),
281         lr=args.lr, momentum=args.momentum, weight_decay=args.weight_decay
282     )
283 else:
284     optimizer = torch.optim.SGD(
285         model.parameters(),
286         lr=args.lr, momentum=args.momentum, weight_decay=args.weight_decay
287     )
288
289 # some preparation for regularizer cancellation.
290 BASE_LR = args.lr
291 CANCEL_EPOCH = args.cancel_epoch # your request
292 WD_AFTER = 1e-4
293 BUMP_X = args.bump_x # 0.01 -> 0.02
294 ETA_MIN = 5e-5
295
296 scheduler = None
297 has_cancelled = False
298
299 # Train + eval each epoch
300 for epoch in range(1, args.epochs + 1):
301     print(f"\nEpoch_{epoch}/{args.epochs}")
302     # ---- resume safety guard ----
303     if epoch > CANCEL_EPOCH and args.use_regulation:
304         args.use_regulation = 0
305         print(f"[INFO]:_Forcing_use_regulation=0_at_epoch_{epoch+1}_(_resume_safety)")
306
307     if (not has_cancelled) and (epoch == CANCEL_EPOCH) and args.use_regulation:
308         args.use_regulation = 0
309         has_cancelled = True
310         print(f"[INFO]:_Cancel_Regularizer_at_Epoch_{epoch+1}_(_human-counted)")
311         print(f"[INFO]:_Turning_on_weight_decay_and_attaching_cosine_scheduler")
312
313         # mutate existing optimizer (no rebuild)
314         set_wd(optimizer, WD_AFTER)
315         mul_lr(optimizer, BUMP_X) # small bump to escape sharp basin
316
317         remaining = max(1, args.epochs - epoch - 1) # e.g., 300 - 150 - 1 = 149
318         scheduler = CosineAnnealingLR(optimizer, T_max=remaining, eta_min=ETA_MIN)
319
320         # sanity log
321         print("[LRs]", [round(g["lr"], 6) for g in optimizer.param_groups],
322               "[WDs]", [g["weight_decay"] for g in optimizer.param_groups])
323
324     (tr_loss, tr_acc, sharpness, sharpness2, hessian,
325     hessian_first, hessian_second, weights_norm) = train_one_epoch(
326         model, train_loader, optimizer, device, args
327     )
328     va = eval_loop(model, val_loader, device, args, desc="Val")
329     print(f"_Train_loss_{tr_loss:.4f}_acc_{tr_acc:.4f}")
330     print(f"_Val_loss_{va['loss']:.4f}_acc_{va['acc']:.4f}")

```



```

331
332     if scheduler is not None:
333         scheduler.step()
334     # save results:
335     np.save(args.output_dir + "/sharpness_" + str(epoch), sharpness.detach().cpu().
336             numpy())
337     np.save(args.output_dir + "/sharpness2_" + str(epoch), sharpness2.detach().cpu().
338             numpy())
339     np.save(args.output_dir + "/hessian_" + str(epoch), hessian.detach().cpu().numpy
340             ())
341     np.save(args.output_dir + "/hessian_first_" + str(epoch), hessian_first.detach().
342             cpu().numpy())
343     np.save(args.output_dir + "/hessian_second_" + str(epoch), hessian_second.detach
344             ().cpu().numpy())
345     np.save(args.output_dir + "/weights_norm_" + str(epoch), weights_norm.numpy())
346     np.save(args.output_dir + "/train_acc_" + str(epoch), tr_acc)
347     np.save(args.output_dir + "/train_loss_" + str(epoch), tr_loss)
348     np.save(args.output_dir + "/val_loss_" + str(epoch), va['loss'])
349     np.save(args.output_dir + "/val_acc_" + str(epoch), va['acc'])
350     print("-" * 50)
351     print(datetime.now())
352
353 if __name__ == "__main__":
354     main()
355     print("Training done")

```

4.7.2 run_script.sh

```

1 #!/bin/bash
2
3 readonly LEARNING_RATE=1e-3
4 readonly DP=0.0
5 readonly DATE=$(date +%m%d)
6 readonly SEED=42
7 readonly TRAINING_EPOCH=300
8 readonly CANCEL_EPOCH=150
9 readonly BSZ=64
10 readonly WEIGHTDECAY=0
11 readonly USE_REGR=1
12 readonly HECOEFF=0.03
13 readonly TAU=2.0
14 readonly WEIGHT_CAP=1
15 readonly CAP_TYPE="frob" # frob or spectral
16 readonly PHI_NORM=1
17 readonly F_MAX_NORM=30.0
18 readonly HEAD_ONLY=0
19 readonly TASK="sst5"
20 readonly TSZ=0
21 readonly BUMPX=2.0
22
23 nohup python -u gpt_sst5.py \
24     --lr=${LEARNING_RATE} \
25     --task=${TASK} \
26     --head_only=${HEAD_ONLY} \
27     --dropout=${DP} \
28     --training_date=${DATE} \
29     --seed=${SEED} \
30     --epochs=${TRAINING_EPOCH} \

```

```
31 --cancel_epoch=${CANCEL_EPOCH} \  
32 --bump_x=${BUMPX} \  
33 --tau=${TAU} \  
34 --train_subset=${TSZ} \  
35 --weight_cap=${WEIGHT_CAP} \  
36 --cap_type=${CAP_TYPE} \  
37 --f_max_norm=${F_MAX_NORM} \  
38 --batch_size=${BSZ} \  
39 --weight_decay=${WEIGHTDECAY} \  
40 --use_regulation=${USE_REGR} \  
41 --phi_norm=${PHI_NORM} \  
42 --lmbd=${HECOEFF} \  
43 > GPT_${TASK}_Seed${SEED}_Date${DATE}_EPOCH${TRAINING_EPOCH}_CANCEL${CANCEL_EPOCH}  
    _BUMPX${BUMPX}_BSZ${BSZ}_TSZ${TSZ}_LR${LEARNING_RATE}_HD${HEAD_ONLY}_DP${DP}_WD${  
    WEIGHTDECAY}_REGR${USE_REGR}_PHIN${PHI_NORM}_LMBD${HECOEFF}_TAU${TAU}_WCAP${  
    WEIGHT_CAP}_CAPT${CAP_TYPE}_FMAX${F_MAX_NORM}_trainlog.txt 2>&1 &
```

Chapter 5

How to Run Each Experiment

5.1 Grokking Experiments

5.1.1 Training

Navigate to the grokking directory and run training:

```
1 cd grokking_nc_rf
2
3 python scripts/train.py \
4     --datadir ./data \
5     --logdir ./logs \
6     --operator "+" \
7     --train_data_pct 50 \
8     --actual_train_data_pct 25 \
9     --n_layers 2 \
10    --n_heads 4 \
11    --d_model 128 \
12    --dropout 0.0 \
13    --batchsize 512 \
14    --max_lr 1.0 \
15    --anneal_lr \
16    --weight_decay 0.0 \
17    --opt_type AdamW \
18    --max_epochs 100000
```

Key parameters:

- `train_data_pct`: total data percentage (e.g., 50)
- `actual_train_data_pct`: effectively seen per epoch (e.g., 25) – the gap creates the grokking phenomenon
- `operator`: arithmetic operator (+, -, *, /, s5, etc.)
- `weight_noise`: Gaussian noise std on weights (sharpness control)

5.1.2 Sharpness Computation

After training, compute Keskar sharpness on checkpoints:

```
1 python scripts/compute_sharpness.py \
2     --datadir ./data \
3     --logdir ./logs
```

5.1.3 Partial Metrics

Compute metrics at specific epochs:

```
1 python scripts/create_partial_metrics.py \  
2   --operator "+" \  
3   --ckpt_path ./ckpts/
```

5.1.4 Visualization

Generate multi-panel plots:

```
1 python scripts/visualize_metrics.py \  
2   --datadir ./data \  
3   --logdir ./logs
```

5.2 NC Experiments

5.2.1 Training

```
1 cd nc_experiments  
2 bash run_script.sh
```

The script launches training with:

- `use_pow=1`: uses NC cluster regularizer (maximises NC score)
- `lambda=1e-3`: regularisation coefficient
- `cancel_epoch=10000`: effectively no cancellation (pure NC suppression)
- `epochs=250, lr=0.01, optimizer=SGD`

The training stops regularisation at `cancel_epoch`. For the two-phase experiment, set `cancel_epoch` to 60 so the model undergoes forced anti-NC then free fine-tuning.

5.2.2 Output Files

Each run creates a directory with per-epoch NumPy arrays:

- `sharpness_N.npy, sharpness2_N.npy`
- `weights_norm_N.npy`
- `cluster_value_N.npy`
- `train_acc_N.npy, val_acc_N.npy`
- `train_loss_N.npy, val_loss_N.npy`

5.3 Relative Flatness: ResNet-18

```
1 cd rf_resnet18  
2  
3 # Train ResNet-18 with flatness regulariser (use_pow=0 -> sharpness2)  
4 bash run_script.sh
```

Key parameters in `run_script.sh`:

- `model=resnet18` (also: `efficientnetb0`, `efficientnet_v2`, `wideresnet-16-8`)
- `use_regr=1`: enables regularisation
- `use_pow=0`: uses flatness (sharpness²) regulariser vs NC
- `lambda=1e-2`: regularisation strength
- `cancel_epoch=200`: two-phase cancellation point
- `loss_func=CE` (also: `ramp`, `squared_hinge`, `logistic_hinge`, `leaky_ramp`)
- `weight_cap=1`, `cap_type=frob`: Frobenius norm constraint on classifier

After cancellation (`epoch > cancel_epoch`):

- Regulariser disabled (`use_regulation=0`)
- Weight decay set to 2×10^{-4}
- Learning rate bumped by $10\times$
- Cosine annealing scheduler attached

5.4 Relative Flatness: Vision Transformer

```
1 cd rf_vit
2 bash run_script.sh
```

- Dataset: ImageNet-100 (subset of ImageNet)
- Model: ViT-Tiny/16 (also supports `small`, `base`)
- `cancel_epoch=150`, `lambda=1e-2`
- `phi_norm=1`: normalises penultimate-layer representations
- `weight_cap=frob`, `f_max_norm=150.0`

5.5 Relative Flatness: BERT

```
1 cd rf_bert
2 # Baseline (no regulariser):
3 bash run_script.sh
4 # Edit run_script.sh: USE_REGR=1, HEC0EFF=0.03, etc.
```

- Dataset: SST-5 (Stanford Sentiment Treebank, 5 classes)
- Model: `bert-base-uncased` via HuggingFace transformers
- Penultimate layer: CLS token hidden state (index 0 of last hidden layer)
- Output head: `model.classifier.weight`

5.6 Relative Flatness: GPT-2

```
1 cd rf_gpt2
2 bash run_script.sh
```

- Dataset: SST-5 (same split as BERT)

- Model: `gpt2` via HuggingFace transformers
- Penultimate layer: last non-padding token hidden state
- Output head: `model.score.weight`
- `cancel_epoch=150, lambda=0.03, bump_x=2.0`

Chapter 6

Sharpness Metric Definition

All experiments share the same per-sample sharpness metric, computed on the final linear classifier layer:

$$\mathbf{p}_i = \text{softmax}(\mathbf{z}_i) \quad (6.1)$$

$$H_i^{(1)} = \sum_{k=1}^K p_{i,k}(1 - p_{i,k}) \quad (\text{probabilistic curvature}) \quad (6.2)$$

$$H_i^{(2)} = \|\phi_i\|_2^2 \quad (\text{representation norm}) \quad (6.3)$$

$$\mathcal{H}_i = H_i^{(1)} \cdot H_i^{(2)} \quad (6.4)$$

$$S_i^{(1)} = \|W_{\text{out}}\|_2 \cdot \mathcal{H}_i \quad (6.5)$$

$$S_i^{(2)} = \|W_{\text{out}}\|_2^2 \cdot \mathcal{H}_i \quad (6.6)$$

where ϕ_i are the penultimate-layer representations for sample i , and W_{out} is the classifier weight matrix.

The regulariser used in the flatness experiments is **sharpness2** = $S^{(2)}$, and the loss with regularisation is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} - \lambda \cdot \mathbb{E}[S^{(2)}]$$

6.1 Neural Collapse Metric

The NC cluster score measures within-class to between-class variance ratio:

$$\text{NC} = \frac{1}{K(K-1)} \sum_{i \neq j} \frac{\sigma_i^2 + \sigma_j^2}{2\|\mu_i - \mu_j\|_2^2}$$

where μ_k and σ_k^2 are the empirical mean and variance of penultimate-layer representations for class k . Lower values indicate stronger collapse.

Chapter 7

Generating Paper Figures

7.1 Figure 1: Grokking NC & Flatness

1. Run grokking training with varying `train_data_pct` (20, 25, 30, 35, 40, 45, 50).
2. Use `scripts/visualize_metrics.py` to generate multi-panel plots.
3. Alternatively, copy `read_loss_acc.ipynb` and `plot_grok_cluster.ipynb` into the results directory and execute.

7.2 Figure 2: NC Clustering (CIFAR-10)

1. Run `nc_experiments/run_script.sh` with `use_pow=1`, `lambda=1e-3`, `cancel_epoch=60`.
2. Use the saved `cluster_value_*.npy` arrays to plot NC score vs. epoch.

7.3 Figure 4: Relative Flatness (ResNet-18 & ViT)

1. Run `rf_resnet18/run_script.sh` with `use_pow=0`, `lambda=1e-2`, `cancel_epoch=200`.
2. Run `rf_vit/run_script.sh` with `lambda=1e-2`, `cancel_epoch=150`.
3. Use `relative_flatness_plot_results.ipynb` to visualise sharpness vs. validation accuracy across epochs.

7.4 Figures 16–17: BERT on SST-5

1. Run `rf_bert/run_script.sh` with varying `lambda` and `cancel_epoch`.
2. The notebook saves per-epoch sharpness, cluster values, and accuracies.

7.5 Figures 18–19: GPT-2 on SST-5

1. Run `rf_gpt2/run_script.sh` with varying `lambda` and `cancel_epoch`.
2. Same output format as BERT experiments.

Chapter 8

Code Overview by Function

8.1 Regulariser Cancellation Protocol

The following pseudo-code shows the two-phase training implemented across all experiments:

```
1 for epoch in range(epochs):
2     if epoch == cancel_epoch:
3         # Phase 1 -> Phase 2 transition
4         use_regulation = 0 # disable regulariser
5         optimizer.weight_decay = 2e-4 # turn on weight decay
6         optimizer.lr *= bump_x # bump LR to escape sharp basin
7         scheduler = CosineAnnealingLR(optimizer, T_max=remaining)
8
9     if use_regulation:
10        loss = ce_loss + lambda * sharpness2 # minimise flatness
11    else:
12        loss = ce_loss # free fine-tuning
13
14    loss.backward()
15    optimizer.step()
16
17    if weight_cap: # Optional: constrain classifier norm
18        if cap_type == "frob":
19            frobenius_cap(model.fc.weight, f_max_norm)
20        elif cap_type == "spectral":
21            spectral_cap(model.fc.weight, s_max=5.0)
22
23    # Save per-epoch metrics
24    np.save(f"sharpness2_{epoch}", sharpness2)
25    np.save(f"cluster_value_{epoch}", cluster_value)
26    np.save(f"val_acc_{epoch}", valid_epoch_acc)
```

8.2 Sharpness Computation (Per-Sample)

The online sharpness is computed during training without extra backward passes:

```
1 with torch.no_grad():
2     probs = torch.softmax(outputs, dim=1)
3     hessian_first = torch.sum(probs * (1 - probs), dim=1)
4
5     # penultimate-layer representations
6     hessian_second = torch.sum(phi ** 2, dim=1)
```

```
7
8 weights_norm = torch.linalg.norm(model.fc.weight)
9
10 hessian = hessian_first * hessian_second
11 sharpness = weights_norm * hessian      #  $S^{(1)}$ 
12 sharpness2 = weights_norm**2 * hessian  #  $S^{(2)}$ 
```

8.3 NC Cluster Score

```
1 def compute_cluster(train_repr, y_predict_train):
2     # Group representations by class
3     class_dict = {key: [] for key in torch.unique(y_predict_train)}
4     for repr_val, label in zip(train_repr, y_predict_train):
5         class_dict[label.item()].append(repr_val)
6
7     # Mean and variance per class
8     mean_var_dict = {}
9     for key, value in class_dict.items():
10         mean = torch.mean(torch.stack(value), dim=0)
11         var = torch.mean(torch.stack([
12             torch.linalg.norm(f - mean) ** 2 for f in value
13         ]))
14         mean_var_dict[key] = [mean, var]
15
16     # Pairwise NC score
17     scores = []
18     for c1 in class_dict.keys():
19         for c2 in class_dict.keys():
20             if c2 == c1: continue
21             mu1, var1 = mean_var_dict[c1]
22             mu2, var2 = mean_var_dict[c2]
23             score = (var1 + var2) / (2 * torch.linalg.norm(mu1 - mu2) ** 2)
24             scores.append(score)
25
26     return torch.mean(torch.stack(scores))
```

Chapter 9

Troubleshooting

9.1 Common Issues

PyTorch Lightning version conflict The grokking code uses `pytorch-lightning==1.2.10`. Newer versions require API changes. Install the exact version: `pip install pytorch-lightning==1.2.10`.

Python 3.11 compatibility The `requirements.txt` specifies `python==3.8` but Python 3.11 works with minor adjustments (e.g., replacing deprecated `pytorch_lightning` imports).

Out of memory Reduce `batch_size` in the `run_script.sh` files. For ViT, use `model_type=tiny` instead of `base`.

Missing datasets `mod` and `blobfile` packages are required for the grokking experiments. Install via `pip install mod blobfile`.

Sympy version Ensure `sympy` is installed for permutation group operations.

ImageNet-100 for ViT The dataset must be downloaded separately and pointed to via `--data_path` in `train_vit.py`.

9.2 Reproducibility Guarantees

- All random seeds are fixed (`torch.manual_seed`, `np.random.seed`, `random.seed`, `torch.backends.cudnn`).
- Hyperparameters are fully configurable via command-line arguments.
- Results are saved as NumPy arrays (per-epoch) and PyTorch checkpoints.
- CSV logs are generated by PyTorch Lightning for the grokking experiments.